

TABLE OF CONTENTS

TABLE OF CONTENTS	1
DETAILED CONTENTS	2
Preface	14
1. BEGINNING WITH FUNDAMENTALS	35
2. WORKING WITH TEXT	72
3. NETWORK ANALYSIS AND MANAGEMENT	82
4. SOFTWARE INSTALLATION AND REMOVAL	94
5. MANAGING PERMISSIONS FOR FILES AND DIRECTORIES	108
6. OVERSEEING SYSTEM PROCESSES	121
7. HANDLING USER ENVIRONMENT VARIABLES	132
8. INTRODUCTION TO BASH SCRIPTING	145
9. TECHNIQUES IN DATA COMPRESSION AND ARCHIVING	155
10. MANAGING FILESYSTEMS AND STORAGE DEVICES	169
11. EXPLORING THE SYSTEM LOGGING MECHANISM	181
12. MANAGING AND EXPLOITING SERVICES	189
13. ENHANCING SECURITY AND ANONYMITY	214
14. WIRELESS NETWORK ANALYSIS AND INSPECTION	230
15. HANDLING THE LINUX KERNEL AND KERNEL MODULES	246
16. TASK AUTOMATION WITH JOB SCHEDULING	256

Conclusion.....	269
-----------------	-----

DETAILED CONTENTS

TABLE OF CONTENTS	1
DETAILED CONTENTS.....	2
Preface.....	1
Overview of This Guide.....	15
The Preference for Linux in Hacking	18
How to Install VirtualBox	21
1. BEGINNING WITH FUNDAMENTALS	35
Introductory Terms and Concepts.....	36
Exploring Kali Linux.....	38
Understanding the Terminal	39
Navigating the Linux Filesystem.....	40
Essential Linux Commands	41
Locating Your Current Directory with pwd	41
Identifying Your User with whoami.....	42
Moving Through the Linux Filesystem	44
Changing Directories with cd.....	44
List Directory Contents with ls.....	46

Accessing Help Resources	54
Using Manual Pages with man	55
Techniques for Locating Files and Directories.....	56
Searching with locate	56
Finding Executables with whereis	57
Locating Binaries in the PATH Variable with which	58
Advanced Searches with find	59
QUICK LOOK AT WILDCARDS.....	61
Sorting and Filtering with grep.....	63
Handling Files and Directories.....	64
Creating Files with cat	64
File Creation with touch	66
Directory Creation	66
Copying Files.....	67
Renaming Files.....	67
Deleting Files.....	68
Deleting Directories.....	69
Exploration and Practice.....	70
Practice Tasks	70
2. WORKING WITH TEXT	72

Techniques for Viewing Files.....	73
Using Head Command.....	74
Using Tail Command	74
Line Numbering with nl	75
Text Filtering with grep	75
Text Replacement with sed	75
Paging Through Files with more and less	77
Managing Output with more	77
Advanced Display and Filtering with less	77
Text Replacement with sed	78
Paging Through Files with more and less	79
Managing Output with more	79
Advanced Display and Filtering with less	79
Challenge: Combine grep, nl, and sed	80
Chapter Summary	81
Practice Exercises.....	81
3. NETWORK ANALYSIS AND MANAGEMENT	82
Network Inspection with ifconfig.....	82
Examining Wi-Fi Devices with iwconfig	84
Modifying Network Information.....	85

IP Address Modification	85
Adjusting Network Mask and Broadcast Address	86
MAC Address Spoofing	86
Acquiring New IP Addresses via DHCP	87
Handling Domain Name System	88
Exploring DNS with dig	89
Switching DNS Servers	90
Custom IP Address Mapping.....	92
Chapter Summary	93
Practice Exercises.....	94
4. SOFTWARE INSTALLATION AND REMOVAL	94
Software Packages and Dependencies.....	94
Using apt for Software Management	95
Locating a Software Package.....	95
Installing New Software.....	96
Uninstalling Existing Software.....	97
Regular Updates of Software Packages.....	98
Upgrading Your System's Software.....	99
Enhancing sources.list with Additional Repositories	100
Installing Applications via GUI.....	103

Deploying Software from git Repositories	105
Chapter Summary	106
Practice Exercises.....	107
5. MANAGING PERMISSIONS FOR FILES AND DIRECTORIES.....	108
User Types in Linux.....	109
Setting Access Permissions.....	109
Assigning Ownership to a User	110
Group Ownership Assignment.....	110
Reviewing Current Permissions.....	111
Modifying Access Rights	112
Using Numerical Codes for Permission Changes	112
Permission Adjustments with UGO	113
Root Execution Permissions for New Tools.....	114
Default Permission Settings with Masks	115
Extra Permissions	116
Temporary Root Access with SUID.....	117
Group Permission with SGID	117
Sticky Bit Explained.....	118
Special Permissions in Hacking Context	119
Chapter Summary	120

Practice Exercises.....	121
6. OVERSEEING SYSTEM PROCESSES	121
Exploring Active Processes	122
Isolating Processes by Name	124
Identifying Resource-Intensive Processes with top	125
Process Control Techniques	126
Modifying Process Priority with nice	126
Terminating Processes.....	128
Running Background Processes.....	129
Bringing Processes to the Foreground	129
Task Scheduling in Linux	129
Chapter Summary	132
Practice Exercises.....	132
7. HANDLING USER ENVIRONMENT VARIABLES.....	132
Environment Variables Basics.....	133
Altering Environment Variables.....	133
Viewing the Entire Environment Variable List	134
Searching Specific Environment Variables	135
Changing Environment Variables	135
Permanent Modification of Variable Values	136

Customizing Your Shell Prompt.....	137
Modifying the PATH.....	140
Correctly Expanding the PATH Variable	141
Replacing The Path Variable.....	141
Creating a User-Defined Variable	142
Saving Environment Variables.....	142
Chapter Summary	143
Practice Exercises.....	143
8. INTRODUCTION TO BASH SCRIPTING.....	145
Basics of Bash Programming.....	115
Crafting Your Initial Script: "Hello, SecurityAiLab!"	115
Setting Permission for Script Execution.....	116
Executing HelloSecurityAiLab.....	116
Script Enhancement with Variables and User Interaction.....	116
Building a Basic Port Scanner Script	118
The Objective of Our Script.....	119
Developing a Basic Port Scanner	120
Enhancing the Port Scanner for MySQL	121
Essential Bash Commands for Scripting.....	122
Chapter Summary	125

Practice Exercises.....	125
9. TECHNIQUES IN DATA COMPRESSION AND ARCHIVING	155
Understanding Data Compression	155
Grouping Files with tar	156
Techniques for File Compression.....	160
Using gzip for Compression	160
File Compression with bzip2	161
The compress Method.....	162
Creating Exact Copies of Storage Devices	163
Chapter Summary	166
Practice Exercises.....	166
10. MANAGING FILESYSTEMS AND STORAGE DEVICES.....	169
Overview of /dev Directory	170
Representation of Storage Devices in Linux	171
Understanding Drive Partitions.....	172
Differentiating Character and Block Devices	174
Using lsblk for Device Information	175
Procedures for Mounting and Unmounting.....	176
Manual Mounting of Storage Devices	176
Safe Unmounting with umount.....	177

Monitoring the Filesystem Health	177
Information on Mounted Disks	178
Disk Error Checking.....	179
Chapter Summary	180
Practice Exercises.....	180
11. EXPLORING THE SYSTEM LOGGING MECHANISM	181
Working with rsyslog Daemon.....	181
Configuring rsyslog.....	182
Understanding rsyslog Rules	182
Log Management with logrotate	184
Techniques for Remaining Stealthy.....	186
Removing Log Traces	187
Disabling System Logging	188
Chapter Summary	189
Practice Exercises.....	189
12. MANAGING AND EXPLOITING SERVICES.....	189
Service Management: Start, Stop, and Restart.....	190
Setting Up an HTTP Server with Apache	191
Initial Setup with Apache	191
Customizing index.html	192

Basic HTML Usage	193
Observing Server Response.....	194
OpenSSH and the Raspberry Spy Pi	194
Configuring a Raspberry Pi for Surveillance.....	195
Preparing the Raspberry Spy Pi	195
Camera Configuration.....	197
Initiating Surveillance.....	199
MySQL Information Extraction	200
Initiating MySQL.....	201
MySQL History	202
Interacting with MySQL Databases	202
Securing MySQL with a Password.....	203
Remote Database Access Techniques.....	205
Connecting to a Database	206
Understanding Database Structures	207
Data Analysis in MySQL.....	209
Using PostgreSQL with Metasploit.....	210
Chapter Summary	213
Practice Exercises.....	213
13. ENHANCING SECURITY AND ANONYMITY	214

Internet Traces and Privacy Concerns	214
Introduction to Tor and Its Mechanism	216
Operational Dynamics of Tor	216
Tor-related Security Aspects	218
Utilizing Proxy Servers	218
Configuring Proxy Settings	220
Exploring Additional Proxy Options	223
Adding More Proxies	224
Dynamic Chaining	224
Random Chaining	225
Proxy Security Considerations	226
Implementing Virtual Private Networks	227
The Concept of Encrypted Email	228
Chapter Summary	229
Practice Exercises	229
14. WIRELESS NETWORK ANALYSIS AND INSPECTION	230
Basics of Wi-Fi Networking	231
Essential Wireless Commands	232
Wi-Fi Recon with aircrack-ng	235
Exploring Bluetooth Connectivity	239

The Functioning of Bluetooth	239
Bluetooth Scanning Techniques.....	240
Scanning for Bluetooth Devices with hcitool	241
Scanning for Services with sdptool	243
Seeing Whether the Devices Are Reachable with l2ping.....	244
Chapter Summary	245
Practice Exercises.....	245
15. HANDLING THE LINUX KERNEL AND KERNEL MODULES	246
Introduction to Kernel Modules	247
Verifying Your Kernel Version.....	248
Kernel Adjustments with sysctl	249
Working with Kernel Modules.....	251
Gathering Module Info with modinfo.....	252
Module Management with modprobe	253
Direct Kernel Module Insertion and Removal	254
Chapter Summary	255
Practice Exercises.....	255
16. TASK AUTOMATION WITH JOB SCHEDULING.....	256
Fundamentals of Automated Scheduling.....	256
Setting Up Automated Backup Tasks.....	260

Scheduling with crontab for MySQLscanner	262
Efficient crontab Shortcuts	264
Using rc Scripts for Startup Jobs	264
Understanding Linux Runlevels	265
Integrating Services into rc.d.....	265
GUI Methods for Bootup Service Addition.....	267
Chapter Summary	268
Practice Exercises.....	268
Conclusion	269

Author Note

Thank you for purchasing this book. This is probably the best decision you've made. Here you'll learn hundreds of concepts while not overwhelming you anywhere. So buckle up and make sure you follow the guide daily.

Before we start I'd like to mention that if you find any section which seems hard, confusing, or anything else which you don't like please just email me at clh@dobrilab.com and I'll update that section within 3-7 days depending on how big it is. My goal here is to help you as much as possible, so don't view this as a "created and done" type of guide. I'm willing to update and improve it per your requests with the sole purpose to help you get in cybersecurity.

With that being said, let's start.

Preface

Hacking is a respected profession in the IT field, requiring a deep and thorough understanding of various IT concepts and technologies. One of the essential skills for anyone interested in hacking is Linux. If you are serious about starting a career in hacking and information security, dedicating time and effort to learn and understand Linux is crucial.

This book is designed for beginners who are just starting to start their journey in hacking, cybersecurity, and penetration testing. It's not intended for experienced hackers or Linux administrators but serves as a starting point for newcomers. We will begin with the basics of Linux and move on to introductory scripting in both Bash. Where applicable, I will include examples from hacking to help illustrate Linux principles and additional techniques.

In the preface, we will discuss the growing importance of ethical hacking in information security. Additionally, I will walk you through the process of setting up a virtual machine to install Kali Linux on your system without affecting your current operating system.

Overview of This Guide

Initial Chapters Overview

Chapter 1: Getting Started with Linux

- Become familiar with the file system and the terminal.
- Learn some basic commands.

Chapter 2: Text Manipulation

- Learn how to manipulate text to locate, inspect, and modify software and files.

Chapter 3: Network Management

- Learn network management.

- Scan for networks, gather information on connections, and disguise your network and DNS details.

Chapter 4: Software Management

- Learn to add, remove, and update software.
- Maintain your system efficiently.

Chapter 5: File and Directory Permissions

- Adjust file and directory permissions to regulate access.
- Explore techniques for escalating privileges.

Chapter 6: Service Management

- Manage services, including starting and stopping processes and distributing resources for enhanced control.

Chapter 7: Environment Variables

- Optimize performance and convenience by managing environment variables.
- Locate and filter variables, modify your PATH variable, and create new environment variables.

Chapter 8: Bash Scripting

- Introduction to bash scripting, an indispensable skill for any serious hacker.
- Construct a script to search for target ports for potential infiltration.

Chapters 9 and 10: File System Management

- Learn to compress and archive files to maintain a tidy system.
- Copy entire storage devices and gather information on files and connected disks.

Advanced Chapters

Chapter 11: Logging System

- Employ and manipulate the logging system to gather intelligence on a target's activities and conceal your own movements.

Chapter 12: Exploiting Linux Services

- Exploit three core Linux services: Apache web server, OpenSSH, and MySQL.
- Set up a web server, create a remote video surveillance system, and learn about databases and their vulnerabilities.

Chapter 13: Security and Anonymity

- Learn to stay secure and anonymous using proxy servers, the Tor network, VPNs, and encrypted email.

Chapter 14: Wireless Networks

- Master basic networking commands, crack Wi-Fi access points, and detect and connect to Bluetooth signals.

Chapter 15: Deep Dive into Linux

- Gain a high-level view of how the kernel operates and how its drivers can be manipulated to deliver malicious software.

Chapter 16: Scheduling Skills

- Learn essential scheduling skills to automate your hacking scripts.

Understanding Ethical Hacking

With the growth of the information security field, ethical hacking, also known as white hat hacking, has surged in importance. Ethical hacking involves attempting to breach and exploit a system to identify its vulnerabilities and fortify it. There are two primary segments of ethical hacking: penetration testing for legitimate information security firms and working for national military or intelligence agencies. Both areas are rapidly expanding with strong demand.

The Role of Penetration Testing

As organizations become more security-conscious and the costs of security breaches rise, many large organizations are now outsourcing security services, including penetration testing. Penetration testing is essentially a sanctioned, commissioned hack to demonstrate the vulnerabilities of a company's network and systems. This process typically starts with a vulnerability assessment to identify potential vulnerabilities, which often includes false positives. The role of the penetration tester is to try to hack these vulnerabilities to determine if they are real and if the organization should invest in mitigating them.

Use in Military and Intelligence Operations

Cyber espionage and cyber warfare are now common practices among nations. Hacking plays a crucial role in military and intelligence-gathering activities. For instance, gaining access to an adversary's war plans and disabling their infrastructure can be pivotal in modern warfare. Hackers thus become key elements in their nation's defense strategy.

The Preference for Linux in Hacking

Open-Source Nature of Linux

Unlike Windows, Linux is open source, meaning the source code is accessible and modifiable. This is crucial for making a system operate in ways it wasn't originally designed to.

Transparency of Linux Systems

Linux is completely transparent, allowing users to see and manipulate all its working components, unlike Windows, which obscures much of its inner workings.

Detailed Control Offered by Linux

Linux offers granular control over the system, allowing almost infinite customization through the terminal.

Linux as the Primary Platform for Hacking Tools

Over 90 percent of hacking tools are designed for Linux. While some tools are available for Windows, they often lack the full capabilities of their Linux counterparts.

Dominance of Linux/Unix in the Future

Linux/Unix systems dominate web servers due to their stability and reliability. They are also prevalent in embedded systems, virtualization, and mobile devices. With the growth of these sectors, the future of IT seems to lie with Linux/Unix systems.

Steps for Downloading Kali Linux

Before we dive in, you need to download and install Kali Linux on your computer. We will be using this Linux distribution throughout the book. Kali Linux, developed by Offensive Security, is a specialized operating system for hacking built on the Debian distribution. It comes equipped with a comprehensive suite of hacking tools.

There are a few different ways to download Kali. The preferred one is through Virtual Box because that's most secure for doing hacking tests. But there are faster and easier (recommend you follow both options so that you know more and have options)

For the second option, please watch the video here. Initially it's from the Command Line Hacker video course, but it'll be useful for you too, so I've included it as a hidden bonus into your purchase:

<https://www.dobrilab.com/community/c/command-line-hacker/quizes/download-linux>

(if you cannot access that link use this link: <https://youtu.be/LuyspQC6oNU>)

You can download Kali Linux from kali.org. From the homepage, click the "Downloads" link at the top of the page, then select Installer Images not Virtual Machine version.

You have 3 options, you want to select either 64-bit or 32-bit.

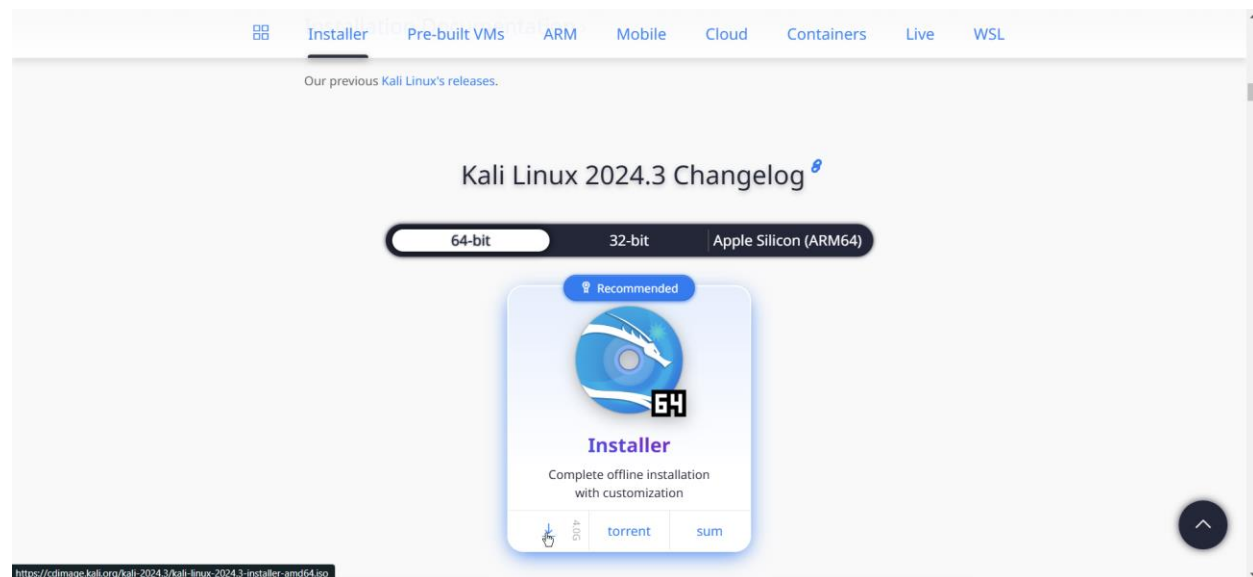
To find which one on Windows, follow these steps:

Open the Settings. Select System. Select About. Under Device specifications and System type, check for 32-bit or 64-bit Windows.

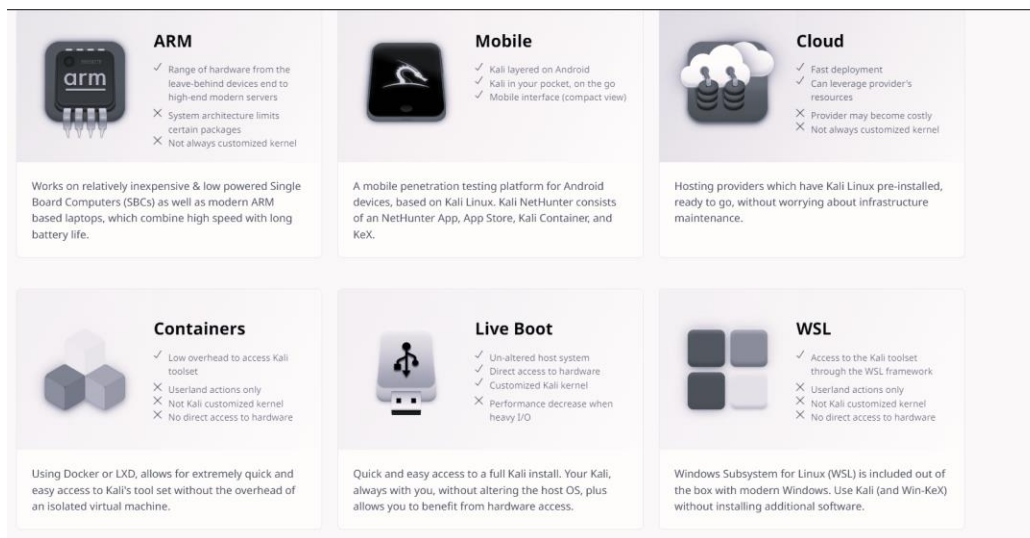
To find which one on Mac, follow these steps:

Go to the Apple menu and choose About This Mac. Below the computer model name, you'll see your processor. If the processor is an Intel Core Solo or Intel Core Duo, it's 32-bit only anything else is 64-bit.

Then just click the download icon. Not the Torrent or the "sum" - you'll now find a .iso file in your downloads folder, this is the operating system image that you'll then put in Virtual Box.



There are also versions for other types of CPUs, like the widely used ARM architecture found in many mobile devices. If you are using a Raspberry Pi, tablet, or other mobile device (phone users will likely prefer Kali NetHunter), be sure to download and install the ARM architecture version of Kali by scrolling down to Download ARM Images and clicking Kali ARM Images.



You've downloaded Kali, but before installing anything, let's discuss virtual machines. Generally, for beginners, installing Kali into a virtual machine is the best approach for learning and practice.

Introduction to Virtual Machines

Virtual machine (VM) technology allows you to run multiple operating systems on a single piece of hardware, such as your laptop or desktop. This means you can continue to use the Windows or MacOS operating system you are familiar with while running a virtual machine of Kali Linux inside that operating system. You don't need to overwrite your existing OS to learn Linux.

There are numerous virtual machine applications available from VMware, Oracle, Microsoft, and others. All are excellent, but here I will guide you through downloading and installing Oracle's free VirtualBox.

How to Install VirtualBox

You can download VirtualBox at <https://www.virtualbox.org/>, as illustrated in Image 1. Click the Downloads link in the left menu, and select the VirtualBox package for your computer's current operating system, which will host the VirtualBox VM. Ensure you download the latest version.

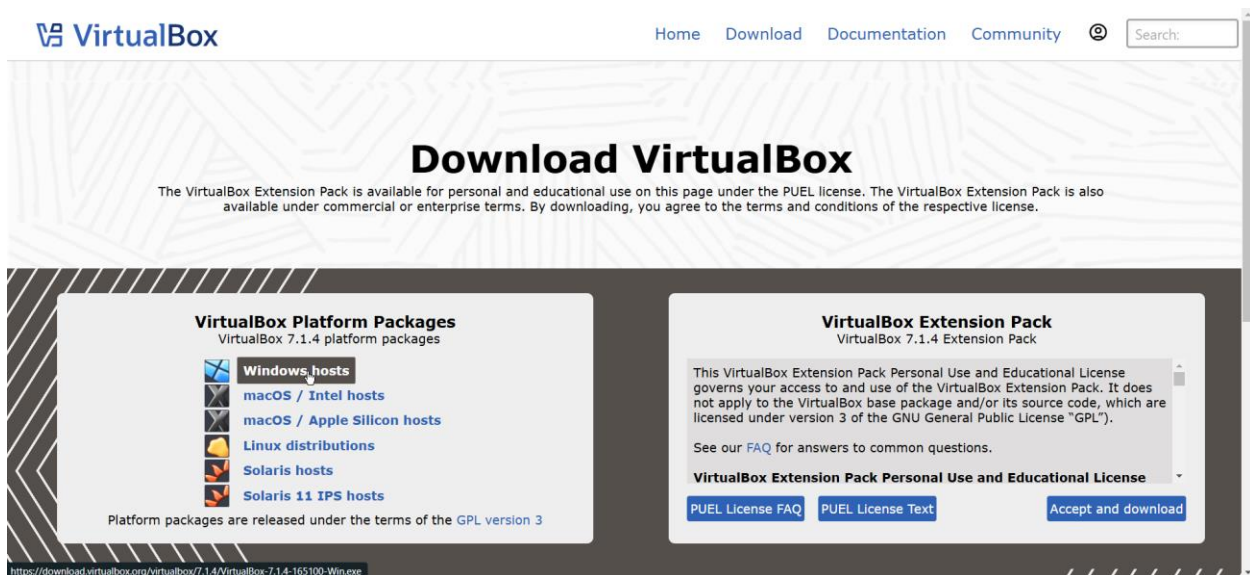
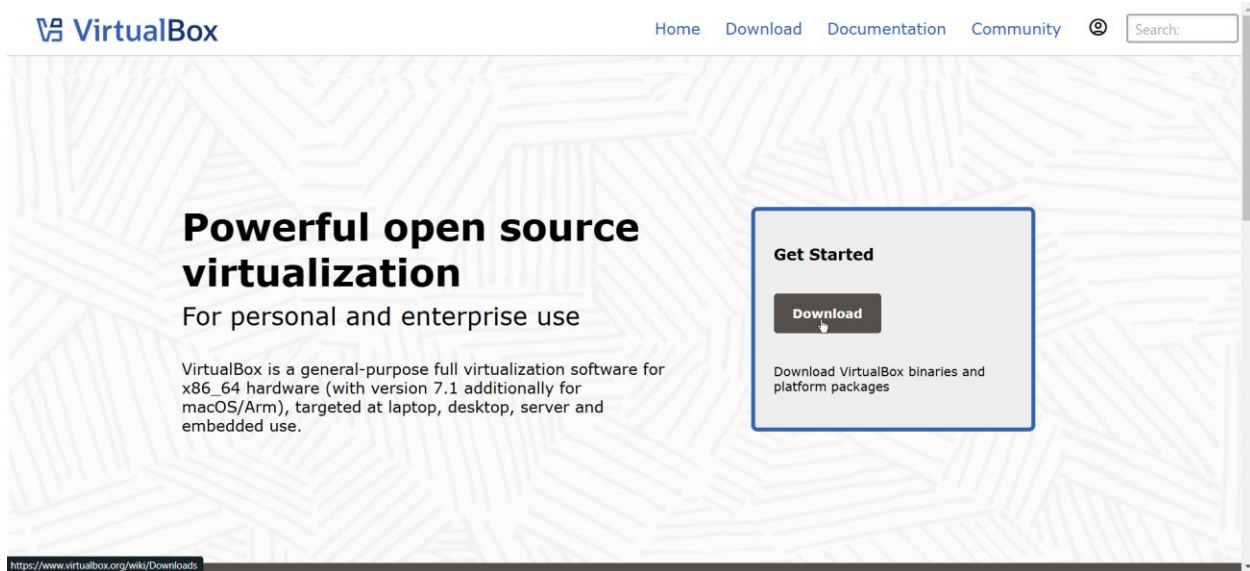


Image 1: VirtualBox home page

When the download is complete, click the setup file, and the familiar Setup Wizard, as shown in Image 2, will greet you.



Image 2: The Setup Wizard dialog

Click **Next**, and you should arrive at the Custom Setup screen, as in Image 3.

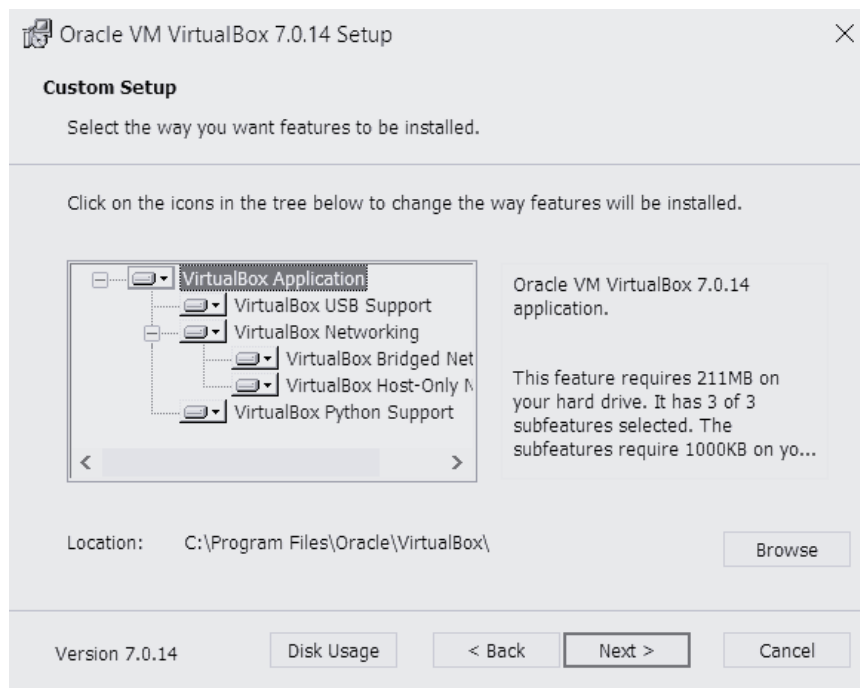


Image 3: The Custom Setup dialog

From this screen, simply click **Next**. Continue clicking **Next** until you reach the Network Interfaces warning screen, then click **Yes**.

Click **Install** to begin the installation process. During this process, you will likely be prompted several times to install device software. These are the virtual networking devices necessary for your virtual machines to communicate. Click **Install** for each prompt.

When the installation is complete, click **Finish**.

Setting Up a Virtual Environment

Now let's begin with setting up your virtual machine. After installing VirtualBox, open it to view the VirtualBox Manager, as seen in Image 4.

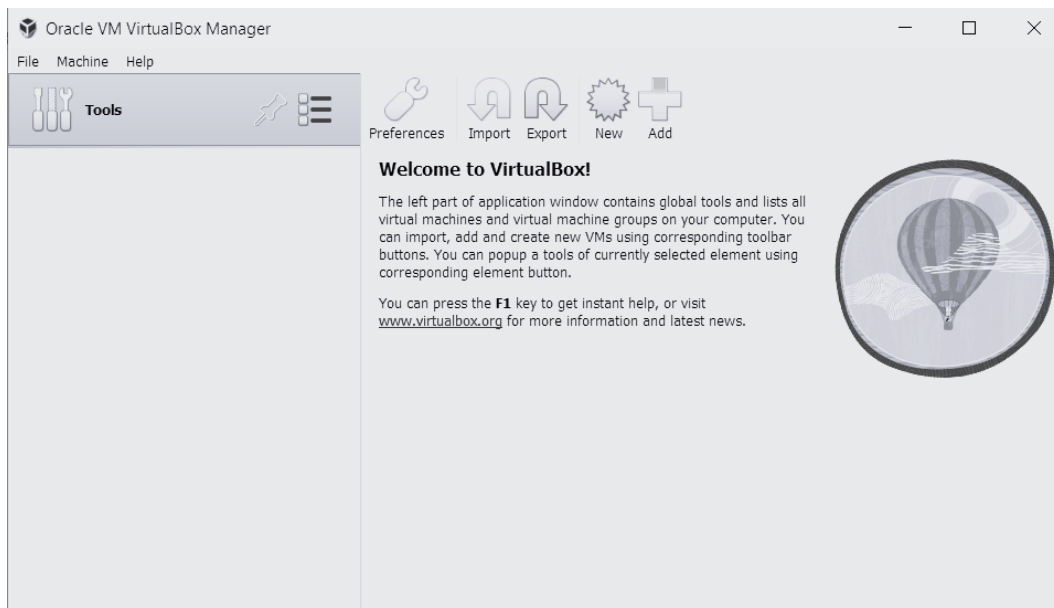


Image 4: The VirtualBox Manager

Click '**New**' in the upper-left corner to create a new virtual machine for Kali Linux. This opens the Create Virtual Machine dialog, shown in Image 5.

Name your machine (I used 'Kali') and select 'Linux' from the Type drop-down menu. Choose 'Debian (64-bit)' from the Version drop-down menu (select 'Debian 32-bit' if

you're using the 32-bit version of Kali). Click 'Next' to proceed to the memory allocation screen (Image 6).

Keep in mind that these settings might slightly vary depending on the version of VirtualBox but you'll always need to just configure storage, give a name to your VM, and select it's type (Debian).

If you get lost anywhere in the steps below or in the book simply either ask <https://chatgpt.com/>, search in YouTube, or just email us at command-line-hacker@securityailab.com – although that'll rarely happen.

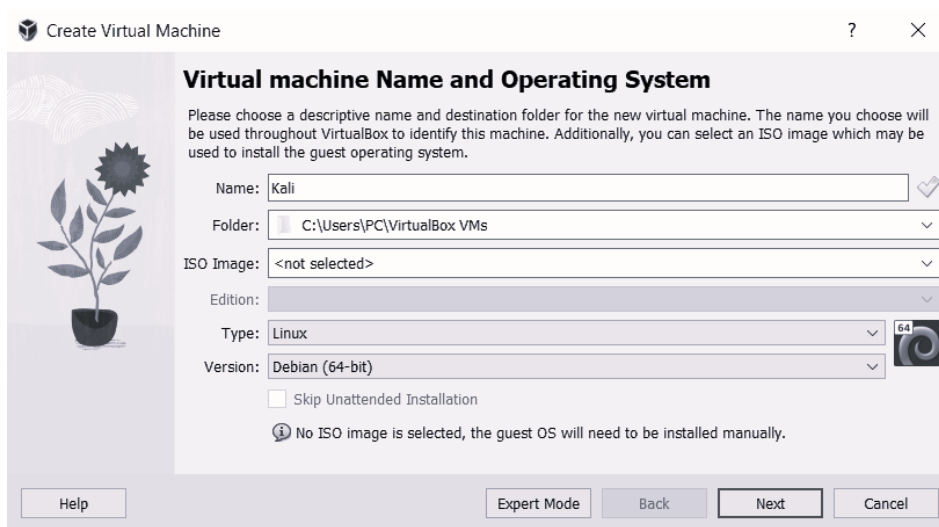


Image 5: The Create Virtual Machine dialog

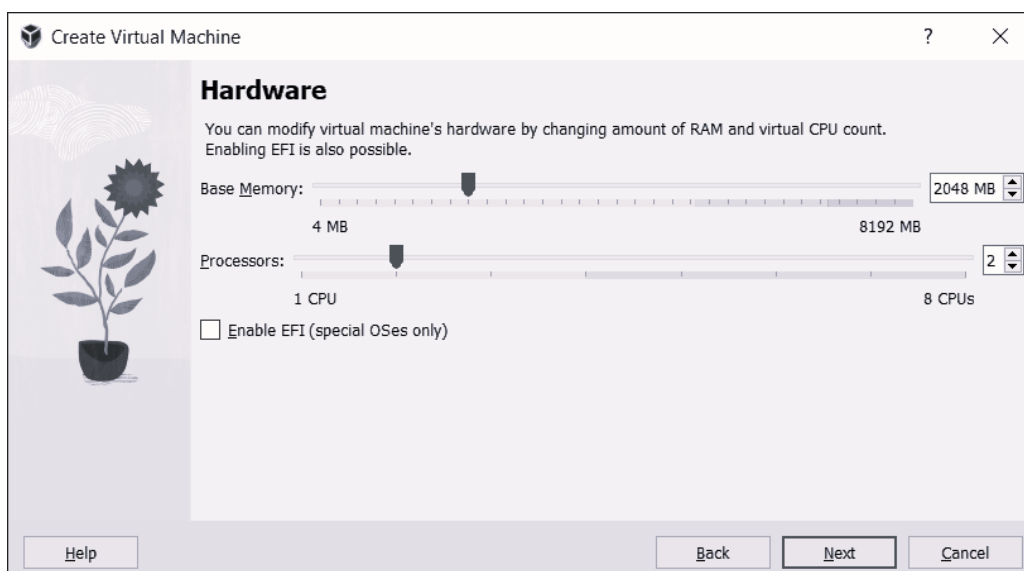


Image 6: Allocating memory

Allocate no more than 25 percent of your total system RAM to your virtual machine.

Next, reach the Hard Disk screen and choose '**Create a Virtual Hard Disk**'. I recommend choosing **dynamically allocated** disks so that your machine doesn't take all the available space, but only the amount it needs.

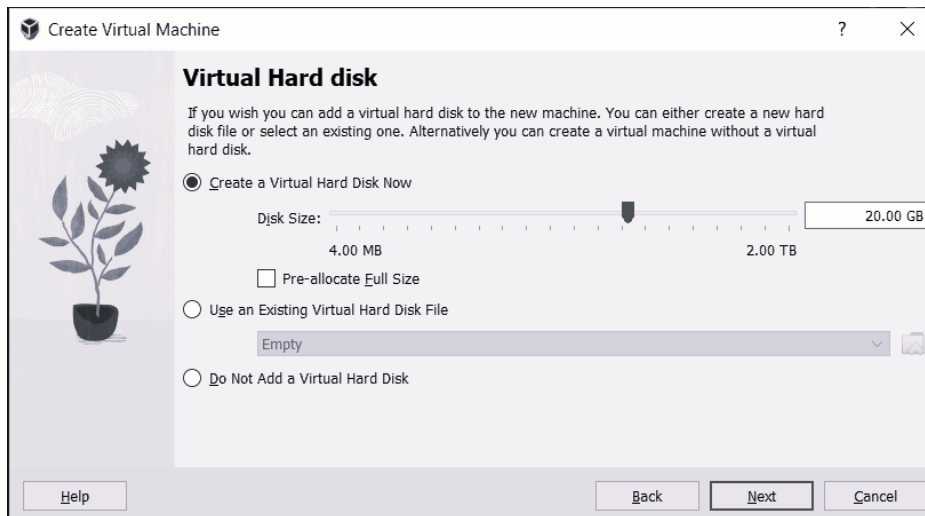


Image 7: Allocating hard drive space

The default hard drive space is 8GB, but allocating 20–25GB is advisable for better performance, and if you've chosen to dynamically allocate hard drive space it won't take the whole space available.

Click '**Create**', and your virtual machine setup is finished.

Installing Kali Linux on Your Virtual Machine

Now, you'll see a screen similar to Image 8. Click the 'Start' button on the VirtualBox Manager to begin the Kali installation.



Image 8: The VirtualBox welcome screen

When prompted for the startup disk, navigate to your downloaded Kali .iso file and select it, as shown in Image 9.

Files that end with .iso are just replicas of a disk. You can for example copy the whole disk on your machine (e.g. Disk C:) and then save it as a .iso file. Now some .iso files are bootable that's the case with Linux OS (e.g. Kali Linux, Ubuntu, RedHat).

So when installing a Linux OS for example Kali Linux, if you look closely or inspect the file you'll see it's a .iso file. And if you click on properties on the Kali Linux OS you've got, you'll see the file is a kali-linux-2024.1-installer-amd64.iso, (depending on the version the name might change)

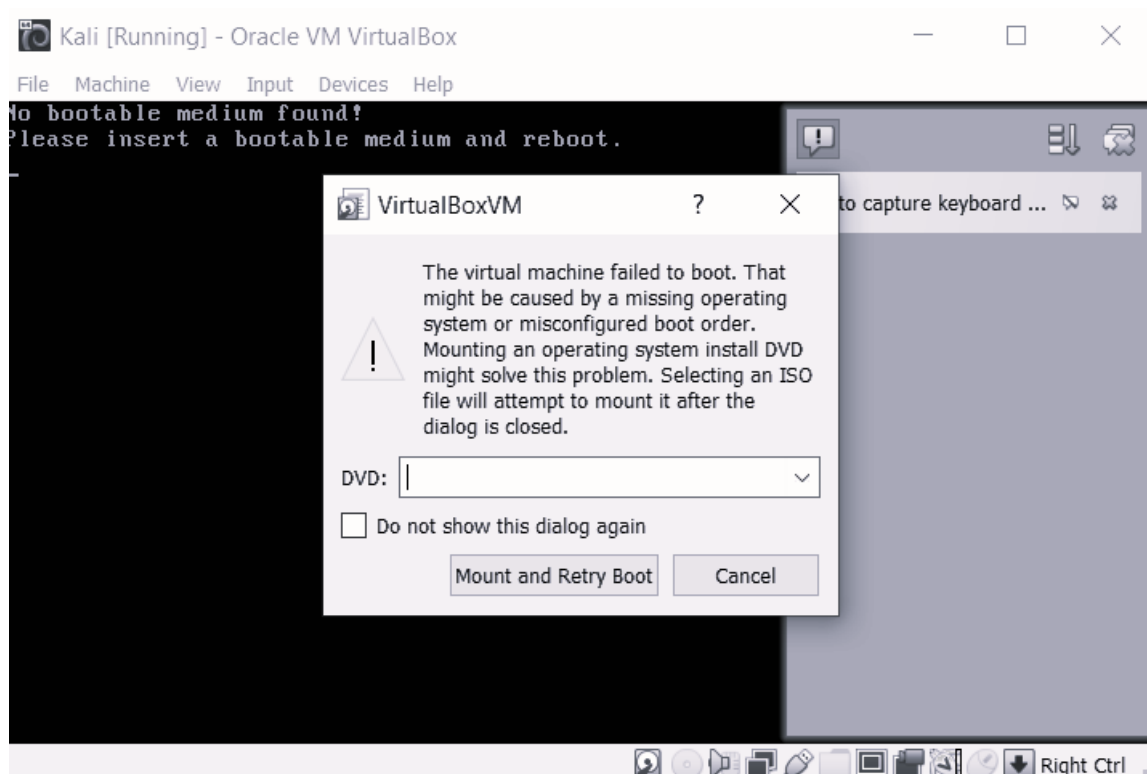


Image 9: Selecting your startup disk

Click '**Start**' to initiate the installation of Kali Linux on your virtual machine.

Initial Configuration of Kali Linux

Kali will present a screen with startup choices, as shown in Image 10. I recommend beginners use the graphical install method.



Image 10: Selecting the install method

If you encounter installation errors, check your system's BIOS for virtualization settings or conflicting software like Hyper-V on Windows systems.

Proceed to select your language, location, and keyboard layout. The system will detect your hardware and network adapters. When prompted, configure your network and enter a hostname, as seen in Image 11.



Image 11: Entering a hostname

Choose a secure password for the root user on the screen shown in Image 12.



Image 12: Choosing a password

Set your time zone, then choose 'Guided – use entire disk' for disk partitioning. When asked to write changes to the disk, as shown in Image 13, confirm and proceed.



Image 13: Writing changes to disk

For the network mirror question, select 'No', and for GRUB installation, as shown in Image 14, select 'Yes'.



Image 14: Installing GRUB

When prompted for the device for GRUB installation, select the given option for the drive. Do NOT select the 'Enter device manually' as this step might corrupt your machine if you incorrectly enter the path, and in most cases the default option given is best. 15.



Image 15: Entering your device manually

After these steps, Kali will complete the installation and prompt for a reboot.



Image 16: The Kali login screen

Log in with your chosen password and username that you assigned when selecting your Hostname and Password. The Kali Linux desktop, ready for use, will be displayed, as shown in Image 17.



Image 17: The Kali home screen

You are now ready to embark on your hacking journey. Welcome to the world of ethical hacking!

1. BEGINNING WITH FUNDAMENTALS

Hackers are inherently practitioners. We thrive on interacting with and manipulating things. We also enjoy creating and occasionally dismantling things. Few of us have the patience to pore over lengthy academic IT theory before getting to what we love the most: hacking. With that perspective, this chapter aims to provide you with basic skills to start operating in Kali Linux immediately.

In this chapter, we won't dive deeply into any single topic. We'll cover just enough to allow you to experiment and discover in Linux. More thorough discussions will follow in later chapters.

Introductory Terms and Concepts

Before we start exploring Linux Basics, it is important to get acquainted with a few terms that will be mentioned later in this chapter.

Binaries:

These are executable files, much like Windows executables. Binaries are usually found in the `/usr/bin` or `/usr/sbin` directory and encompass utilities such as `ps`, `cat`, `ls`, and `cd` (all of which we'll cover in this chapter), as well as applications like the wireless hacking tool `aircrack-ng` and the intrusion detection system `Snort`.

Case Sensitivity:

Linux is case-sensitive, unlike Windows. This means `Desktop`, `desktop`, and `DeskTop` would each be distinct file or directory names. This often frustrates those transitioning from Windows, especially if you encounter a "file or directory not found" error - it's likely a case sensitivity issue.

Directory:

Equivalent to a Windows folder, a directory is a method of organizing files, typically in a hierarchical structure.

Home:

Every user has a /home directory, where files you create are usually saved by default.

Kali:

Kali Linux is a version of Linux specifically developed for penetration testing. It comes with a myriad of tools pre-installed, which saves the time it would take to download and install them individually.

Root:

Similar to nearly every operating system, Linux has an administrator or superuser account, intended for a trusted user who can perform almost any action on the system. This includes tasks like reconfiguring the system, adding users, and modifying passwords. In Linux, this account is known as root. As a hacker or penetration tester, you will frequently need to use the root account to gain control over the system. Many hacking tools require root account access.

Script:

This is a sequence of commands executed in an interpretive environment, transforming each line into source code. Numerous hacking tools are simply scripts. Scripts can be executed with the bash interpreter or other scripting language interpreters like Python, Perl, or Ruby. Among hackers, Python is currently the most favored interpreter.

Shell:

This is a command execution environment in Linux. The most commonly used shell is bash, which stands for Bourne-again shell. Other popular shells include the C shell and Z shell. In this book, we will use the bash shell exclusively.

Terminal:

This is a command line interface (CLI).

With these basic concepts covered, we will gradually build up the essential Linux skills needed to become a hacker or penetration tester. This first chapter will guide you through the initial steps in Kali Linux.

Exploring Kali Linux

Upon starting Kali Linux, you'll be presented with a login screen. Use the standard user account credentials you set up during the installation process to log in.



Image 1-1: Logging into Kali using the root account

Once you've successfully logged in, you'll see the Kali Linux desktop. This environment is streamlined for efficiency, with important tools and features within easy reach.



The desktop provides access to a variety of applications and utilities. Key among these is the terminal interface and the file system, which are essential for navigating and utilizing Kali Linux to its full potential.

Understanding the Terminal

To start using Kali, the first step is to open the terminal, Kali's command line interface. The terminal icon is located at the bottom of the desktop, or you can open it by pressing CTRL-ALT-T. The terminal you open should look similar to the one in Image 1-3.

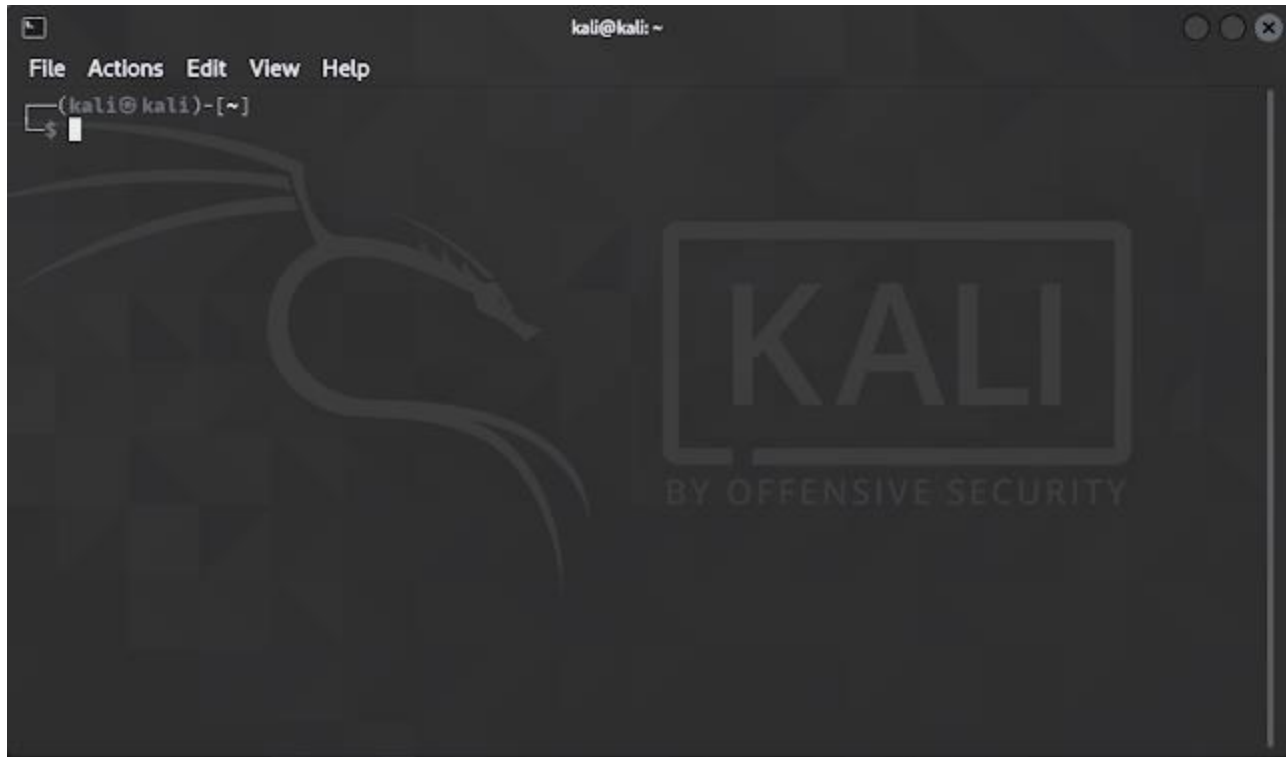


Image 1-3: The Kali terminal

This terminal launches the command line environment, known as the shell, where you can execute commands and write scripts. Although Linux features various shell environments, the most prevalent is the bash shell, which is the default in Kali and several other Linux distributions.

To change your password in Kali, use the command `passwd`.

Navigating the Linux Filesystem

The structure of the Linux filesystem is different from that of Windows. In Linux, there isn't a physical drive like the C: drive at the base of the filesystem. Instead, Linux employs a logical filesystem. The very top of this filesystem structure is `/`, often referred to as the root of the filesystem, resembling an upside-down tree (see Image 1-4). It's important to distinguish this from the root user. Though these terms might be

confusing initially, they will become clearer as you grow accustomed to Linux.

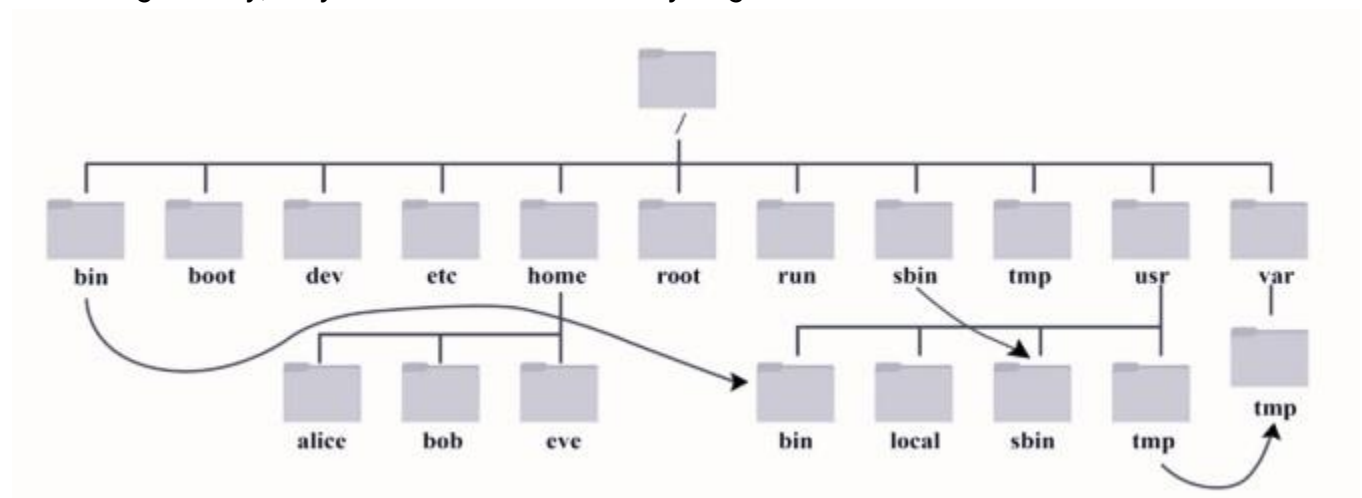


Image 1-4: The Linux filesystem

At the top of the filesystem tree is the root (/), and some of the key subdirectories to be aware of include: /root (the home directory of the powerful root user), /etc (usually contains Linux configuration files), /home (the user's home directory), /mnt (where other filesystems are attached or mounted), /media (where CDs and USB devices are typically attached or mounted), /bin (where application binaries are located, similar to executables in Windows), and /lib (where libraries, akin to Windows DLLs, can be found). We will explore these vital directories more throughout this book.

It's crucial to remember that you should not log in as root for routine tasks. This is a security measure; if someone hacks your system while you're logged in as root, they would gain root privileges and control over your system. Use a regular user account for daily operations like starting applications, web browsing, and running tools like Wireshark.

Essential Linux Commands

Let's start with some fundamental commands that will get you operational in Linux.

Locating Your Current Directory with pwd

In a command line interface (CLI) like Linux, as opposed to a graphical user interface (GUI) environment such as Windows or macOS, it's not always evident which directory

you're in. Knowing your current directory is essential when navigating to a new location. The 'print working directory' command, `pwd`, informs you of your position in the directory hierarchy.

Type `pwd` in your terminal to find out your current location:

```
kali >pwd  
/root
```

In this instance, Linux indicates that I'm in the root user's directory. If you've logged in as root by starting the red terminal in Kali, you'll likely be in the root user's directory too, situated just below the top level of the filesystem structure (/).

Should you be in a different directory, `pwd` will display that directory's name.

For example, if you login as a normal user named "kali", then write the command `pwd` that prints your current working directory you'd see:

```
kali >pwd  
/home/kali
```

Or if you somehow go to the `/etc` directory (right under the / (root) directory where you have configuration files), you'd see this:

```
kali >pwd  
/etc
```

Identifying Your User with `whoami`

In Linux, the superuser or system administrator, known as `root`, holds all necessary system privileges for tasks like adding users, altering passwords, and adjusting privileges for files (i.e. who can delete, edit, view files). As a hacker, you generally require these privileges to operate the programs and commands needed (many hacking tools mandate root privileges), so logging in as `root` is typical.

If you're unsure whether you're logged in as `root` or another user, the `whoami` command will show your current user identity:

```
kali >whoami
```

```
root
```

If I were logged in under a different user, like my personal account, whoami would have shown that username instead, like this:

```
kali >whoami
```

```
Kali
```

This is handy when you need to either check your username to then insert it in different scripts or utilities, or when you just want to verify you're a root user thus have root privileges.

Note on User Accounts and Permissions in Kali Linux

Recent versions of Kali Linux have changed how user accounts are handled during installation. Instead of assigning root privileges by default, you are now encouraged to create a standard user account for non-administrative activities. This shift improves security by limiting the impact of mistakes or potential exploits.

Understanding Permissions and Root Access

Many commands in this book require root privileges. If you encounter a "Permission Denied" error, it means the action requires administrative permissions. To resolve this:

1. **Use sudo for Individual Commands:**
2. Prefix the command with sudo, for example:

```
sudo [command]
```

Sudo is designed to give you "root" permissions for only 1 command, but you're only allowed to use "sudo" if you're in the "sudoer group" i.e. the admin of the system (root) has allowed you to use sudo. The username you entered while installing kali linux is part of the "sudoer group" by default, we'll later learn how you can both create users and

assign them to different user groups (like the “sudo group” which allows you to use the “sudo” command that grants “root” permissions for only 1 command.)

3. Switch to the Root User Temporarily:

Use the following command to switch to the root user:

```
sudo su
```

Enter the password you set during installation when prompted. Keep in mind, here “su” means “Switch User” so you can use it to go back to a normal user by writing:

```
su kali
```

Now you should be the user named “kali”.

If you don’t use sudo or switch to the root user when necessary, you might receive a “Permission not allowed” error message.

For tasks in this book involving system files or restricted directories, always ensure you have the necessary privileges by using sudo or switching to the root user temporarily. This will help you avoid errors and streamline your workflow.

Moving Through the Linux Filesystem

Mastering navigation within the terminal-based filesystem is a key Linux skill. You need to be adept at moving around to locate applications, files, and directories in various locations. Unlike a GUI-based system, where directories are visually evident, navigating within a CLI is text-based, requiring specific commands (it also means that if you write WhoAmI you’ll get an error message as Linux is case sensitive, you’ll instead need to write all lowercase -> whoami)

Changing Directories with cd

Use the cd (change directory) command to shift between directories in the terminal. For instance, to switch to the /etc directory that houses configuration files, input:

```
root@kali >cd /etc
```

```
root@kali - [/etc]
```

The prompt changes to root@kali - [/etc], confirming we're now in the /etc directory. To verify, enter pwd which should print our working directory (i.e. our current directory/folder) :

```
root@kali - [/etc]
```

```
> pwd
```

```
/etc
```

To ascend one level in the file structure (towards the root, or /), use cd followed by double dots (.). This will just switch to the directory above your current (remember the filesystem image), if you were in /home/kali and wrote cd.. You'd go to /home:

/home/kali



In our current example:

```
root@kali - [/etc]
```

```
> cd ..
```

```
root@kali - [/]
```

```
> pwd
```

```
/
```

This command elevates us from /etc to the / root directory. You can ascend multiple levels by matching the number of double-dot pairs to the levels you wish to rise and adding a "/" in between each of them. For example:

- Use '..' to ascend one level.
- Use '../..' to ascend two levels.
- Use '../../..' to ascend three levels, and so on.

Each `..` represents moving up one directory level.
Forward slashes `/` are used to separate the `..` components.

Assume the current directory is `/home/user/Documents/Projects`:

- `cd ..` moves to **`/home/user/Documents`**
- `cd ../..` moves to **`/home/user`**
- `cd ../../..` moves to **`/home`**

To illustrate, to move up two levels, input `cd` followed by two double-dot sets separated by a space:

```
root@kali - [/home/kali]  
> cd ../../
```

- To go directly to the root level from any location, enter `cd /`, with `/` symbolizing the filesystem's root.

List Directory Contents with `ls`

The `ls` (list) command, akin to the `dir` command in Windows, allows you to view the contents of a directory, including files and subdirectories. It will just list all the files in your current directory/folder (which is from where `ls` comes from => list -> `ls`). This is the same as going to File Explorer or Finder to view all the files in a certain folder, but in Linux you do it in the terminal.

```
root@kali - [/]  
> ls
```

```
bin  
initrd.img  
media  
run  
var  
boot  
initrd.img.old  
mnt  
sbin  
vmlinuz
```

This command lists all files and directories within the current directory. You can also target a specific directory by appending its name after `ls`; for instance, `ls /etc` displays the contents of the `/etc` directory regardless the fact you currently are in the `/home/kali` directory, not the `/etc` directory – as long as you specify the directory after “`ls`” it’ll be listed, if you specify nothing, your current directory will be listed.

To gain more details about the files and directories, such as permissions, owner, size, and last modification date, include the `-l` option after `ls` (where `l` stands for long), known as long listing.

The `-l` option provides detailed information about each file and directory in the following format:

```
<permissions> <number of links> <owner> <group> <size> <last modified  
date> <name>
```

The very first column, like `lrwxrwxrwx` or `drwxr-xr-x`, provides information about the **type of item** (file, directory, or other) and the **permissions** granted to different users.

Let's take an example:

`drwxr-xr-x`

Here's what each part means:

- **File Type (d):**
 - The first character indicates the type of item:
 - `d`: Directory (a folder that contains other files or directories).
 - `-`: Regular file (a text document, script, or anything that's not a directory).
 - `l`: Symbolic link (a shortcut pointing to another file or directory, we'll touch on that later).
- **Permissions (rwxr-xr-x):**
 - The next nine characters are grouped into **three sets** of three characters. Each set represents the permissions for a specific group of users:
 - The **first set** is for the **owner** (the person who created or owns the file).
 - The **second set** is for the **group** (you can create groups in Linux (view them as different teams e.g. Security group will be user accounts all about security and they shouldn't access files in Financing group) here the group is by default the primary group of the owner).
 - The **third set** is for **others** (every other user on the Linux machine or Linux server).

Within each group you have 3 possible permissions:

- `r`: Read permission (can view or list the contents of the file or directory).
- `w`: Write permission (can modify the file or create/delete files in a directory).
- `x`: Execute permission (can run a file if it's a program or enter a directory).

- -: No permissions (e.g. rw- means a user lacks execute permission as there is a "-" on the place of "x" for execute).
- Permissions are sorted in this manner:
 - **R**[read] **W**[write/modify] **X**[run/execute]
 - **rwX**

Example Analysis:

drwxr-xr-x

- **d**: It's a directory.
- **rwX**: The owner has read, write, and execute permissions.
- **r-x`**: Members of the group can read and execute, but not write.
- **r-x**: Everyone else can read and execute, but not write.

Another Example:

lrwxrwxrwx

- **l**: It's a symbolic link.
- **rwXrwxrwx**: Everyone (owner, group, others) has full permissions to read, write, and execute.

The symbolic link will also include the target it points to, like:
bin -> usr/bin

This means **bin** is a shortcut to the **usr/bin** directory.

A symbolic link is like a shortcut that points to another file or directory. It doesn't hold the actual content but redirects operations to its target.

For example, if **/bin** is a symbolic link to **/usr/bin**:

- When you `cd /bin`, you're actually navigating to `/usr/bin`, so you land in that directory.

- When you `ls /bin`, it shows the contents of `/usr/bin` because that's where the link points.

In short, any interaction with `/bin` (like listing files, running a program, or accessing a file) is redirected to `/usr/bin` as if `/bin` were just another name for `/usr/bin`.

2. Number of Links

The next column in the output of `ls -l` shows the **number of links** associated with each file or directory.

Think of a **link** as a name for a file or directory. By default, every file or directory has at least one name, which is the one you see when you list it with `ls`. However, Linux allows you to create **extra names** for the same file—these are called **hard links**.

Imagine this situation:

- You have a file called `notes.txt` with important information.
- You want a **backup name** for it so you can easily access it from another folder or avoid losing it by accident. A hard link lets you create another name for the same file without copying it. If one name gets deleted, the file is still accessible using the other name.

Let's walk through a simple example:

1. You create a file called `notes.txt` in your current folder
2. Run `ls -l` to see the file:

```
ls -l
-rw-r--r-- 1 user user 16 Nov 25 15:30 notes.txt
```

The 1 in the second column means this file has **one link**—its current name, `notes.txt`.

3. Create a **hard link** called `backup_notes.txt`:

4. Run `ls -l` again:

```
ls -l
```

```
-rw-r--r-- 2 user user 16 Nov 25 15:30 notes.txt
-rw-r--r-- 2 user user 16 Nov 25 15:30 backup_notes.txt
```

Now both `notes.txt` and `backup_notes.txt` have a 2 in the second column. This means they are **two names** for the same file.

Because `notes.txt` and `backup_notes.txt` are hard links to the same file, any changes you make using one name are instantly reflected in the other.

1. Edit the file `backup_notes.txt`
2. Check the content of both files

The content of `notes.txt` has been updated too because both names refer to the same data.

If you delete one of the names, the file is still accessible using the other name.

3. Owner

The third column specifies the **owner** of the file or directory. This is typically the user who created it. Owners have specific permissions (the first set in the `rwX` format).

Example:

```
drwxr-xr-x 3 root root 4096 Jan 16 06:36 home
```

- The owner of `home` is `root`, meaning the user `root` has the permissions `rwX` (read, write, execute).

4. Group

The fourth column shows the **group** associated with the file or directory. A group is a collection of users (there can be a Security group with users only involved in security tasks and a Financing group with users involved in financing tasks, one shouldn't have access to the files of the other), all group members share the permissions specified for the group (the second set in `rwX`).

Example:

```
drwxr-xr-x  3 root Pentesting 4096 Jan 16 06:36 home
```

- The group for home is Pentesting, meaning members of the Pentesting group have the permissions `r-x` (read, execute, but can't edit the directory owner, permissions, etc.).

5. File Size

The fifth column displays the **size** of the file in bytes.

- For **files**, this is the actual file size.
- For **directories**, it shows the metadata size (usually 4096 bytes). This does **not** include the size of the files within the directory.

Example:

```
drwxr-xr-x 123 root root 12288 Jan 26 02:37 etc
```

- The directory etc has a metadata size of **12288 bytes**.

Human-Readable Sizes: To display sizes in KB, MB, or GB, use the `-h` option:

```
ls -lh
```

```
drwxr-xr-x 123 root root 12K Jan 26 02:37 etc
```

6. Last Modified Date and Time

The sixth column shows the **last time the file or directory was modified**. This is useful for tracking changes.

Example:

```
drwxr-xr-x  3 root root 4096 Jan 16 06:36 boot
```

- The boot directory was last modified on **January 16th** at **06:36 AM**.

7. Name

Finally, the last column is the **name** of the file or directory.

Example:

```
bin -> usr/bin
```

- bin is a symbolic link pointing to usr/bin.

Hidden Files with `ls -a`

Hidden directories (and files) in Linux are those whose names start with a dot (`.`), such as `.bashrc`. They are not shown by default when you use the `ls` command because they usually contain configuration files, temporary data, or application settings that users don't need to interact with regularly. This keeps the directory listing clean and less cluttered.

To see hidden directories, you can use the `-a` option with `ls`:

```
ls -la
```

```
drwxr-xr-x 19 root root 4096 Jan 16 06:32 .  
drwxr-xr-x 19 root root 4096 Jan 16 06:32 ..  
drwx----- 2 root root 4096 Jan 16 06:32 .cache
```

Here,

- `.` represents the current directory
- `..` is the parent directory
- `.cache` is a hidden directory.

Hidden directories like `.cache` are often used by applications to store temporary or optimization-related data. You might also find configuration files for tools and programs in hidden directories, such as `.bashrc`, which customizes your shell environment.

These directories are useful when you need to adjust application settings, troubleshoot, or clear old data. However, they're hidden by default to prevent accidental modification or deletion, ensuring the system runs smoothly while keeping your workspace uncluttered.

Accessing Help Resources

Almost every command, application, or utility in Linux has a dedicated help file, providing valuable guidance on its usage. For instance, if you require assistance with `pwd` command you can easily access its help information by using the following command:

```
kali > pwd --help
```

Take note of the double dash (--); in Linux, it's common to use a double dash before word options like "help" and a single dash (-) for single-letter options such as -h. This is because as you saw earlier, you can combine multiple options together (e.g. -la) so writing "-help" would imply "-h", "-e", "-l", "-p" unless you use "--".

Upon executing this command, you'll receive a concise description of the tool and instructions on how to utilize it. In some cases, you can use either -h or -? to access the help file. For example, when seeking guidance on the port-scanning tool, nmap:

```
kali > nmap -h
```

However, it's worth noting that not all applications support all three options (--help, -h, and -?). If one option doesn't work, don't hesitate to try another.

Using Manual Pages with man

Apart from the help option, most commands and applications have a manual (man) page that offers comprehensive information, including descriptions and command summaries. To access a man page, simply enter the command or application after writing "man." For instance, to explore the aircrack-ng manual:

```
kali > man aircrack-ng
```

Aircrack-ng 1.2 beta3 - wireless WEP/WPA cracking utility

SYNOPSIS

```
aircrack-ng [options] <capture file(s)>
```

DESCRIPTION

Aircrack-ng is an 802.11 WEP and WPA-PSK keys cracking program that can recover keys once enough data packets have been captured. It implements the standard FMS attack along with some optimizations like KoreK attacks, as well as the PTW attack, thus making the attack much faster compared to other WEP cracking tools.

--snip--

This opens up the manual for aircrack-ng, providing a wealth of information beyond what the help screen offers. You can navigate through this manual using the ENTER key, or page up and down using the PG DN and PG UP keys. To exit the manual, type "q" (standing for "quit") to return to the command prompt.

Techniques for Locating Files and Directories

Navigating Linux may seem challenging initially, but mastering a few basic commands and techniques will make the command line feel much friendlier. The following commands will help you efficiently locate items from the terminal.

Searching with locate

The "locate" command is perhaps the simplest to use. By specifying a keyword, it searches your entire filesystem and identifies all instances of that word. For instance, to find aircrack-ng:

```
kali > locate aircrack-ng
usrbin/aircrack-ng
```



```
usrshare/applications/kali-aircrack-ng.desktop
usrshare/desktop-directories/05-1-01-aircrack-ng.directory
--snip--
/var/lib/dpkg/info/aircrack-ng.md5sums
```

If the `locate` command isn't installed on your Kali Linux machine by default you can just use the following commands to install it:

```
kali > sudo apt update
```

```
kali > sudo apt install locate
```

It's important to note that the "`locate`" command has its limitations. It can sometimes provide overwhelming results due to it displaying any file which contains the specified keywords, no matter where it is or what type it is, and the database it relies on is usually updated just once a day, which may result in delays in detecting recently created files.

Finding Executables with `whereis`

When searching for a binary file, "`whereis`" is your tool of choice. Binary files are commands. E.g. `ls` = binary file located in `/usr/bin`. `cd` = binary file located in `/usr/bin`.

The `whereis` command not only reveals the command's location but also its source and man page if available. Here's an example for `aircrack-ng`:

```
kali > whereis aircrack-ng
aircrack-ng: /usr/bin/aircrack-ng /usr/share/man/man1/aircrack-ng.1.gz
```

In this case, "`whereis`" returns the location of the `aircrack-ng` binaries and its man page, providing a more efficient and enlightening search.

Locating Binaries in the PATH Variable with which

The `which` command is a straightforward tool used to identify the exact location of a binary file (the executable version of a command) listed in the **PATH variable**. The **PATH variable** is an variable in Linux that tells the system where to search for executable files when you type a command in the terminal.

When you run a command, such as `ls`, the system doesn't automatically know where the binary file for `ls` is located. Instead, it refers to the directories listed in the **PATH variable** and searches them one by one until it finds the command. If the system doesn't find the command in any of the directories specified in the PATH variable, you get an error like:

Command Not Found

For now, you only need to know that the PATH variable includes directories where commands are likely stored, such as:

- **/usr/bin**: This directory contains most of the system's user-level commands, like `ls`, `pwd`, and `whoami`.
- **/usr/sbin**: This directory contains commands that typically require root privileges, such as administrative tools.

For example, when you type `ls`, the system looks for a file named `ls` in each directory listed in the PATH variable, starting with the first directory. If it finds the binary file for `ls` in `/usr/bin`, the command is executed.

The `which` command is helpful when you want to know exactly where the binary file for a command is located. Unlike the `whereis` command (which shows additional information, like the location of the manual page), `which` focuses solely on the command's binary file.

Example Usage:

```
kali > which aircrack-ng  
/usr/bin/aircrack-ng
```

In this case, which shows that the aircrack-ng command is located in /usr/bin. This means that the system found the binary file for aircrack-ng in that directory, as specified in the PATH variable.

```
kali > which aircrack-ng  
/usr/bin/aircrack-ng
```

In this case, "which" discovers the binary file in the directories specified in the PATH variable.

Advanced Searches with find

The "find" command is the most versatile searching utility in Linux. It allows you to initiate searches from any designated directory and look for various parameters, including filenames, creation or modification dates, owners, groups, permissions, and sizes.

The basic syntax for "find" is as follows:

```
find {directory} {options} {expression}
```

For instance, if you're searching for a file named "apache2" in the root directory, you would use the following command:

```
kali > find / -type f -name apache2
```

This search starts from the root directory ("/"), specifies that it's searching for an ordinary file ("-type f"), and targets the filename "apache2." The results provide all instances found.

My results for this search are shown here:

```
kali >find / -type f -name apache2
usr/lib/apache2/mpm-itk/apache2
usr/lib/apache2/mpm-event/apache2
usr/lib/apache2/mpm-worker/apache2
usr/lib/apache2/mpm-prefork/apache2
/etc/cron.daily/apache2
/etc/logrotate.d/apache2
```

Keep in mind that searching through every directory can be time-consuming. To speed up the process, consider narrowing your search to the directory where you expect to find the file you need. In our case, searching for a configuration file led us to the "/etc" directory, which contains subdirectories. This more focused search gives quicker results.

Additionally, "find" displays exact name matches by default. If a file has an extension, such as "apache2.conf," a simple search may not locate it. You can overcome this limitation by using wildcards, which match multiple characters. Various wildcard forms are available, including "*", ".", ",", "?," and "[]."

Summary Of Searching Commands

To locate files and directories efficiently, use these commands based on your needs:

locate is the quickest way to search your entire filesystem for a file containing a specific keyword. It's fast but relies on a periodically updated database, so it may not show very recent files and can include irrelevant results.

whereis helps you find binaries, their manual pages, and sometimes their source files. It's useful for understanding where commands like `ls` or `cd` are stored and their exact manuals.

which focuses on locating executable files in the directories specified by the `PATH` variable. Use it to check where a command like `ls` or `pwd` is being executed from, without showing extra details like manuals.

find is the most flexible search tool. It allows precise searches by filename, type, size, permissions, and more. It's slower than `locate` but perfect for pinpointing exact matches or searching specific directories.

Each command serves a distinct purpose, so choose based on whether you need speed, comprehensive results, or detailed filtering.

QUICK LOOK AT WILDCARDS

Now, let's explore wildcards—a powerful feature in Linux that allows you to perform flexible searches.

The ? Wildcard

The `?` wildcard matches exactly **one character**. Use it when the files or directories you're looking for differ by just one character.

Imagine you're in a directory containing the following files:

```
dog  log  fog  blog
```

To list files with exactly three characters you can use the command:

```
> ls ?og
dog  log  fog
```

- This matches dog, log, and fog because they all have one character before og.
- It does **not** match blog because blog has two characters before og.

The [] Wildcard

The [] wildcard matches **any one character inside the brackets**. Use it when you know the possible variations for a specific character in the name.

Using the same files from before:

```
dog  log  fog  blog
```

When you try to list files that start with either d or l:

```
> ls [dl]og
dog  log
```

- This matches dog and log because their first letters (d and l) are in the brackets.
- It does **not** match fog or blog because their first letters aren't included.

The * Wildcard

The * wildcard is the most flexible—it matches **any number of characters**, including none. Use it when you want to search broadly for names that follow a general pattern.

For example to list all files that end with og:

```
> ls *og
dog  log  fog  blog
```

The * matches anything before og, so it finds all files ending with og.

To list all files that start with b:

```
ls b*  
blog
```

The * matches everything after b, so it finds blog.

If you were in a directory with the following files:

```
notes.txt  report.txt  image.png  summary.docx
```

If you want to list only the .txt files, you can use the * wildcard with the ls command:

```
ls *.txt  
notes.txt  report.txt
```

Here, the * matches everything before .txt, so it lists all files ending with .txt. This is especially useful when you have many files in a directory and only want to see specific types, like text files.

Sorting and Filtering with grep

When working on the command line, you often need to search for specific keywords within text. This is where the "grep" command comes in handy as a powerful filtering tool.

"grep" is frequently used when you want to search for keywords within the output of other commands. Linux allows you to send the output of one command as input to another through a process called piping (using the "|" symbol).

For example, consider the "ps" command used to display information about running processes. To view all processes on a Linux system, you can use the "ps" command with the "aux" switches as follows:

```
kali > ps aux
```

This provides a list of all processes on the system. But what if you're interested in finding a specific process? You can use "grep" to filter the output. For instance, to check if the "apache2" service is running:

```
kali > ps aux | grep apache2
root 4851 0.2 0.7 37548 7668 ? Ss 10:14 0:00 usrsbin/apache2 -k start
root 4906 0.0 0.4 37572 4228 ? S 10:14 0:00 usrsbin/apache2 -k start
root 4910 0.0 0.4 37572 4228 ? Ss 10:14 0:00 usrsbin/apache2 -k start
--snip--
```

This command instructs Linux to display all services and then passes that output to "grep," which searches for the keyword "apache2" and displays only the relevant output. This efficient method saves time and effort when searching through lengthy lists of processes.

Handling Files and Directories

Once you've located your files and directories, you'll want to perform various actions on them, such as creating, copying, renaming, and deleting. Let's explore these operations:

Creating Files with cat

There are several ways to create files in Linux. For now, let's focus on two simple methods. First, the "cat" command, short for concatenate, can be used not only for displaying file contents but also for creating small files. For larger files, using a text editor is recommended.

To create a file with "cat," use the following syntax:

```
kali > cat > filename
```

For example:

```
kali > cat > hackingskills  
Hacking is the most importnat skill, someone can have!
```

Linux enters interactive mode and waits for you to start entering content for the file. Simply type the content and press CTRL-D to exit. To view the file's content, use:

```
kali > cat filename
```

To append content to a file, use "cat" with a double redirect (">>"):

```
kali > cat >> hackingskills  
Everyone should learn hacking
```

And now we get:

```
kali >cat hackingskills  
Hacking is the most importnat skill, someone can have! Everyone should learn  
hacking
```

File Creation with touch

The "touch" command is another method to create files in Linux. Originally intended to modify file details, it also creates a file if it doesn't already exist. To create a new file with "touch," use:

```
kali > touch filename
```

For example:

```
kali > touch newfile  
kali > ls -l  
newfile
```

The good thing about touch is you can create any type of files. By default if you don't append an extension (e.g. .png, .txt, .jpg) to a file you will create a text file, but unlike "cat", when you use "touch" you can create any sort of files by specifying their file extension.

Directory Creation

To create directories in Linux, use the "mkdir" command, which stands for "make directory." For instance, to create a directory named "newdirectory," enter:

```
kali > mkdir newdirectory
```

You can navigate to the newly created directory with the "cd" command:

```
kali > cd newdirectory
```

Copying Files

Copying files is accomplished using the "cp" command. This command creates a duplicate of the file in a new location while retaining the original.

For example, to create a file named "oldfile" and copy it to "/root/newdirectory," renaming it in the process:

```
kali > touch oldfile  
kali > cp oldfile /root/newdirectory/newfile
```

Renaming the file during copying is optional; you can simply provide a new name at the end of the directory path. If no renaming is specified, the file retains its original name.

After navigating to "newdirectory," you'll see an exact copy of "oldfile" named "newfile":

```
kali > cd newdirectory  
kali > ls  
newfile oldfile
```

Renaming Files

Linux lacks a dedicated command for renaming files, unlike Windows and some other operating systems, but it offers the versatile "mv" (move) command.

The "mv" command serves not only to move files or directories to new locations but also to give an existing file a fresh name. To rename "newfile" to "newfile2," execute the following command:

```
kali > mv newfile newfile2
kali > ls
oldfile newfile2
```

Now, upon listing (ls) the directory, you'll find "newfile2" but not "newfile," as it has been successfully renamed. The same approach applies to directories.

Don't confuse the "mv" command with the "cp" command, "cp" will create a duplicate of the same file (where you also have the option to rename it if you decide to put an exact file name after the /path/ specified), "mv" on the other hand will not duplicate the file instead it'll just change it's name.

The "mv" command has another functionality, which is to move files from one location to another. For instance, if you want to move newfile2 to the /root/Documents/ directory, you can use the following command:

```
kali > mv newfile2 /root/Documents/
```

This will remove newfile2 from the current directory and place it inside the /root/Documents/ directory. Upon listing (ls) your current directory, you'll no longer see newfile2, as it has been successfully moved to its new location. Similarly, this works for directories, allowing you to relocate entire folders.

Deleting Files

To delete a file, you can conveniently use the "rm" command, as shown below:

```
kali > rm newfile2
```

A long listing of the directory will confirm that the file has been removed.

Deleting Directories

Deleting a directory follows a similar pattern to the "rm" command for files, but you append "dir" (for directory) to it. For example:

```
kali > rmdir newdirectory  
rmdir: failed to remove 'newdirectory': Directory not empty
```

It's worth noting that "rmdir" won't remove a non-empty directory. Instead, it will issue a warning message, as seen in this example. You must empty the directory of its contents before attempting removal. This safeguard prevents accidental deletion of objects you didn't intend to remove.

If you do wish to remove a directory and its contents in one go, you can utilize the "-r" switch with "rm," like so:

```
kali > rm -r newdirectory
```

A word of caution: exercise care when using the "-r" option with "rm," especially initially, as it's easy to unintentionally delete valuable files and directories. For instance, using "rm -r" in your home directory would erase every file and directory therein, likely not your intended outcome.

Though if you want you can try to use the "rm -rf --no-preserve-root /" to delete your entire operating system. Before that though, close your virtual box machine, go to Virtual Box, there press on the 3 dotted lines on your Kali Linux VM, press on Snapshots, click on "take" button. Then start your machine, and once broken, close it once again, go to Snapshots, click on the "Snapshot 1" you created and press on Restore button on top.

In the "rm -rf --no-preserve-root /" command:

- **rm:** The command to remove files or directories.

- **-r**: Recursive, meaning it deletes directories and their contents.
- **-f**: Force, meaning it skips prompts and permissions checks.
- **--no-preserve-root**: A flag that overrides the safeguard preventing accidental deletion of the root directory (/).
- **/**: The root directory, which contains the entire file system.

Exploration and Practice

With these fundamental skills for navigating the filesystem, take some time to explore and experiment with your Linux system. Familiarizing yourself with the terminal is best accomplished by putting your newfound skills into practice. In the forthcoming chapters, we will delve even deeper into our hacking distribution.

Practice Tasks

Before proceeding to Chapter 2, solidify your understanding of the concepts from this chapter by attempting the following exercises:

Home Directory Navigation and Listing:

- Go to your home directory. Use `pwd` to confirm your current location.
- Use `ls -a` to list all contents, including hidden files, of your home directory.
- Navigate to the `/etc` directory and list its contents with `ls`.
- Pick any two directories from `/etc`, navigate into them one after the other using `cd`, and use `ls` to explore their contents.
- Return to your home directory and use `pwd` to confirm you're back.

Creating Directories and Files:

- Create a directory named `WorkProjects` in your home directory using `mkdir`.
- Navigate into `WorkProjects` with `cd`.
- Create three files named `Project1.txt`, `Project2.txt`, and `Notes.txt` using `touch`.
- Use `ls -l` to list the files within `WorkProjects` with detailed information to confirm file creation.

User and Directory Identification:

- Use the `id` command to identify the current user and their group memberships.
- Use the `pwd` command to determine your current working directory.
- Navigate to the `/usr` directory using `cd` and then use `pwd` to verify you're in the `/usr` directory.
- Return to your home directory using `cd ~` and verify your location with `pwd` once again.

Finding Command Paths and Information:

- Use the `which` command to find the full path of the `grep` command and write it to a file named `CommandPaths.txt` in your home directory.
- Use `which` again to find the full path of the `mkdir` command and append it to `CommandPaths.txt`.
- Use the `whereis` command to find additional information and locations related to the `touch` command and append this information to `CommandPaths.txt`.
- Additionally, use the `locate` command to find all instances of `ls` and append this information to `CommandPaths.txt`.
- Use `cat CommandPaths.txt` to review the paths and information you've gathered.

Steps:

1. Use `which grep > ~/CommandPaths.txt`.
2. Use `which mkdir >> ~/CommandPaths.txt`.
3. Use `whereis touch >> ~/CommandPaths.txt`.
4. Use `locate ls >> ~/CommandPaths.txt`.
5. Use `cat ~/CommandPaths.txt` to view the collected information.

Using Manual Pages:

- Use the `man cp` command to access the manual page for the `cp` command. Practice navigating by using the arrow keys to scroll up and down, then exit the manual page.
- Note: Look at the options for `-i` (requires confirmation for overwriting an existing file) and `-r` (copies the directory along with all its contents).

- Access the manual page for another command, such as `man mv`, and explore the different options and examples provided. Note the option for `-i` (requires confirmation for overwriting an existing file) and write it to a file named `ManualNotes.txt`.
- Use the `man intro` command to get an introduction to the manual pages and understand the conventions used in them. Add a summary of this information to `ManualNotes.txt`.
- Use `cat ManualNotes.txt` to review your notes and ensure you understand the content.

Steps:

1. `man cp`
2. Use arrow keys to navigate up and down.
3. Identify and note the options `-i` and `-r`.
4. Press `q` to exit the manual page.
5. `man mv`
6. Identify and note the option `-i`.
7. `echo "-i: Interactive move" > ~/ManualNotes.txt`
8. `man intro`
9. Summarize the conventions.
10. `echo "Summary of man pages conventions" >> ~/ManualNotes.txt`
11. `cat ~/ManualNotes.txt`

2. WORKING WITH TEXT

In Linux everything you interact with directly is considered a file, and more often than not, these files are of the textual variety. For example, all configuration files in Linux are text files. Therefore, reconfiguring an application is as simple as opening the configuration file, making the necessary text changes, saving the file, and then restarting the application. Voila! Your reconfiguration is complete.

Given the prevalence of text files, mastering text manipulation is crucial for efficiently managing Linux and its applications. In this chapter, you'll delve into several essential commands and techniques for manipulating text in the Linux environment.

To illustrate these concepts, I'll use files from one of the world's leading Network Intrusion Detection Systems (NIDS), Snort. Snort, initially developed by Marty Roesch and now owned by Cisco, is renowned for its effectiveness in detecting intrusions by hackers. Therefore, understanding NIDS is crucial, whether you aspire to thwart attacks or exploit vulnerabilities to avoid detection.

NOTE: If your version of Kali Linux doesn't come preinstalled with Snort, you can easily obtain the necessary files from the Kali repository by running the command:

```
sudo apt update
```

That will update the Linux packages and then just install snort with:

```
sudo apt install snort
```

If any other command (such as `locate`) isn't available you can do the same thing swapping "snort" with "locate"

Techniques for Viewing Files

As demonstrated in Chapter 1, the most basic command for displaying text content is probably `cat`, but it does have its limitations. You can use `cat` to view the Snort configuration file (`snort.lua`) located in `/etc/snort`:

```
kali > cat /etc/snort/snort.lua
```

The content of the entire `snort.lua` file will be displayed on your screen and will continue until it reaches the end of the file. However, this method isn't always the most convenient or practical way to work with text files.

In the following sections, we'll explore two alternative commands: `head` and `tail`, which allow you to view specific portions of a file's content, making it easier to focus on key information.

Using Head Command

If you only need to view the beginning of a file, you can use the head command. By default, head displays the first 10 lines of a file. For example, to view the first 10 lines of the `snort.lua` file:

```
kali > head /etc/snort/snort.lua
```

This command will display the initial 10 lines of `snort.lua`. If you wish to see more or fewer lines, you can specify the desired quantity using the `-n` switch followed by the number of lines. For instance, to view the first 20 lines of the file:

```
kali > head -20 /etc/snort/snort.lua
```

You'll now see only the first 20 lines of `snort.lua` on your terminal screen.

Using Tail Command

Similar to head, the tail command is used to view specific lines of a file, but it focuses on the end of the file. To view the last lines of `snort.lua`, you can use the following command:

```
kali > tail /etc/snort/snort.lua
```

By default, tail displays the last 10 lines of the file. However, you can specify a different number of lines by using the `-n` switch. For example, to view the last 20 lines:

```
kali > tail -20 /etc/snort/snort.lua
```

This command will display the last 20 lines of `snort.lua`, allowing you to easily access important information located towards the end of the file.

Line Numbering with nl

In some cases, especially when dealing with lengthy files like `snort.lua`, it can be highly beneficial to have line numbers displayed. Line numbers make it easier to reference specific lines within the file and track changes effectively. To display a file with line numbers, you can use the `nl` (number lines) command. Simply execute the following command:

```
kali > nl /etc/snort/snort.lua
```

Text Filtering with grep

The `grep` command is arguably the most widely used text manipulation tool in the Linux arsenal. It allows you to filter the content of a file, displaying only the lines that match specific criteria. For example, if you want to view all lines containing the word "output" in your `snort.lua` file, you can use `grep` in combination with `cat` as follows:

```
kali > cat /etc/snort/snort.lua | grep output
```

This command will first display the `snort.lua` file using `cat` and then use a pipe (`|`) to send its output to `grep`. `grep` will then filter the lines to show only those that contain the word "output."

The `grep` command is exceptionally powerful and essential for working efficiently in the Linux environment. It can save you hours of manual searching by quickly pinpointing every occurrence of a word or pattern in a file.

Text Replacement with sed

Now, let's explore how to use the sed command for finding and replacing text. sed allows you to search for occurrences of a word or text pattern and perform actions on them. It operates similarly to the Find and Replace function in Windows.

To search for the word "HOME_NET" in the snort.lua file using grep, you can execute the following command:

```
kali > cat /etc/snort/snort.lua | grep HOME_NET
```

If you wish to replace every occurrence of "HOME_NET" with "MY_HOME_NET" (remember that Linux is case-sensitive) and save the modified file as snort2.lua, you can use the following sed command:

```
kali > sed s/HOME_NET/MY_HOME_NET/g /etc/snort/snort.lua > snort2.lua
```

Now, if you use grep to search for "HOME_NET" in snort2.lua, you'll find no instances of it. However, searching for "MY_HOME_NET" will reveal the occurrences.

If you want to replace only the first occurrence of "HOME_NET," you can omit the g command like this:

```
kali > sed 's/HOME_NET/MY_HOME_NET/' /etc/snort/snort.lua > snort2.lua
```

Additionally, sed allows you to replace specific occurrences of a word by specifying the occurrence number. For example, to replace only the second occurrence of "HOME_NET," you can use the following command:

```
kali > sed 's/HOME_NET/MY_HOME_NET/2' /etc/snort/snort.lua > snort2.lua
```

Paging Through Files with more and less

While `cat` is useful for displaying and creating small files, it has limitations when dealing with large files. It scrolls through the entire file, which can be impractical for large files. To address this, we have two other viewing utilities: `more` and `less`.

Managing Output with more

The `more` command displays a file one page at a time, allowing you to page down using the ENTER key. It's the utility often used for viewing man pages. To open `snort.lua` with `more`, use the following command:

```
kali > more /etc/snort/snort.lua
```

Advanced Display and Filtering with less

`less` is similar to `more` but offers additional functionality. It allows you to scroll through a file at your own pace and filter it for specific terms. Open `snort.lua` with `less` as follows:

```
kali > less /etc/snort/snort.lua
```

In the bottom-left corner of the screen, you'll notice that `less` highlights the path to the file. By pressing the forward slash ("/") key, you can search for terms in the file. For example, if you need to locate the section of the configuration file related to "HOME_NET," you can simply search for it:

```
/HOME_NET
```

This command will take you directly to the first occurrence of "HOME_NET" and highlight it. You can navigate to the next occurrence by typing "n" for next.

Text Replacement with sed

Now, let's explore how to use the sed command for finding and replacing text. sed allows you to search for occurrences of a word or text pattern and perform actions on them. It operates similarly to the Find and Replace function in Windows.

To search for the word "HOME_NET" in the snort.lua file using grep, you can execute the following command:

```
kali > cat /etc/snort/snort.lua | grep HOME_NET
```

If you wish to replace every occurrence of "HOME_NET" with "MY_HOME_NET" (remember that Linux is case-sensitive) and save the modified file as snort2.lua, you can use the following sed command:

```
kali > sed s/HOME_NET/MY_HOME_NET/g /etc/snort/snort.lua > snort2.lua
```

Here we are telling Kali to find any instance (by using g) of HOME_NET and replace it with MY_HOME_NET for the file /etc/snort/snort.lua and make those changes to the file snort2.lua

Now, if you use grep to search for "HOME_NET" in snort2.lua, you'll find no instances of it. However, searching for "MY_HOME_NET" will reveal the occurrences.

If you want to replace only the first occurrence of "HOME_NET," you can omit the g command like this:

```
kali > sed 's/HOME_NET/MY_HOME_NET/' /etc/snort/snort.lua > snort2.lua
```

Additionally, sed allows you to replace specific occurrences of a word by specifying the occurrence number. For example, to replace only the second occurrence of "HOME_NET," you can use the following command:

```
kali > sed 's/HOME_NET/MY_HOME_NET/2' /etc/snort/snort.lua > snort2.lua
```

Paging Through Files with more and less

While `cat` is useful for displaying and creating small files, it has limitations when dealing with large files. It scrolls through the entire file, which can be impractical for large files. To address this, we have two other viewing utilities: `more` and `less`.

Managing Output with more

The `more` command displays a file one page at a time, allowing you to page down using the ENTER key. It's the utility often used for viewing man pages. To open `snort.lua` with `more`, use the following command:

```
kali > more /etc/snort/snort.lua
```

Advanced Display and Filtering with less

`less` is similar to `more` but offers additional functionality. It allows you to scroll through a file at your own pace and filter it for specific terms. Open `snort.lua` with `less` as follows:

```
kali > less /etc/snort/snort.lua
```

In the bottom-left corner of the screen, you'll notice that `less` highlights the path to the file. By pressing the forward slash ("/") key, you can search for terms in the file. For example, if you need to locate the section of the configuration file related to "HOME_NET," you can simply search for it:

```
/HOME_NET
```

This command will take you directly to the first occurrence of "HOME_NET" and highlight it. You can navigate to the next occurrence by typing "n" for next.

Challenge: Combine grep, nl, and sed

Now, let's tackle a hacker challenge that involves using the commands `grep`, `nl`, and `sed`. Imagine you want to display the five lines immediately preceding a line that contains a specific pattern, such as `HOME_NET`, in the `snort.lua` file. How would you do it? Keep in mind that there are numerous options available for these commands beyond what we've discussed. You can always explore more by using the built-in Linux command `man`. For example, to access the manual for the `sed` command, you can use `man sed`.

There are multiple approaches to solving this challenge. Here, I'll demonstrate one method, and your task is to discover another approach.

First, we'll find the line number that contains `HOME_NET` using `nl` and then extract the preceding five lines as well as the line itself. Execute the following commands:

```
kali > nl /etc/snort/snort.lua | grep "HOME_NET"
```

Output:

```
18  -- HOME_NET and EXTERNAL_NET must be set now

20  HOME_NET = 'any'
```

The line containing `HOME_NET` is line number 20. To extract the desired lines, execute the following command:

```
kali > nl /etc/snort/snort.lua | sed -n '15,20p'
```

Output:

```
13  -- 7. configure outputs
```



```

14  -- 8. configure tweaks

15  -----
-----

16  -- 1. configure defaults

17  -----
-----

```

This command uses `sed` to print lines 15 to 20 of the `snort.lua` file, which includes the five lines preceding the line with `HOME_NET`.

And although it may seem like the output here is wrong because it lists from line 13 to line 17 it actually isn't. Because `nl` marks only non-empty lines, while `sed` just prints the lines from 15 to 20, including empty lines which `nl` skips to mark, thus it looks like it is 2 lines behind, when it just don't count empty lines before that.

Chapter Summary

Linux offers various methods for manipulating text, each with its own strengths and weaknesses. In this chapter, we've explored some of the most valuable techniques. I encourage you to try each one and develop your own preferences. For instance, I find "grep" indispensable, and I use "less" extensively, but your experience may lead to different choices.

Practice Exercises

Before moving on to Chapter 3, I recommend practicing the skills you've learned in this chapter with the following exercises:

1. Navigate to the directory `/usr/share/wordlists/metasploit`, which contains multiple wordlists used for password brute-forcing in various password-protected devices using Metasploit, a popular pentesting and hacking framework.

2. Use the “cat” command to view the contents of the file `passwords.lst`.
3. Use the “more” command to display the file `passwords.lst`.
4. Use the “less” command to view the file `passwords.lst`.
5. Utilize the “nl” command to add line numbers to the passwords in `passwords.lst`. There should be 5006 passwords.
6. Use the “tail” command to view the last 18 passwords in `passwords.lst`.
7. Use the “cat” command to display `passwords.lst` and pipe it into a command that finds all the passwords containing “09”

3. NETWORK ANALYSIS AND MANAGEMENT

Understanding networking is crucial for any aspiring hacker. In numerous scenarios, hacking involves actions over a network, and a proficient hacker must possess the skills to connect to and interact with those networks. Whether you need to establish a connection with your Internet Protocol (IP) address concealed or redirect a target's Domain Name System (DNS) queries to your system, Linux network knowledge is essential. This chapter is your guide to essential Linux tools for analyzing and managing networks, a crucial skill for your network-hacking journey.

Network Inspection with `ifconfig`

The “`ifconfig`” command stands as one of the fundamental tools for inspecting and engaging with active network interfaces. A simple entry of “`ifconfig`” in the terminal provides a glimpse into your active network connections. Give it a try, and you'll receive output like this:

```
kali > ifconfig
eth0      Link encap:Ethernet  HWaddr 02:1a:4b:7c:9d:ef
          inet addr:192.168.10.20  Bcast:192.168.10.255  Mask:255.255.255.0

lo        Link encap:Local Loopback
          inet addr:127.0.1.1  Mask:255.0.0.0

wlan0     Link encap:Ethernet  HWaddr 12:34:56:78:9a:bc
```

As you can see, the “ifconfig” command displays information about the active network interfaces(i.e. Hardware in your PC that allows it to connect to network devices e.g. ethernet cable plug hole) on your system.

At the top of the output, you see the name of the initial detected interface, eth0, signifying Ethernet0 (Linux initiates counting from 0). This typically represents the first wired network connection. If more wired Ethernet interfaces exist, they follow the same format (eth1, eth2, and so on).

Then is the network type (Ethernet), followed by HWaddr and an address, which is the globally unique MAC address embedded in all network hardware. In this case, it refers to the network interface card (NIC).

The second line provides information about the IP address assigned to the network interface (in this instance, 192.168.10.20). Additionally, it includes the broadcast address (Bcast) used for sending information to all IPs connected to this network, the group, and the network mask (Mask), which determines which part of the full IP is for the Network and which the part is for machine/host only.

While the output also contains more technical details, they fall beyond the scope of this introductory chapter on Linux networking fundamentals.

The following section of the output introduces another network connection known as `lo`, meaning loopback address or localhost. This software address shows connections to your own system, exclusively accessible by software and services running on your system. It's commonly represented by the IP address `127.0.1.1`.

The third connection, `wlan0`, appears only if you have a wireless interface or adapter. Note that it also reveals the MAC address of the wireless device (`HWaddr`).

The insights provided by `ifconfig` help you to connect to and manipulate your local area network (LAN) settings, a fundamental skill for hacking.

Examining Wi-Fi Devices with `iwconfig`

If you have a wireless adapter, the `iwconfig` command will really help you gathering information for wireless hacking. This information covers the adapter's IP address, MAC address, operational mode, and more, making it particularly useful when using wireless hacking tools like `aircrack-ng`.

Using the terminal, let's explore wireless devices with `iwconfig`.

```
kali > iwconfig
wlan0 IEEE 802.11bg ESSID:off/any
Mode:Managed Access Point: Not Associated Tx-Power=20 dBm

lo no wireless extensions
eth0 no wireless extensions
```

The output tells us that the only network interface featuring wireless extensions is wlan0, aligning with our expectations. Neither lo nor eth0 have any wireless extensions.

For wlan0, iwconfig shows the 802.11 IEEE wireless standards supported by the device—b and g, representing two early wireless communication standards. Modern wireless devices often incorporate n as well, the latest standard.

Furthermore, iwconfig reveals the mode of the wireless extension, marked as Mode:Managed in this instance, as opposed to monitor or promiscuous mode. Promiscuous mode is essential for cracking wireless passwords.

We also discern that the wireless adapter is not currently connected (Not Associated) to an access point (AP), and its signal strength is 20 dBm. Signal strength becomes significant for various applications, which we'll learn further in Chapter 14.

Modifying Network Information

The ability to modify your IP address and other network information is really valuable. It allows you to access other networks while appearing as a trusted device on those networks. For instance, in a denial-of-service (DoS) attack, you can spoof your IP address, making the attack seem to originate from a different source. This can help avoid IP capture during forensic analysis. This task is relatively straightforward in Linux and can be done using the ifconfig command.

IP Address Modification

To change your IP address, use the ifconfig command followed by the interface you wish to reassign and the new IP address you want to assign to that interface. For example, to assign the IP address 192.168.8.107 to interface eth0, enter the following command:

```
kali > sudo ifconfig eth0 192.168.8.107
```

When executed correctly, Linux will silently return the command prompt without any messages.

Afterward, when you check your network connections again using `ifconfig`, you will see that your IP address has been updated to the new one you assigned.

Adjusting Network Mask and Broadcast Address

You can also modify your network mask (netmask) and broadcast address using the `ifconfig` command. For instance, if you want to assign the `eth0` interface with a netmask of `255.255.8.0` and a broadcast address of `192.168.1.255`, input the following command:

```
kali > sudo ifconfig eth0 192.168.8.107 netmask 255.255.8.0 broadcast  
192.168.1.255
```

Once again, if you've executed the commands correctly, Linux will respond with a new command prompt. Now, run `ifconfig` again to confirm that each of the parameters has been updated accordingly.

Note here though that Kali might not change your broadcast address to this exact address as it must align with the host IP and the netmask address, if the broadcast IP doesn't align it will automatically change the part not aligning.

MAC Address Spoofing

You can also utilize `ifconfig` to change your MAC address (or `HWaddr`). The MAC address, which is globally unique, often serves as a security measure to protect networks from unauthorized access or to trace potential intruders. Changing your MAC address to spoof a different one is a straightforward technique that can neutralize these security measures, making it a valuable tool for bypassing network access controls.

To spoof your MAC address, follow these steps:

Use the `ifconfig` command with the `down` option to take down the interface, such as `eth0` in this case.

```
kali > sudo ifconfig eth0 down
```

Next, enter the `ifconfig` command followed by the interface name (`hw` for hardware or `ether` for Ethernet) and the new spoofed MAC address.

```
kali > sudo ifconfig eth0 hw ether 01:23:45:56:78:91
```

Finally, bring the interface back up with the `up` option for the changes to take effect.

```
kali > sudo ifconfig eth0 up
```

Upon inspecting your settings using `ifconfig` again, you should observe that the `HWaddr` has been updated to your newly spoofed MAC address.

Acquiring New IP Addresses via DHCP

Linux features a Dynamic Host Configuration Protocol (DHCP) server that runs a background process called `dhcpd` (`dhcp daemon`). The DHCP server assigns IP addresses to systems on the subnet and maintains logs of IP assignments. This functionality is valuable for forensic analysts tracking hackers after an attack.

Especially it just automatically configures IP addresses and network settings, to where you get a new IP address and change other info with one command, really minimizing manual configuration.

Typically, to connect to the internet from a LAN, you require a DHCP-assigned IP address. Also, after configuring a static IP address, you may need to obtain a new DHCP-assigned IP address. You can achieve this without rebooting your system. Instead, follow these steps:

Request a new IP address from DHCP by using the DHCP client with the `dhclient` command followed by the interface you want the address assigned to.

```
kali > dhclient eth0
```

The `dhclient` command sends a DHCPDISCOVER request from the specified network interface (e.g., `eth0`). It then receives a DHCPOFFER from the DHCP server (e.g., 192.168.128.121) and confirms the IP assignment to the DHCP server with a dhcp request.

Afterward, check your network settings using `ifconfig` to verify the changes.

```
kali > ifconfig
```

Depending on the DHCP server's configuration, the assigned IP address may differ. Upon executing `ifconfig`, you should see that the DHCP server has assigned a new IP address, a new broadcast address, and a new netmask to your network interface `eth0`.

Handling Domain Name System

When hacking, valuable information about a target can be found within its Domain Name System (DNS). DNS plays a critical role on the internet, primarily serving to

translate domain names into IP addresses. However, hackers can leverage DNS to gather intelligence about their targets.

Exploring DNS with dig

DNS plays a crucial role in translating domain names, like securityailab.com, into the corresponding IP addresses, making sure your system knows how to reach them. Without DNS, remembering thousands of IP addresses for favorite websites would be a daunting task.

For hackers, the dig command can help gather DNS information about a target domain. This information serves as a critical component for spying before launching an attack. It may include the IP address of the target's nameserver, the target's email server, and even details about subdomains and IP addresses.

To illustrate, entering the command `dig securityailab.com ns` with the ns option (short for nameserver) outputs the nameserver for securityailab.com, as shown below.

```
kali > dig securityailab.com ns
; <<>> DiG 9.18.7-1-Debian <<>> securityailab.com ns
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 36388
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;securityailab.com.                IN      NS

;; ANSWER SECTION:
securityailab.com.                300     IN      NS      bonnie.ns.cloudflare.com.
securityailab.com.                300     IN      NS      dexter.ns.cloudflare.com.
```

Additionally, you can use the dig command to gather information about email servers associated with a domain by including the mx option (mx stands for mail exchange server). This information is useful when targeting email systems..

```
kali > dig securityailab.com mx

; <<>> DiG 9.18.7-1-Debian <<>> securityailab.com mx
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49491
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;securityailab.com.          IN      MX

;; ANSWER SECTION:
securityailab.com.          300     IN      MX      0 securityailab.com.
```

The most prevalent Linux DNS server is the Berkeley Internet Name Domain (BIND). While some Linux users may interchangeably refer to DNS as BIND, it's essential to understand that DNS and BIND both serve the purpose of mapping individual domain names to IP addresses.

Switching DNS Servers

In certain situations, you might wish to use a different DNS server. To do so, you'll need to edit a plaintext file named `resolv.conf` on your system. Open this file in a text editor, such as vim, by executing a command like `kali > vim /etc/resolv.conf`.



```
File Actions Edit View Help
Generated by NetworkManager
nameserver 192.168.8.1
```

We see that the nameserver is set to a DNS server at 192.168.8.1. While this configuration works fine, if you intend to switch to Google's public DNS server at 8.8.8.8, you can simply add the following line to resolv.conf to specify the new nameserver: `nameserver 8.8.8.8`. Be sure to save the file afterward. Alternatively, you can achieve the same result from the command line using:

```
kali > echo "nameserver 8.8.8.8" > resolv.conf
```

This command replaces the current content of resolv.conf with the line specifying Google's DNS server. Your resolv.conf file should look like.



```
File Actions Edit View Help
# Generated by NetworkManager
nameserver 8.8.8.8
```

After inspecting resolv.conf, you'll notice that DNS requests are directed to Google's DNS server rather than the local one. Your system will now query the Google public DNS server to resolve domain names to IP addresses.

This might result in a slightly longer time for domain resolution, typically in milliseconds. To balance speed and flexibility, you can retain the local DNS server in resolv.conf and follow it with a public DNS server. The operating system queries each

DNS server in the order specified in resolv.conf, only resorting to the public DNS server if the domain name can't be found locally.

NOTE: If you are using a DHCP address and the DHCP server provides DNS settings, the DHCP server will replace the contents of the resolv.conf file when it renews the DHCP address.

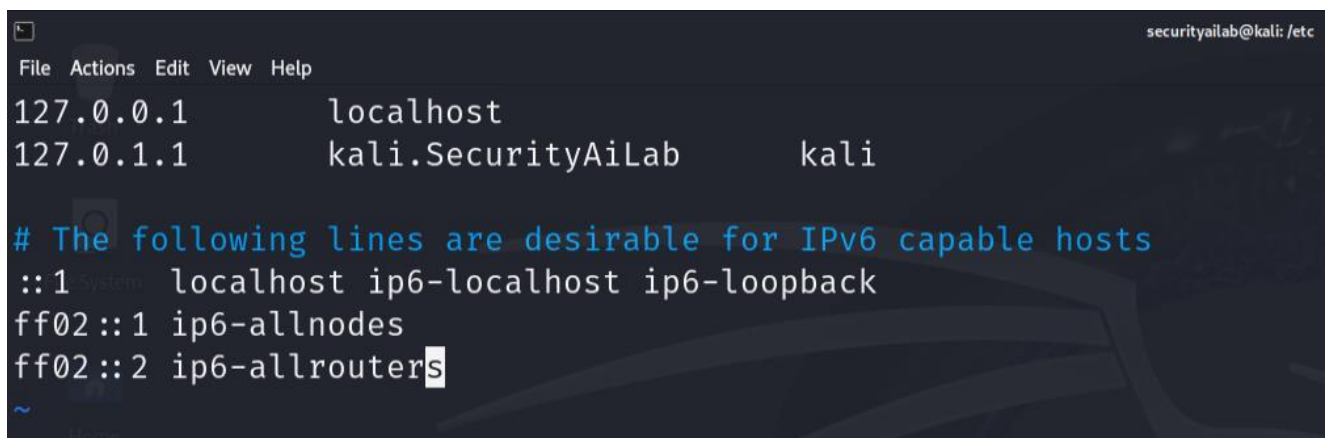
Custom IP Address Mapping

A special file on your system, known as the hosts file, plays a role similar to DNS (Domain Name System) in translating domain names to IP addresses. The hosts file can be found at /etc/hosts, and just like DNS, you can use it to specify your custom IP address-domain name mappings. Essentially, you can decide which IP address your browser should connect to when you enter a domain name like www.microsoft.com (or any other domain) into your browser, instead of leaving it to the DNS server's decision. As a hacker, this capability can be valuable for hijacking a TCP connection on your local network and redirecting traffic to a malicious web server, using tools such as dnsspoof.

To access the hosts file from the command line, enter the following command (you can replace vim with your preferred text editor e.g. nano):

```
kali > vim /etc/hosts
```

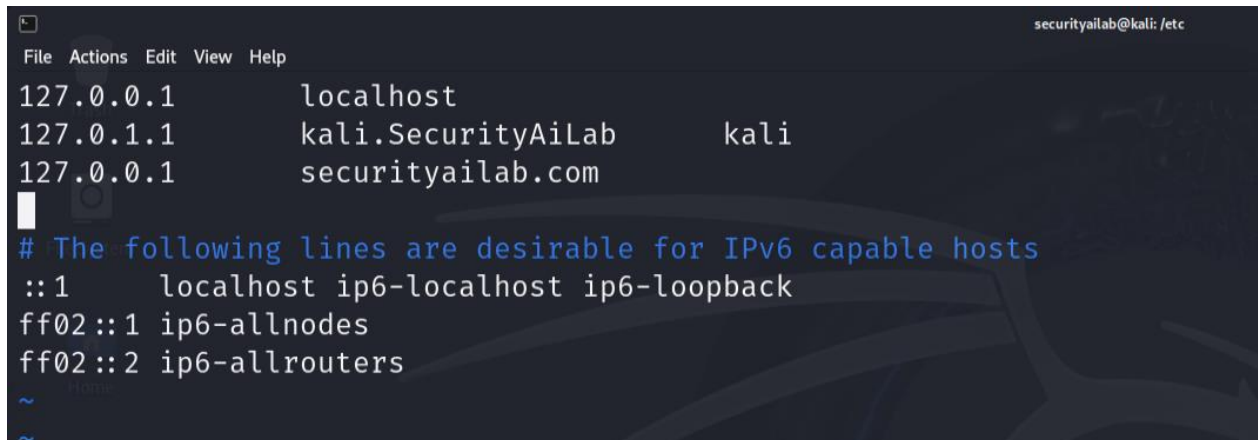
You will now see your hosts file

A screenshot of a terminal window with a dark background. The terminal shows the contents of the /etc/hosts file being edited with the nano text editor. The window title is 'securityailab@kali: /etc'. The menu bar shows 'File Actions Edit View Help'. The file content includes mappings for 127.0.0.1 to localhost and 127.0.1.1 to kali.SecurityAiLab kali. There is also a section for IPv6 with comments and mappings for ::1, ff02::1, and ff02::2. The cursor is at the end of the line 'ff02::2 ip6-allrouters'.

```
securityailab@kali: /etc
File Actions Edit View Help
127.0.0.1      localhost
127.0.1.1      kali.SecurityAiLab      kali

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

By default, the hosts file typically contains mappings only for your localhost (at 127.0.0.1) and your system's hostname (e.g., Kali.SecurityAiLab, at 127.0.1.1). However, you can add any IP address mapped to any domain you desire. As an example, you could map <https://securityailab.com/> to your local website at 127.0.0.1.



```
securityailab@kali: /etc
File Actions Edit View Help
127.0.0.1      localhost
127.0.1.1      kali.SecurityAiLab      kali
127.0.0.1      securityailab.com
# The following lines are desirable for IPv6 capable hosts
::1           localhost ip6-localhost ip6-loopback
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
~
~
```

Ensure that you press TAB between the IP address and the domain key, not the spacebar.

As you learn more about hacking and about tools like dnsspoof and Ettercap, you'll leverage the hosts file to direct traffic from your LAN that accesses <https://securityailab.com/> to your web server at 127.0.0.1. Thus maybe fishing your friends or clegues connected to your LAN.

Seems straightforward, doesn't it?

Chapter Summary

Every hacker requires fundamental Linux networking skills to connect, analyze, and manage networks. As you advance, these skills will prove increasingly valuable for reconnaissance, spoofing, and connecting to target systems.

Practice Exercises

Before proceeding to Chapter 4, apply the knowledge you've gained in this chapter by completing the following exercises:

1. Display the status of all network interfaces on your system.
2. Change the IP address on eth0 to 10.0.0.50.
3. Modify the MAC address on eth0 to AA:BB:CC:DD:EE
4. Check if lo is active and connected to a network.
5. Release and renew the DHCP lease for eth0.
6. Identify the DNS servers and mail servers for example.org.
7. Add Cloudflare's DNS 1.1.1.1 as a secondary DNS server in your /etc/resolv.conf file.

4. SOFTWARE INSTALLATION AND REMOVAL

In Linux, and indeed in any operating system, one of the most fundamental tasks is managing software—installing new applications and removing unwanted ones. Whether you're adding new functionality or freeing up disk space, understanding how to manipulate software is essential.

Software Packages and Dependencies

Often, software relies on other components to function correctly. To simplify the process, many Linux distributions use software packages. These packages are collections of files, typically including libraries and dependencies, required for a piece of software to run seamlessly. When you install a package, it ensures that all necessary files are installed together, along with scripts to facilitate software management.

In this chapter, we will look at three essential methods for handling software in Linux: using the apt package manager, employing GUI-based installation managers, and leveraging git.

Using apt for Software Management

For Debian-based Linux distributions, like Kali and Ubuntu, the default software manager is the Advanced Packaging Tool, apt, which primarily employs the apt-get command. However, as of the latest updates, you can simply use apt directly in place of apt-get for most tasks, which makes the commands easier to remember and type. This applies throughout this guide; you can replace any instance of apt-get with apt, and it will work as expected.

Locating a Software Package

Before downloading a software package, you can check its availability in your repository—a repository being a centralized location where your operating system stores essential information, like a vault. apt offers a straightforward search function to verify if a package exists in your repository. The syntax is as follows:

```
apt search keyword
```

By using the apt search command, you search the apt cache, which stores package names.

Keep in mind that if you run into any errors while running the apt command try using `sudo apt-get update`, this will update the packages on your Kali machine which might fix the issue.

For instance, if you're looking for the intrusion detection system Snort:

```
kali > apt search snort
```

```
fwsnort - Snort-to-iptables rule translator
```

```
ippl - IP protocols logger
```

```
snort - flexible Network Intrusion Detection System
```

```
snort-common - flexible Network Intrusion Detection System - common files
```

The search results will display numerous files containing the keyword "snort." Among these results, you'll find "snort – flexible Network Intrusion Detection System," which is the package we're interested in.

Installing New Software

Now that you've confirmed that the Snort package is available in your repository, you can use apt to download and install it. To install software from your operating system's default repository via the terminal, use the apt command, followed by "install" and the package name:

```
apt install packagename
```

Let's put this into action by installing Snort on your system. Enter the following command:

```
kali > sudo apt install snort
```

```
plaintext
```

Copy code

```
Reading package lists... Done
```

```
Building dependency tree
```

```
Reading state information... Done
```

```
Suggested packages:
```

```
    snort-doc
```

```
The following NEW packages will be installed:
```

```
    snort
```

```
0 upgraded, 1 newly installed, 0 to remove and 318 not upgraded.
```

```
Need to get 827 kB of archives.
```

```
After this operation, 1.3 MB of additional disk space will be used.
```

```
Do you want to continue? [Y/n]?
```

The output will provide details about what's being installed. If everything appears in order, type "y" when prompted, and the software installation will begin.

Uninstalling Existing Software

To uninstall software, use apt with the "remove" option, followed by the software's name. Here's an example:

```
kali > sudo apt remove snort
```

```
Reading package lists... Done
```

```
Building dependency tree
```

```
Reading state information... Done
```

```
The following packages were automatically installed and are no longer required:
```

```
libdaq2 oinkmaster snort-common-libraries snort-rules-default
```

```
Use 'apt autoremove' to remove them.
```

```
The following packages will be REMOVED:
```

```
snort
```

```
Do you want to continue? [Y/n]?
```

Similar to the installation process, you'll see real-time progress, and you'll be asked to confirm the removal. You can enter "y" to proceed with the uninstallation. Note that the "remove" command doesn't delete configuration files, allowing you to reinstall the same package without reconfiguring it in the future.

If you wish to remove both the software and its configuration files simultaneously, you can use the "purge" option, as demonstrated here:

```
kali > sudo apt purge snort
```

```
Reading package lists... Done
```

```
Building dependency tree
```

```
Reading state information... Done
```

```
The following packages will be REMOVED:
```

```
snort* libdaq2* oinkmaster* snort-common-libraries* snort-rules-default*
```

```
Do you want to continue? [Y/n]?
```

When prompted, enter "y" to continue with the removal of the software package and its associated configuration files.

It's worth noting that during the removal process, you may encounter the message "The following packages were automatically installed and are no longer required." This indicates that when you initially installed Snort, it brought along some dependencies or libraries necessary for its operation. Now, as you remove Snort, these additional components are no longer needed, so they are also removed to maintain a clean and modular system.

Regular Updates of Software Packages

Periodically, software repositories receive updates, including new software additions or newer versions of existing software. These updates, however, don't reach your system automatically; you must initiate the process. It's important to note that updating is distinct from upgrading: updating refreshes the list of available packages in the repository, while upgrading updates the software packages themselves.

To update your system, use the apt command with sudo followed by the "update" keyword. This command scans all packages on your system, checks for available updates, and downloads them.

```
kali > sudo apt update
```

```
Get:1 http://http.kali.org/kali kali-rolling InRelease [30.5 kB]
```

```
Get:2 http://http.kali.org/kali kali-rolling/main amd64 Packages [15.2 MB]
```

```
Get:3 http://http.kali.org/kali kali-rolling/contrib amd64 Packages [107 kB]
```

```
Fetched 15.3 MB in 1min 10s (218 kB/s)
```

```
Reading package lists... Done
```

The list of available software in your system's repository will be refreshed. If the update is successful, your terminal will display "Reading package lists... Done". Keep in mind that the repository's name and other values, such as time and size, may vary on your system. This command also often solves many errors you might encounter while using apt.

Upgrading Your System's Software

To upgrade the software packages already installed on your system, use `sudo apt upgrade`. As this process may introduce changes to your software, ensure you are logged in as the root user or use the `sudo` command before executing `apt upgrade`.

Executing this command upgrades every package on your system that apt recognizes, which includes packages stored in the repository. This process can be time-consuming, potentially rendering your system temporarily unavailable for other tasks. An example of upgrading packages is provided below:

```
kali > sudo apt upgrade
```

```
Reading package lists... Done
```

```
Building dependency tree... Done
```

```
Calculating upgrade... Done
```

```
The following packages were automatically installed and are no longer required:
```

```
The following packages will be upgraded:
```

```
libpython3.9 python3.9 python3.9-dev python3.9-minimal
```

```
1101 upgraded, 0 newly installed, 0 to remove and 318 not upgraded.
```

```
Need to get 827 MB of archives.
```

```
After this operation, 408 MB disk space will be freed.
```

```
Do you want to continue? [Y/n]?
```

The output will estimate the required hard drive space for the software packages. If you wish to proceed and have sufficient disk space, enter "y" when prompted.

Enhancing sources.list with Additional Repositories

Software repositories are servers housing software for specific Linux distributions. These repositories are tailored and configured for their respective distributions, making them incompatible with others. Although repositories may contain similar software, differences in versions or entirely different software may exist.

While you'll primarily use the Kali repository, which offers a wide array of security and hacking software, it may not cover all specialized tools or even some standard software. To address this, it's advisable to add backup repositories that your system can search through if a specific software is not found in the Kali repository.

The list of repositories that your system checks for software is stored in the sources.list file. You can edit this file to specify which repositories you want to use for downloading software. A common practice is to add Ubuntu repositories after Kali's repositories. This configuration ensures that when you request to download a new software package,

your system will first check the Kali repository and then the Ubuntu repository if the software isn't found in Kali's repository.

To locate and open the `sources.list` file, go to `/etc/apt/sources.list` and use any text editor to modify it. Here, Leafpad is used as an example. Run the following command in your terminal, replacing "leafpad" with the name of your preferred text editor (for example vim):

```
kali > sudo leafpad /etc/apt/sources.list
```

After entering this command, a window will appear, displaying Kali's default repositories.

[illegible]

Different Linux distributions categorize repositories into separate sections. For instance, Ubuntu categorizes its repositories as follows:

- **main**: Contains supported open-source software.
- **universe**: Contains community-maintained open-source software.
- **multiverse**: Contains software with copyright or legal restrictions.
- **restricted**: Contains proprietary device drivers.

- **backports:** Contains packages from later releases.

You can see an example output of Ubuntu's repositories (highlighted in purple).

```
GNU nano 6.2 /etc/apt/sources.list
# See http://help.ubuntu.com/community/UpgradeNotes for how to upgrade to
# newer versions of the distribution.
deb http://archive.ubuntu.com/ubuntu/ jammy restricted main
# deb-src http://archive.ubuntu.com/ubuntu/ jammy main restricted

## Major bug fix updates produced after the final release of the
## distribution.
deb http://archive.ubuntu.com/ubuntu/ jammy-updates restricted main
# deb-src http://archive.ubuntu.com/ubuntu/ jammy-updates main restricted

## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team. Also, please note that software in universe WILL NOT receive any
## review or updates from the Ubuntu security team.
# deb-src http://archive.ubuntu.com/ubuntu/ jammy universe
# deb-src http://archive.ubuntu.com/ubuntu/ jammy-updates universe

## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team, and may not be under a free licence. Please satisfy yourself as to
## your rights to use the software. Also, please note that software in
## multiverse WILL NOT receive any review or updates from the Ubuntu
## security team.
deb http://archive.ubuntu.com/ubuntu/ jammy multiverse
# deb-src http://archive.ubuntu.com/ubuntu/ jammy multiverse
deb http://archive.ubuntu.com/ubuntu/ jammy-updates multiverse
# deb-src http://archive.ubuntu.com/ubuntu/ jammy-updates multiverse

[ File '/etc/apt/sources.list' is unwritable ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^G Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-6 Copy
```

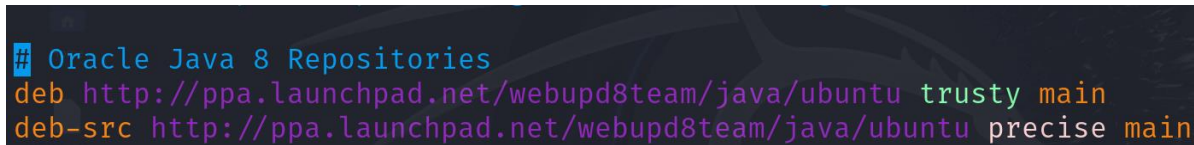
You shouldn't use testing, experimental, or unstable repositories in your sources.list, as they might introduce problematic software to your system. Such software, which isn't thoroughly tested, can break your system.

When you download a new software package, the system searches through the repositories listed in sources.list and stops when it finds the package you need. Prioritize checking the compatibility of the repository with your system. Because both Kali and Ubuntu are built on Debian, these repositories tend to work well with each other.

To add a repository, edit the sources.list file by appending the repository's name to the list and then save the file. For example, if you want to install Oracle Java 8 on Kali, you can add the WebUpd8 repository to access the required package. As of the time of writing, you would need to add the following repository locations to sources.list to include the necessary repositories:

```
deb http://ppa.launchpad.net/webupd8team/java/ubuntu trusty main
```

```
deb-src http://ppa.launchpad.net/webupd8team/java/ubuntu precise main
```

A terminal window with a dark background and light blue text. The text shows the command to add the Oracle Java 8 repository to the system's sources list.

```
## Oracle Java 8 Repositories
deb http://ppa.launchpad.net/webupd8team/java/ubuntu trusty main
deb-src http://ppa.launchpad.net/webupd8team/java/ubuntu precise main
```

Installing Applications via GUI

In more recent versions of Kali Linux, you won't find a GUI-based software installation tool by default, but you can easily install one using the apt command. Two commonly used GUI-based installation tools are Synaptic and Gdebi. Let's install Synaptic and use it to install our Snort package:

```
kali > sudo apt install synaptic
```

```
Reading package lists... Done
```

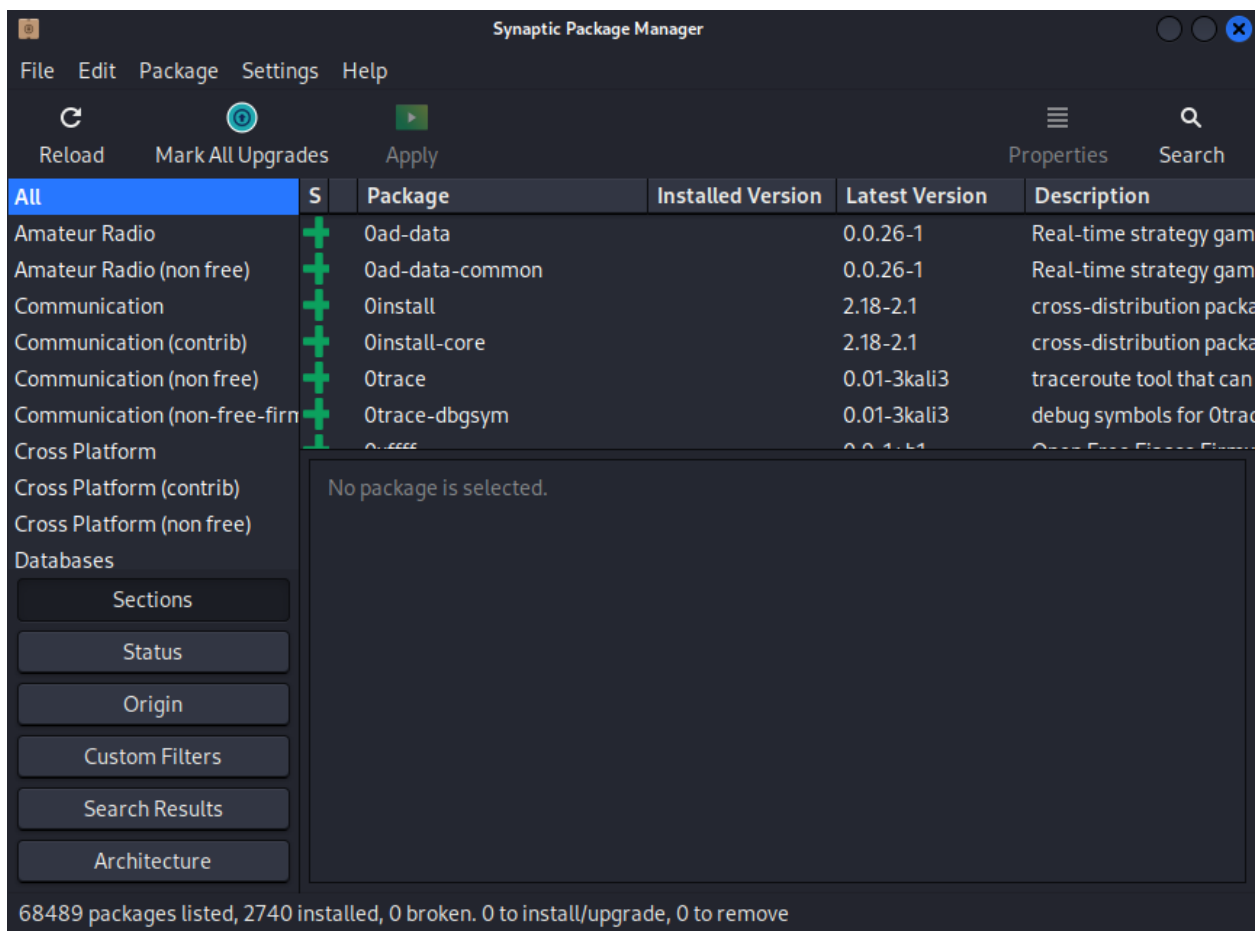
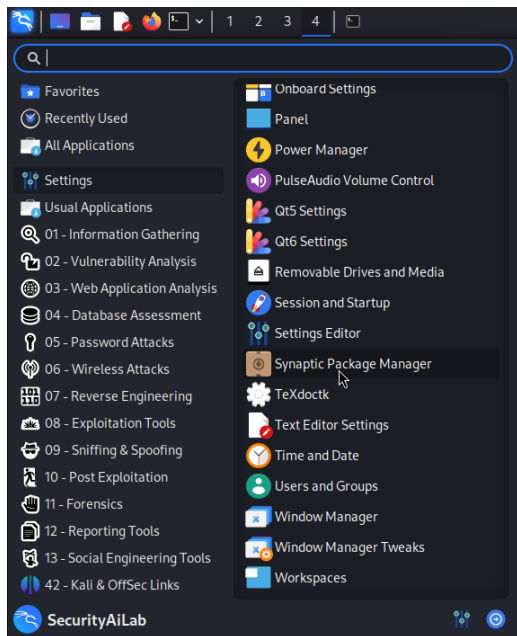
```
Building dependency tree
```

```
Reading state information... Done
```

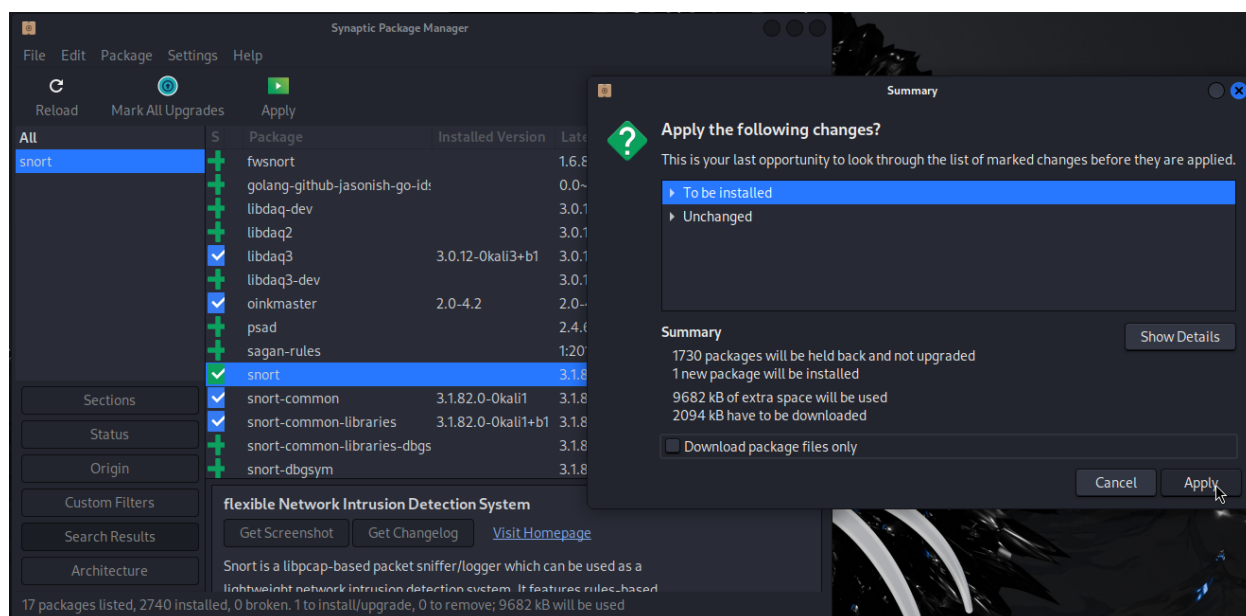
```
Processing triggers for menu (2.1.47)...
```

If you run into any errors while executing apt install just update your packages through `sudo apt-get update`.

After executing this command, you'll see output indicating the progress of the installation process. Once Synaptic is installed, you can launch it from "Settings" ► "Synaptic Package Manager." This will open a window.



Now, you can search for the package you need. Click on the "Search" tab to open a search window. Since we're searching for Snort again, enter "snort" into the search field and click "Search." Scroll through the search results until you find the Snort package. Double click on "snort" and then click the "Apply" tab. Synaptic will download and install Snort from the repository along with any necessary dependencies.



Deploying Software from git Repositories

Sometimes, the software you need might not be available in any of the standard repositories, especially if it's brand new. However, you might find it on GitHub (<https://www.github.com/>), a platform where developers share software for others to download, use, and develop. For example, if you're looking for "bluediving," a Bluetooth hacking and pentesting suite, and it's not in the Kali repository, you can search for it on GitHub by entering "bluediving" in the search bar.

Once you've located the software on GitHub, you can install it from the terminal using the git clone command followed by its GitHub URL. For instance, if "bluediving" is located at <https://www.github.com/balle/bluediving.git>, you can clone it to your system with the command below:

```
kali > git clone https://www.github.com/balle/bluediving.git
```

```
Cloning into 'bluediving'...
```

```
remote: Counting objects: 131, done.
```

```
remote: Total 131 (delta 0), reused 0 (delta 0), pack-reused 131
```

```
Receiving objects: 100% (131/131), 900.81 KiB | 646.00 KiB/s, done.
```

```
Resolving deltas: 100% (9/9), done.
```

```
Checking connectivity... done.
```

The git clone command copies all the data and files from the GitHub folder to your system. You can verify the successful download by using the long listing command `ls -l`:

```
kali > ls -l
```

If the cloning process was successful, you should see output similar to the following, confirming that "bluediving" has been cloned to your system and a new directory for its files has been created:

```
total 80
```

```
drwxr-xr-x 7 root root 3046 Oct 10 21:03 bluediving
```

```
drwxr-xr-x 2 root root 3046 Jun 8 9:17 Desktop
```

```
drwxr-xr-x 2 root root 3046 Jun 8 9:17 Documents
```

```
drwxr-xr-x 2 root root 3046 Jun 8 9:17 Downloads
```

```
drwxr-xr-x 2 root root 3046 Jun 8 9:17 Music
```

As shown, the "bluediving" files are successfully cloned to your system. Note that bluediving isn't just a file but a directory of files.

Chapter Summary

In this chapter, we explored several methods for downloading and installing new software on your Linux system. Software package managers like apt, GUI-based

installers such as Synaptic, and cloning software repositories using git. You'll use these skills daily working as a penetration tester.

Practice Exercises

Before proceeding to Chapter 5, I recommend testing your skills from this chapter by completing this exercises:

1. Install a Specific Software Package:

- Use the apt package manager to install a specific software package, such as nmap, from the Kali repository. Execute the following command: `sudo apt install nmap`
- Verify that nmap is installed correctly by running: `nmap --version`

2. Remove a Specific Software Package:

- After verifying the installation, uninstall nmap using the following command: `sudo apt remove nmap`
- Confirm the removal by attempting to run nmap again: `nmap --version`

3. Update the Repository List:

- Update the package lists for the Kali repository to ensure you have the latest information: `sudo apt update`
- After updating, install a software package such as htop to confirm that the update worked: `sudo apt install htop`

4. Upgrade Installed Software Packages:

- Upgrade all existing software packages on your system: `sudo apt upgrade`
- If prompted, review the list of packages to be upgraded and proceed by entering y.

5. Add a New Repository and Install Software from It:

- Open the sources.list file and add a new repository, such as the WebUpd8 team repository for Oracle Java 8: `sudo vim /etc/apt/sources.list`
- Add the following lines to the file:
`deb http://ppa.launchpad.net/webupd8team/java/ubuntu trusty main`
`deb-src http://ppa.launchpad.net/webupd8team/java/ubuntu trusty main`
- Save the file and update your package list again: `sudo apt update`
- Install a package from the newly added repository, such as oracle-java8-installer: `sudo apt install oracle-java8-installer`

6. Clone a GitHub Repository:

- Choose a new software project from GitHub, such as dirsearch, a directory brute-forcing tool: `git clone https://github.com/maurosoria/dirsearch.git`
- After cloning, navigate into the directory and list its contents: `cd dirsearch` then `ls -l`
- Run the tool to ensure it was cloned successfully: `python3 dirsearch.py -h`

5. MANAGING PERMISSIONS FOR FILES AND DIRECTORIES

In Linux, not everyone should have the same level of access to files and directories. That's why Linux has built-in ways to manage and secure access. The system administrator—usually the root user—or the file's owner can control who gets to read, write, or execute a file. These permissions can be set specifically for the file's owner, certain user groups, and everyone else. This control is especially important in a multiuser environment, like an enterprise system, where proper permissions can prevent a lot of headaches.

In this chapter, I'll walk you through how to check and adjust permissions for files and directories so that only the right people have access. We'll also cover how to set default permissions and look into special permissions. Plus, we'll explore how hackers might try to exploit permission settings to gain unauthorized access to a system.

User Types in Linux

As you probably know, in Linux, the root user is the one with the most power and control over the system. He user has almost unlimited access to everything on the system. On the other hand, other users usually have much more limited access and permissions compared to the root user.

These regular users are often organized into groups based on what they do. For example, in a business, you might have groups for finance, engineering, sales, and so on. In IT, you might see groups like developers, network administrators, or database administrators. The idea behind these groups is to make it easier to manage permissions—by assigning permissions to a group, everyone in that group automatically gets those permissions. This not only makes things simpler but also helps keep the system secure.

By default, the root user belongs to the root group, similar to an owner or Administrator group. When you add a new user to the system, they need to be assigned to a group to inherit that group's permissions.

Setting Access Permissions

Every file and directory must be assigned a particular level of permission for different users or groups. There are three primary levels of permission:

Read (r): This permission allows users to open and view a file.

Write (w): This permission permits users to view and edit a file.

Execute (x): This permission enables users to execute a file(e.g. running it), though not necessarily view or edit it.

With these permissions, the root user can allocate the appropriate level of access to users based on their requirements. When a file is created, the user who creates it becomes the file's owner, and the owning group is typically the user's current group. The owner can then grant various access privileges. Let's explore how to change permissions and transfer ownership to individual users and groups.

Assigning Ownership to a User

To transfer ownership of a file to a different user, allowing them to control permissions, you can use the "chown" (change owner) command. For instance:

```
kali > sudo chown bob /tmp/bobsfile
```

In this command, we specify the username ("bob") to whom we're transferring ownership and the location and name of the file ("bobsfile"). This action grants Bob ownership of "bobsfile."

If you try to do the same and don't have a user called bob you can add him by writing:

```
kali > sudo adduser bob
```

Group Ownership Assignment

To transfer ownership of a file from one group to another, you can use the "chgrp" (change group) command. This can be valuable when multiple hackers or pentesters are collaborating on a project. For example:

```
kali > sudo chgrp security newIDS
```

In this command, we assign ownership of "newIDS" to the "security" group. Now, let's learn how to check if these ownership changes and permission allocations have been successful.

If you try to do the same and don't have a group called security you can add it by writing:

```
kali > sudo groupadd security
```

Reviewing Current Permissions

To determine the permissions assigned to users for a file or directory, use the "ls" command with the "-l" (long) switch. This will display the contents of a directory in a detailed format, including permissions. For example:

```
kali > ls -l usrshare/hashcat
```

File Mode	Owner	Group	File size (bytes)	Last modified date/time	File/directory name
drwxr-xr-x	3	root	root	4096 Jul 20 2016	acpi
-rw-r--r--	1	root	root	3028 Apr 21 2016	adduser.conf
drwxr-xr-x	2	root	root	4096 Mar 16 07:14	alternatives
drwxr-xr-x	3	root	root	4096 Mar 4 10:59	apache2
drwxr-xr-x	3	root	root	4096 Jul 20 2016	apm
drwxr-xr-x	3	root	root	4096 Mar 13 15:04	apparmor
drwxr-xr-x	8	root	root	4096 Mar 16 07:14	apparmor.d
drwxr-xr-x	3	root	root	4096 Feb 4 16:51	appport
drwxr-xr-x	6	root	root	4096 Mar 13 07:48	apt
-rw-r-----	1	root	daemon	144 Jan 14 2016	at.deny
-rw-r--r--	1	root	root	2188 Sep 1 2015	bash.bashrc
-rw-r--r--	1	root	root	45 Aug 12 2015	bash_completion
drwxr-xr-x	2	root	root	4096 Feb 25 23:18	bash_completion.d

The output will contain information about the file type, permissions for the owner, group, and other users, the number of links, file owner, file size, creation or modification date, and the file name. Focus on the seemingly cryptic strings of letters and dashes on the left edge of each line, which represent permissions.

The first character indicates the file type, where "d" signifies a directory and a dash "-" indicates a file. The next section consists of three sets of three characters, representing permissions for the owner, group, and other users, respectively. Each set consists of "r" for read, "w" for write, and "x" for execute permissions. A dash "-" indicates the absence

of a specific permission. Users can have permission to execute either binaries or scripts.

For instance, a line like "-rw-r--r--" indicates that the owner has read and write permissions, the group has read permission but not write or execute permissions, and other users also have only read permission. These permissions are not immutable; as a root user or file owner, you can modify them.

Modifying Access Rights

To modify permissions of files and directories, you can use the "chmod" (change mode) command. Only the root user or the file's owner has the authority to change permissions. In the upcoming sections, we'll explore two methods for changing permissions: using numerical representations and symbolic representations.

Using Numerical Codes for Permission Changes

A convenient shortcut for indicating permissions is to use a single number to represent a set of rwx permissions. Beneath the surface, permissions in the operating system are represented in binary, where ON and OFF switches are represented by 1 and 0. Visualize rwx permissions as three ON/OFF switches, so when all permissions are granted, it's equivalent to 111 in binary translating to ON ON ON.

You can easily condense a this set of binary digits into a single digit by converting it into an octal, which is an eight-digit number system ranging from 0 to 7. In this system, each octal digit is a set of three binary digits. This allows us to express an entire set of read, write, and execute (rwx) permissions with just one digit(e.g. 777 instead of 111 111 111). Below is a table, which shows all possible permission combinations along with their corresponding octal and binary versions:

Binary	Octal	rwx
000	0	---
001	1	--x
010	2	-w-
011	3	-wx
100	4	r--
101	5	r-x
110	6	rw-
111	7	rwX

Now, let's work through some examples. If we wish to set only the read permission, we refer to the table above and find the value for read, which is "4" as an octal, 100 as a binary, and `r--` as human readable permissions.

Next, if we want to set the permission to "`-wx`", we apply the same approach, identifying what sets the "w" and "x" permissions, resulting in "`- w x = 0 2 1`". Notably, the octal representation for "`-wx`" is 3, the sum of the individual values for "w" and "x": $2 + 1 = 3$.

Finally, when all three permissions are active, it appears as "`rwX = 4 2 1`", and the sum of $4 + 2 + 1$ equals 7. Therefore, in Linux, when all permission switches are on, they are represented by the octal value 7.

To represent all permissions for the owner, group, and all users, it can be written as "`7 7 7`". Here's where the shortcut proves valuable. By using the "`chmod`" command followed by three octal digits (one for each `rwX` set) and a filename, we can modify permissions for each type of user. For example:

```
kali > chmod 774 hashcat.hcstat
drwxr-xr-x 5 root root 4096 Dec 5 10:47 charsets
-rwxrwxr-- 1 root root 33685504 June 28 2024 hashcat.hcstat
-rw-r--r-- 1 root root 33685504 June 28 2024 hashcat.hctune
drwxr-xr-x 2 root root 4096 Dec 5 10:47 masks
drwxr-xr-x 2 root root 4096 Dec 5 10:47 OpenCL
```

Referring to the above table, this command grants all permissions to the owner and the group, while everyone else (other) only receives read permission.

To confirm whether these permissions have changed, you can run "`ls -l`" on the directory and inspect the "`hashcat.hcstat`" line.

Permission Adjustments with UGO

While the numeric method is common for changing permissions in Linux, some prefer the symbolic method provided by "`chmod`," which is equally effective. This method is often called the UGO syntax, representing User, Group, and Others.

The UGO syntax is straightforward. Use the "chmod" command, followed by the users you want to modify permissions for (u for User, g for Group, o for Others), and one of three operators: "-" to remove a permission, "+" to add a permission, and "=" to set a permission. After the operator, specify the permission (rwx) and the filename.

For instance, to remove write permission from the user who owns the "hashcat.hcstat" file:

```
kali > chmod u-w hashcat.hcstat
```

```
drwxr-xr-x 5 root root 4096 Dec 5 10:47 charsets
-r-xr-xr-- 1 root root 33685504 June 28 2024 hashcat.hcstat
-rw-r--r-- 1 root root 33685504 June 28 2024 hashcat.hctune
drwxr-xr-x 2 root root 4096 Dec 5 10:47 masks
drwxr-xr-x 2 root root 4096 Dec 5 10:47 OpenCL
```

This command removes ("-") the write ("w") permission for the user ("u") from "hashcat.hcstat."

After executing this command, you can check the permissions using "ls -l" and observe the changes.

Root Execution Permissions for New Tools

As a hacker, you often need to download new hacking tools. However, Linux assigns default permissions of 666 for files and 777 for directories. This means that by default, you won't be able to execute a downloaded file and may encounter a "Permission denied" message. In such cases, you must grant yourself root and execute permissions using "chmod" to execute the file.

For instance, if you download a new hacker tool named "newhackertool" and place it in the root user's directory (/), you can observe the permissions:

```
kali > ls -l
```

```
total 80
-rw-r--r-- 1 root root 1072 Jun 5 11.17 newhackertool
```

Here, "newhackertool" lacks execute permission for anyone. To grant yourself execute permission, you can use the following command:

```
kali > chmod 766 newhackertool
```

Now, when you list the directory using "ls -l," you'll see that "newhackertool" has execute permission for the owner:

```
kali > ls -l
total 80
-rwxrw-rw- 1 root root 1072 Dec 5 11.17 newhackertool
```

This grants you (the owner) all permissions, including execute, while the group and others have read and write permissions ($4 + 2 = 6$).

Default Permission Settings with Masks

As previously discussed, Linux automatically assigns base permissions, typically 666 for files and 777 for directories. However, you have the capability to alter the default permissions assigned to files and directories created by each user using the umask method. The umask method involves specifying the permissions you want to remove from the base permissions on a file or directory, thus enhancing security.

The umask is a three-digit octal number corresponding to the three permission digits. This number is subtracted from the default permissions to determine the new permissions status. Essentially, when a new file or directory is created, its permissions are set to the default value minus the umask value, as illustrated below:

New files	New directories
666	777
-022	-022
644	755

For instance, when the umask is set to 022, a new file with the original default permissions of 666 will now have permissions of $666 - 022 = 644$. This means the owner has read and write permissions, while the group and all other users have only read permission.

In Kali Linux, like most Debian-based systems, the default umask is preconfigured to 022, resulting in default permissions of 644 for files and 755 for directories.

It's important to note that the umask value is not universal for all users on the system. Each user can set a personal default umask value in their personal `.profile` or `.bashrc` file. To view the current umask value when logged in as a user, simply use the `umask` command.

To change the umask value for a user, just write `umask [level]` for example

```
umask 004
```

Keep in mind this will only last in your current terminal session, in some version of Kali you could modify it permanently through the file `~/.profile` or the file `~/.bashrc` for your current user.

Extra Permissions

In addition to the standard "read," "write," and "execute" permissions (rwx), Linux features three special permissions that are slightly more complex: Set User ID (SUID),

Set Group ID (SGID), and the Sticky Bit. Each of these will be discussed in detail in the following sections.

Temporary Root Access with SUID

As you already know, a user can execute a file only if they have the necessary execution permission. However, some files require root user privileges during execution, even for non-root users.

In such cases, you can temporarily grant the owner's execution privileges to the file by setting the SUID bit on the program. Essentially, the SUID bit allows any user to execute the file with the permissions of the owner, but these permissions do not extend beyond the use of that file. So this is a way to give root permissions to someone but only for 1 file. You don't make him a root user, you just give him access to the file.

To set the SUID bit, you add a 4 before the regular permissions. For instance, a file with resulting permissions of 644 is represented as 4644 when the SUID bit is set. While setting the SUID bit is not a common task for typical users, you can use the command

```
chmod 4644 filename
```

Now any user will be able to execute this file, even though he isn't a root user. Just like the when running the passwd command used for changing passwords, even though it is located in the /usr/bin/passwd and is owned by the root user you'll still be able to run the command because it's permissions are with SUID on the place of Execute:

```
ls -l /usr/bin/passwd
```

```
-rwsr-xr-x 1 root root 68248 Oct  4 2024 /usr/bin/passwd
```

Setting SUID(and SGID) is risky as you're allowing anybody to execute the file, which might in turn modify the file

Group Permission with SGID

The SGID bit also provides temporary elevated permissions but grants the permissions of the file owner's group, rather than the file's owner. This means that, with the SGID bit set, a user even without elevated permission can execute a file if he is part of this group.

When applied to a directory, the SGID bit functions differently. If set on a directory, ownership of new files created in that directory is assigned to the group of the directory, not the file creator's group.

For example, if where both the red_team and blue_team groups create files within a directory named "Security." Normally, the files would have 'group ownership' corresponding to the group of the user who created them—either red_team or blue_team. However, if the "Security" directory has the SGID bit set and the directory's group is Security, then any files created within it by members of either team will automatically have their group ownership set to Security, not red_team or blue_team.

This is particularly useful when multiple users share a directory, as all users in that group can execute the files, not just a single user.

The SGID bit is represented as 2 before the regular permissions. For example, a new file with resulting permissions of 644 would be represented as 2644 when the SGID bit is set. To set the SGID bit, you can use the command

```
chmod 2644 filename
```

Sticky Bit Explained

Lastly, there's the Sticky Bit, a permission bit that can be set on a directory to allow only the owner of a file within that directory (or the root user) to delete or rename the file. While the Sticky Bit was originally a legacy feature from older Unix systems, modern Linux systems still support it, and it's often used in directories like /tmp to prevent users from deleting or renaming each other's files.

To set the Sticky Bit on a directory, add a 1 before the directory permissions. For example, to set the Sticky Bit on a directory with 755 permissions, you would use the command

```
chmod 1755 directory_name
```

I know it might seem confusing at first to understand who gets which permissions and how it works. Keep in mind that whenever you use the chmod command, it changes the permissions of a file or directory, and these permissions apply to the owner, the group, and others. So if you create a file and then use chmod to change its permissions, you

are modifying how you, as the owner, as well as the group and others, can access that file. Remember the image below:

Owner	Group	Other
rwx	r-x	r-x
$4+2+1$	$4+0+1$	$4+0+1$
7	5	5

Special Permissions in Hacking Context

As a hacker, your knowledge of special permissions can be leveraged to exploit Linux systems through privilege escalation. Privilege escalation involves a regular user gaining root or sysadmin privileges, which come with extensive permissions, essentially allowing you to do anything on the system.

One method to achieve privilege escalation is by exploiting the SUID bit. System administrators or software developers may set the SUID bit on a program, granting that program access to files with root privileges. For example, scripts that need to change passwords often have the SUID bit set. As a hacker, you can utilize this permission to temporarily gain root privileges and potentially engage in malicious activities, such as accessing password files like `/etc/shadow`.

Let's find files with the SUID bit set on our Kali system. If you recall from Chapter 1, we introduced the `find` command, a powerful tool with somewhat complex syntax. If needed, take a moment to review the `find` syntax in Chapter 1.

In this case, our objective is to locate files anywhere on the filesystem owned by the root user or other sysadmins, with the permissions set to 4000. We can accomplish this using the command

```
find / -user root -perm -4000
```

With this command, we instruct Kali to initiate the search from the root directory (`/`). It then traverses through the entire filesystem hierarchy looking for files owned by the root user (specified with `-user root`, you can modify this to `-user bob` for example) and possessing the SUID permission bit set (`-perm -4000`).

Executing this command reveals a multitude of files with the SUID bit set.

```
/usr/lib/mysql/plugin/auth_pam_tool_dir/auth_pam_tool
```

```
/usr/lib/xorg/Xorg.wrap
```

```
/usr/lib/polkit-1/polkit-agent-helper-1
```

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

```
/usr/lib/openssh/ssh-keysign
```

Let's navigate to the /usr/bin directory, where many of these files are located, and perform a long listing to examine the sudo file, as demonstrated in the code below

Change directories to /usr/bin with `cd /usr/bin`, then list the files with `ls -l`. You might see something like:

```
-rwxr-xr-x 1 root root 176272 Jul 18 2024 stunnel4
```

```
-rwxr-xr-x 1 root root 26696 Mar 17 2024 sucrack
```

```
-rwsr-xr-x 1 root root 140944 Jul 5 2024 sudo
```

In the listing, observe the permissions for the owner of the sudo file, indicated as an **S** in place of the **x**. This **s** signifies that the SUID bit is set. Consequently, anyone who runs the 'sudo' file inherits the privileges of the root user. This scenario poses a potential security risk for system administrators and a tempting attack vector for hackers. Some applications require access to the /etc/shadow file to perform their tasks. If an attacker can compromise such an application, they gain access to the Linux system's passwords.

Chapter Summary

Linux's use of permissions for user's or group's files and directories from other users serves both offensive and defensive purposes. You now have the knowledge needed to manage these permissions effectively and exploit potential vulnerabilities in the security system, particularly focusing on SUID and SGID bits.

Practice Exercises

Before advancing to Chapter 6, put your newfound knowledge from this chapter to the test with the following exercises:

Select a directory:

- Choose the `/var/log` directory and execute a long listing (`ls -l /var/log`).
- Take note of the permissions assigned to each file and subdirectory within `/var/log`.

Grant Execute Permissions:

- Identify the file `/var/log/syslog`, which you likely don't have execute permission for (you can check this with `ls -l /var/log/syslog`).
- First, grant yourself execute permissions using the numerical method: `chmod 744 /var/log/syslog`.
- Then, revert it to its original state and try the UGO method: `chmod u+x /var/log/syslog`.

Modify File Ownership:

- Choose the file `/var/log/dpkg.log`.
- Modify its ownership using the `chown` command to change the owner to your user account: `sudo chown [your-username] /var/log/dpkg.log`.

Locate Files with the SGID Bit Set:

- Use the `find` command to locate all files with the SGID bit set in the `/usr/bin` directory: `find /usr/bin -perm /2000`.

6. OVERSEEING SYSTEM PROCESSES

In any Linux system, there are typically numerous processes running simultaneously, ranging from a few hundred to even thousands. A process, in simple terms, is a running program that consumes system resources like CPU, GPT, RAM, etc. These processes

consist of a wide range of functions, including terminals, web servers, executing commands, managing databases, running GUI interfaces, and much more.

Understanding how to effectively manage these processes is important for hacking. For instance, when a hacker gains control of a system, they might identify and terminate specific processes, such as antivirus applications or firewalls. To achieve this, the hacker must first learn how to locate the target process. Additionally, hackers might want to schedule scanning scripts to run at specific intervals to identify vulnerable systems, which we will also cover in this chapter.

In this chapter, you will begin by learning how to view and identify running processes and how to determine which processes are consuming the most system resources. Additionally, you will learn process management techniques, like running processes in the background, setting process priorities, and terminating processes. Finally, you will schedule processes to run on specified days, dates, and at particular times.

Exploring Active Processes

In most scenarios, the initial step in process management is to observe the active processes on your system. To do this you'll use the often regarded as a Linux administrator's best friend, the "ps" command. Executing this command in your terminal will show the active processes:

```
kali > ps
```

The output will show the Process ID (PID), terminal associated with the process (TTY), cumulative execution time (TIME), and the command that initiated the process (CMD). For example:

```
PID TTY TIME CMD
```

```
39659 pts/0 00:00:01 bash
```

```
39665 pts/0 00:00:00 ps
```

The Linux kernel, which serves as the core of the operating system, assigns a unique Process ID (PID) to each process sequentially as they are created to differentiate them. When working with these processes in Linux, it is crucial to remember their PIDs, as they are more significant than the process names.

By itself, the "ps" command provides minimal information. Running "ps" without any options displays a list of processes initiated by the currently logged-in user (in our case, "root") and the processes running on that terminal. The output will typically show that the "bash" shell is active and running, in addition to the fact that the "ps" command itself was executed. However, we often require more detailed information, especially regarding processes initiated by other users and system background processes. Without this information we know only limited info about the system processes.

To obtain a more detailed view, you can use the "ps" command with the "aux" options. This command will display all processes running on the system, not only the ones the user has initiated:

```
kali > ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.1	165808	1332	?	Ss	10:00	0:02	/sbin/init
root	2	0.0	0.0	0	0	?	S	10:00	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	I<	10:00	0:00	[rcu_gp]
root	4	0.0	0.0	0	0	?	I<	10:00	0:00	[rcu_par_gp]
root	4587	0.0	0.0	11320	916	pts/0	R+	10:04	0:00	ps aux

In the output, you'll notice an extensive list of processes, which may extend beyond the visible screen. The first process, "init," is listed in the final column, and the last process is the "ps aux" command itself. The most crucial columns in this output are:

- **USER:** Indicates the user who initiated the process.
- **PID:** Represents the unique Process ID.
- **%CPU:** Specifies the percentage of CPU resources consumed by the process.
- **%MEM:** Indicates the percentage of RAM memory used by the process.
- **COMMAND:** Displays the name of the command that initiated the process.

In general, to interact with a specific process, you will need to reference its PID. Let's proceed to explore how to leverage this identifier for various actions.

Isolating Processes by Name

When dealing with processes, it's often overwhelming to see a list of all the processes on the screen. Most of the time, you're interested in finding information about a specific process. To achieve this, we can utilize the filtering capabilities of the "grep" command, as introduced in Chapter 1.

For example, to check if the Metasploit framework is running, you would start Metasploit with the following command:

```
kali > msfconsole
```

Once the framework is running, check if it appears in the list of running processes:

```
kali > ps aux | grep msfconsole
```

In the filtered output, you will see all the processes that match the term "msfconsole." This includes the PostgreSQL database used by Metasploit and the "msfconsole" program itself. The output might look like this:

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	39756	0.0	0.0	4304	716	pts/2	Ss+	15:13	0:00	sh -c service postgresql start && msfdb init & msfconsole
root	39759	35.1	15.2	4304	227888	pts/2	Sl+	15:13	1:36	ruby usrbin/msfconsole
root	39892	0.0	0.0	4304	940	pts/2	S+	15:18	0:00	grep msfconsole

From this, you can extract valuable information. For example, if you need to determine the resource usage of Metasploit, you can refer to the %CPU and %MEM columns, which indicate how much CPU and memory the process is using.

Identifying Resource-Intensive Processes with top

The "top" command provides a real-time view of processes ordered by resource usage. This command updates automatically, usually every 3 seconds, and is useful for identifying which processes are consuming the most resources:

```
kali > top
```

```
top - 15:31:17 up 2 days,  4:50,  4 users,  load average: 0.00, 0.04, 0.09
```

```
Tasks: 176 total,   1 running, 175 sleeping,   0 stopped,   0 zombie
```

```
%Cpu(s):  1.3 us,   0.7 sy,   0.0 ni, 97.4 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
```

```
KiB Mem : 1491220 total,   64848 free,   488272 used,   938100 buff/cache
```

```
KiB Swap: 1046524 total, 1044356 free,    2168 used.  784476 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
39759	root	20	0	893180	247232	11488	S	0.7	16.6	1:47.88	ruby
39859	root	20	0	27308	16796	14272	S	0.3	1.2	0:01.38	postgres
39933	root	20	0	293936	61500	29108	S	0.7	4.1	0:23.72	Xorg

The output provides information about the system's overall status, including uptime, number of tasks, CPU usage breakdown, and memory statistics. The processes are listed with their PIDs, owners, resource usage percentages, and more. System administrators often keep "top" running in a terminal to monitor resource utilization.

While "top" is running, you can press the 'H' or '?' key to access interactive commands and 'Q' to quit "top." We will revisit "top" shortly in the sections on changing process priority and killing processes.

If you want to filter for a specific process you can press "o" then either write PID=[the process ID] or COMMAND=[the process name].

And if you want to increase/decrease the time it takes for top to update press "d" then the number of seconds it should delay each update (e.g. 60).

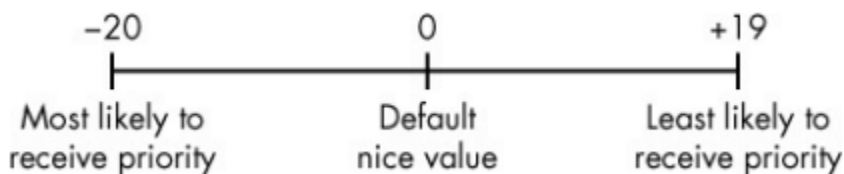
Process Control Techniques

Hackers often engage in multi-process activities, and an operating system like Kali is well-suited for such tasks. A hacker might run a port scanner, vulnerability scanner, and an exploit simultaneously. Efficiently managing these processes is essential to optimize system resources and accomplish tasks effectively. In this section, I will guide you through the process of managing multiple processes.

Modifying Process Priority with nice

The "nice" command influences a process's priority, **when you start the process**. This means that the process hasn't been activated yet, and by using "nice," you are both starting the process and defining its priority at the same time. Nice values range from -20 to +19, with 0 being the default value.

A **lower** "nice" value (e.g., -20) means **higher priority**, so the process gets more CPU time. On the other hand, a **higher** "nice" value (e.g., +19) means **lower priority**, so the process gets less CPU time. So, -20 is the highest priority you can set, and +19 is the lowest.



CPU time refers to the amount of time the CPU spends on a particular process. Processes with higher priority (lower nice value) are given more CPU time, meaning they get a larger percentage of the CPU's resources to perform their tasks. Conversely, processes with lower priority (higher nice value) are given less CPU time, meaning they receive a smaller percentage of the CPU's resources.

When you use the "nice" command, you are starting a new process with a defined priority.

To start a process with higher priority (meaning it will get more CPU time and therefore more CPU usage in percentage), you would use:

```
kali > nice -n -10 binslowprocess
```

Here, we set the nice value to **-10**, which gives the process higher priority, so it'll use a higher percentage of the CPU's power).

Conversely, to start a process with lower priority (meaning it will get less CPU time and therefore less CPU usage in percentage), you would use:

```
kali > nice -n 10 binslowprocess
```

In this case, the process starts with a nice value of **10**, meaning it will get a lower priority and use less CPU power compared to processes with higher priority.

If the process is already running and you want to adjust its priority, you use the "renice" command.

For example, to **lower the priority** of a running process with PID 6996 (reducing its CPU usage), you would set a **high** nice value:

```
kali > renice 20 6996
```

If you want to **increase the priority** of an existing process (giving it more CPU time and a higher CPU usage percentage), you would use a **lower** nice value. However, only the root user can set the nice value to a negative number for higher priority. For example:

```
kali > renice -10 6996
```

This command would increase the process's priority by setting its nice value to **-10**.

You can also use the "top" command to adjust the nice value. While "top" is running, press the "R" key, provide the PID, and then input the desired nice value.

```
root@kali: ~  
File Actions Edit View Help  
top - 15:26:48 up 52 min, 2 users, load average: 0.03, 0.05, 0.00  
Tasks: 164 total, 1 running, 163 sleeping, 0 stopped, 0 zombie  
%Cpu(s): 1.5 us, 0.5 sy, 0.0 ni, 97.6 id, 0.0 wa, 0.0 hi, 0.0 si,  
MiB Mem : 3929.1 total, 2542.4 free, 817.0 used, 569.7 buff/c  
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 2857.8 avail  
PID to renice [default pid = 823] 1160  
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+  
823 root 20 0 480792 192812 62108 S 3.7 4.8 0:44.82  
1160 root 20 0 457264 44448 33280 S 0.7 1.1 0:02.15
```

```
Renice PID 1160 to value -20
```

Terminating Processes

Occasionally, a process may excessively consume system resources. Such processes are often referred to as "zombie processes." In such cases, you may need to halt them using the "kill" command.

The "kill" command syntax is "kill -signal PID." Commonly used kill signals include:

- **SIGHUP (1):** Stops and restarts the process with the same PID.
- **SIGINT (2):** A weak kill signal that works in most cases.
- **SIGQUIT (3):** Terminates the process and saves its state in a file named "core."
- **SIGTERM (15):** The default signal. It allows the process to catch the signal and perform cleanup(e.g. save data, close files) before shutting down. If the process doesn't handle it, it will be forcefully stopped.
- **SIGKILL (9):** The absolute kill signal that ensures the process's termination, without cleaning (i.e. saving) anything

For instance, to restart a process using the HUP signal, you would use:

```
kali > kill -1 6996
```

To terminate a zombie or malicious process, use the absolute kill signal:


```
kali > kill -9 6996
```

If you're unaware of a process's PID, you can use the "killall" command to terminate it by name:

```
kali > killall -9 zombieprocess
```

Lastly, you can also use the "top" command to terminate a process by pressing the "K" key and entering the PID.

Running Background Processes

In Linux, whether you operate within a command-line interface or a graphical user interface, you are essentially working within a shell. Typically, when a command is executed, the shell awaits its completion before presenting another command prompt.

However, there are occasions when you want a process to run in the background. For instance, to start the "vim" text editor in the background, you would use '&' for example:

```
kali > vim /usr/share/wordlists/john.lst &
```

This command runs the text editor in the background and listing the john.lst wordlist (in the /usr/share/wordlists directory), freeing the terminal for other tasks.

Bringing Processes to the Foreground

To bring a background process to the foreground, use the "fg" command with the number of the background process and a "%" before it:

```
kali > fg %1
```

If our process with vim was the first background process to be ran (i.e. the output was "[1]") then right now we should see the editor for our wordlist

Task Scheduling in Linux

Both Linux system administrators and hackers often need to schedule processes to execute at specific times. The "at" command is useful for scheduling a job to run once

at a specified future time. The time argument can be presented in various formats, such as:

- **at 7:20pm**: Scheduled to run at 7:20 PM on the current day.
- **at noon**: Scheduled to run at 12 PM on the current day.
- **at now + 20 minutes**: Scheduled to run in 20 minutes from the current time.

Keep in mind you cannot schedule recurring task (e.g. every day at 7:20 AM) using the 'at' command.

For example, to schedule "myscanningscript" to run at 7:20 AM on the current day, you would use:

```
kali > at 7:20am
```

Then at the prompt, input:

```
at > /root/myscanningscript
```

This schedules "myscanningscript" for execution at 7:20 AM.

If you want to schedule a script to run exactly one week from now, you can use the following command:

```
kali > at now + 1 week
```

After entering this command, you'll be prompted to enter the command or script you want to execute. For instance, to run a script named `weeklybackup.sh`:

```
at> /root/weeklybackup.sh
```

Press `Ctrl+D` to save the job. This schedules `weeklybackup.sh` to run exactly one week from the current time.

A problem that comes with using 'now + 1 month' for example is that you cannot set an exact time, it'll always execute in 1 month from your current time. To solve that you can just mention the exact date:

```
kali > at 10:30am Dec 25 2024
```

Here's a table with the most common scheduling times:

Time Format	Meaning
at 7:20pm	Scheduled to run at 7:20 PM on the current day
at 7:20pm June 25	Scheduled to run at 7:20 PM on June 25
at noon	Scheduled to run at noon on the current day
at noon June 25	Scheduled to run at noon on June 25
at tomorrow	Scheduled to run tomorrow
at now + 20 minutes	Scheduled to run in 20 minutes from the current time
at now + 10 hours	Scheduled to run in 10 hours from the current time
at now + 5 days	Scheduled to run in five days from the current date
at now + 3 weeks	Scheduled to run in three weeks from the current date
at 7:20pm 06/25/2025	Scheduled to run at 7:20 PM on June 25, 2025

Chapter Summary

The management of processes in a Linux environment is a fundamental skill for all Linux users and especially hackers. Proficiency in viewing, locating, terminating, prioritizing, and scheduling processes is indispensable for effective Linux system administration. As a hacker you'll especially manage processes, whether it is terminating antivirus software or bypassing firewalls.

Practice Exercises

Before proceeding to Chapter 7, I recommend testing your newly acquired skills from this chapter by completing the following exercises:

- Execute the "ps" command with the "aux" options on your system and take note of the first and last processes listed.
- Use the "top" command and identify the two processes consuming the most system resources.
- Then use the "kill" command to terminate 1 of these processes using the most resources.
- And for the other use "renice" command to lower the priority of it to +19.
- Create a script named "myscanning" (the content doesn't matter) using a text editor and schedule it to run next Wednesday at 1 AM.

7. HANDLING USER ENVIRONMENT VARIABLES

To fully master Linux, it's essential to learn what environment variables are and how to manage them. If you do so you'll enhance the performance, convenience, and even stealth in your Linux machine.

In technical terms, there are two main types of variables: shell variables and environment variables. A variable by itself is a specific value or piece of data stored in a computer's memory. For example in programming a variables can be an integer (i.e. number) that is "x = 10" or a string "SecurityAiLab".

Environment variables are system-wide variables integrated into your system and interface, controlling how your system appears, behaves, and "feels" to the user. Importantly, these environment variables are inherited by all shells or processes. For example, you know the "~" in Linux? It is a variable that is the home directory of your current user "~ = /home/username "

Shell variables on the other hand, typically represented in lowercase, are only valid within the shell in which they are defined.

In this chapter, we'll focus on the fundamentals of environment and shell variables, without digging too deeply into the nuances that differentiate them.

Environment Variables Basics

Variables are essentially strings organized in key-value pairs. Typically, each pair takes the form of **KEY=value**. In situations where multiple values are involved, they are structured as **KEY=value1:value2**. In line with the Linux convention, if a value contains spaces, it should be enclosed in quotation marks.

Each user, including the root user, is has a default set of environment variables that control how the system operates, appears, and feels (e.g. that "~ = /home/username"). You have the power to modify the values of these variables to optimize your system's performance, or even maintain a degree of anonymity if necessary.

Altering Environment Variables

You can gain insight into all your default environment variables by executing the `env` command in your terminal from any directory. Here's how you can do it:

```
kali > env
```

This will display something like the following:

```
XDG_VTNR=7
SSHAGENT_PID=922
XDG_SESSION_ID=2
XDG_GREETER_DATA_DIR=/var/lib/lightdm/data/root
GLADE_PIXMAP_PATH=:echo
TERM=xterm
SHELL=/bin/bash
USER=root
PATH=usrlocal/sbin:usr/local/bin:usr/sbin:/sbin/bin
HOME=/root
```

The output will display a list of default environment variables. These variables, such as `XDG_VTNR`, `SSHAGENT_PID`, `TERM`, and many more, collectively shape the behavior of your system. Environment variables are consistently represented in uppercase, distinguishing them from shell variables. It's worth noting that these are only the default environment variables that come with your system. You can create your own custom variables, and we'll cover how to include those in the output shortly.

Viewing the Entire Environment Variable List

To get a detailed list of all environment variables, including shell variables, local variables, and even shell functions, you can use the `set` command. Executing this command will output a pretty lengthy list of environment variables on your machine. The output may be quite long, likely spanning multiple screens, so you can pipe the `set` command's output to the `more` command, like so:

```
kali > set | more
```

The output will begin with something like:

```
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdlist:complete_fullquote:expand_aliases:extglob
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
```

This approach allows you to check each variable one line at a time. You can press the `ENTER` key to advance to the next line. As a reminder from Chapter 2, you can exit this view by typing `q` and return to the command prompt.

Searching Specific Environment Variables

While using `set` with `more` is useful for making the extensive list of variables more manageable, it can still be somewhat overwhelming if you're searching for a particular variable. In such cases, using the `grep` command for filtering can be a huge time-saver. Let's take the `HISTSIZE` variable as an example. This variable controls the maximum number of commands stored in your command history file, allowing you to recall previously entered commands using the up and down arrow keys. Note that `HISTSIZE` stores the number of commands that can be stored, not the actual commands themselves.

To locate and display the `HISTSIZE` variable, use the `set` command in conjunction with `grep` as follows:

```
kali > set | grep HISTSIZE
```

The output will show:

```
HISTSIZE=1000
```

This command identifies the `HISTSIZE` variable. The default value for `HISTSIZE` is typically set to 1000 on most systems, indicating that your terminal will remember the last 1,000 commands by default.

Changing Environment Variables

Let's now see how to modify the value of a variable. As mentioned earlier, the `HISTSIZE` variable determines the number of commands to store in the history file. There are instances when you might not want your system to retain past commands, perhaps to avoid leaving traces of your activities. In such cases, you can set the `HISTSIZE` variable to 0, preventing the system from storing any of your previous commands.

To change the value, you would enter:

```
kali > HISTSIZE=0
```

Now, when you attempt to use the up-and down-arrow keys to recall your commands, you'll only get the last command you've used because the system no longer stores the other ones. This approach is stealthy but can be inconvenient.

Permanent Modification of Variable Values

When you modify an environment variable in Kali Linux, the change is confined to that specific environment, which, in this case, is the current terminal session (i.e., the window you've opened). This means that when you close the terminal, any changes you made will be lost, and the values will revert to their defaults.

To make these changes permanent in Kali Linux, you should add them to your shell's startup file. This ensures that the new values are set every time you start a new terminal session. For Bash users on Kali, this file is typically `~/ .bashrc` or `~/ .zshrc`

For instance, after changing `HISTSIZE`, you can make it permanent by adding the following line to your `~/ .bashrc` file:

```
export HISTSIZE=0
```

Now, the `HISTSIZE` variable will retain its value of `0` even when you exit this terminal and open a different one in Kali Linux.

To reset the `HISTSIZE` variable back to `1,000`, you can modify your `~/ .bashrc` file again and change the line to:

```
export HISTSIZE=1000
```

You'll also have to delete the current `HISTSIZE` value by just searching for `/HISTSIZE` in `vim` for example.

This code will set the `HISTSIZE` variable to `1,000` and ensure it's exported to all your environments every time you start a new session in Kali Linux.

Alternatively, if you want to make a temporary change that lasts only for the current session, you can use the following commands:

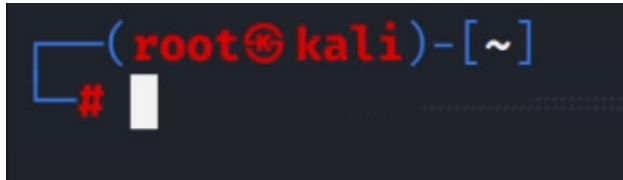
```
kali > HISTSIZE=0  
kali > export HISTSIZE
```

This sequence sets the `HISTSIZE` variable to `0` and exports it for the current session. Remember, this change will be lost once you close the terminal.

Customizing Your Shell Prompt

The shell prompt, which is another environment variable, provides essential information, including your current user and working directory. The default shell prompt in Kali follows this format: "username@hostname"

If you're operating as the root user, the default prompt becomes: "root@kali"



You can personalize the name in the default shell prompt by adjusting the value of the `PS1` variable. The `PS1` variable has a set of placeholders for the information you want to display in the prompt, including:

- **\u**: The current user's name
- **\h**: The hostname
- **\W**: The base name of the current working directory

This customization can be especially useful if you have multiple shells on different systems or you just want to look cool ;)

But this simplicity only applies to Bash, if you write in Zshell (which is the type of shell Kali uses) you'll seem something like this:

```
PS1='$'%F{%(#.blue.green)}└─${debian_chroot:+($debian_chroot)-}${VIRTUAL_ENV:+($  
basename $VIRTUAL_ENV)-}(%B%F{%(#.red.blue)}%nⓈm%b%F{%(#.blue.green)})-[%B%F{r  
eset}% (6~.%-1~/.../%4~.%5~)%b%F{%(#.blue.green)}]\n└─%B{#. %F{red}#. %F{blue}$)%b%F{  
reset} '
```

I know.. It looks scary, but let's break it down.

Wherever you see a **%F** means that it's just changing the color of the shell prompt elements. For example, `%F{blue}` makes the text blue. You can stop a coloring with `%f`.

In your shell prompt, you see something like this:

```
%F{%(#.blue.green)}
```

This means: **if the user is root**(specified by #), display blue; **if it's a regular user**(specified by "."), display green. Zsh's conditional logic, %(#.), checks if you're root or not. It's a fancy way of automatically coloring your prompt depending on your user permissions.

You can just skim over these parts for now.

This means that now we've got this value without the coloring:

```
PS1=$ _ ${debian_chroot:+($debian_chroot)-}${VIRTUAL_ENV:+($  
basename $VIRTUAL_ENV))-}(%n⊕%m )-[  
%(6~.%-1~/.../%4~.%5~) ]\n ( # . $ )
```

```
(root⊕kali)-[~]  
#
```

There's \${debian_chroot:+(\$debian_chroot)}. If you're working in a **chroot** environment (an isolated filesystem used for security or development purposes), it will show the name of that environment in the prompt. Otherwise, it stays hidden.

```
PS1=$ _ ${VIRTUAL_ENV:+($  
basename $VIRTUAL_ENV))-}(%n⊕%m )-[  
%(6~.%-1~/.../%4~.%5~) ]\n ( # . $ )
```

For the username and hostname, instead of Bash's \u and \h, Zsh uses %n (username) and %m (hostname). This snippet:

```
(%B%F{%(#.red.blue)}%n⊕%m%b)
```

Displays the username in **red** if you're root, and in **blue** if you're a normal user. The %B and %b make the username bold.

The `Ⓢ` is just a decorative Unicode character that separates the username and hostname.

```
PS1=$ basename $VIRTUAL_ENV))-(  
%nⓈ%m  
%(6~.%-1~/.../%4~.%5~) ]\nL (#. $)
```

The part handling the working directory is:

```
[(6~.%-1~/.../%4~.%5~)]
```

This replaces the `\W` in Bash. Here, Zsh is set up to display the **current directory** but truncates it if it's too long (for instance, by replacing some of it with `...`).

Prompt Symbol (Root vs. User)

The prompt symbol itself is:



```
LB%B%(#.F{red}#.F{blue}$)%b
```

This checks if you're **root** or not. If you are root, it shows `#` in red; if not, it shows `$` in blue. This is more visual than the basic `$` you'd see in Bash.

Now you can for example, modify your terminal prompt to:

```
kali > PS1="World's Best Hacker: #"
```

Now, each time you use this terminal, you'll be reminded that you are the "World's Best Hacker." Keep in mind that any subsequent terminal you open will still feature the default prompt. This is because the `PS1` variable only retains values for your current terminal session. If you genuinely appreciate this new prompt and want it to appear in every terminal, export it in the `~/.zshrc` file by changing:

```
twoline)
```

```
PROMPT=$'%F{%(#.blue.green)} ┌───${debian_chroot:+($debian_chroot)─}${VIRTUAL_ENV:
+($ (basename
$VIRTUAL_ENV))─}(%B%F{%(#.red.blue)}%n'$prompt_symbol$'%m%b%F{%(#.blue.green)}) -
[%B%F{reset}%(6~.%-
1~/.../%4~.%5~)%b%F{%(#.blue.green)}}\n └─%B%(#.F{red}#.F{blue}$)%b%F{reset} '

;;
```

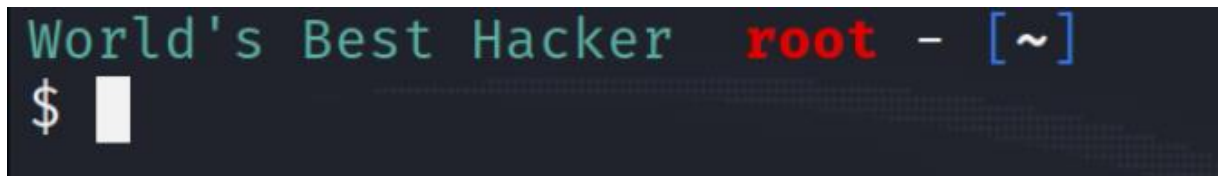
Which you can find by just writing /twoline) in the vim editor. So change the PROMPT= to the PS1 value you want, for example:

```
twoline)

PROMPT="%F{green}World's Best Hacker%f %B%F{red} %n%b%f
- %F{blue}[%f%B%~%b%F{blue}]%f

;;
```

Now it is by default your shell prompt.



Modifying the PATH

One of the most crucial variables in your environment is the PATH variable, which determines where your shell searches for commands you enter, such as cd, ls, and echo. Most commands are located in the sbin or bin subdirectories, like usr/local/sbin or usr/local/bin. If the bash shell fails to find a command in any of the directories listed in your PATH variable, it will return the "command not found" error, even if the command exists in a directory not included in your PATH.

To check which directories are stored in your PATH variable, you can use the echo command on its contents, like this:

```
kali > echo $PATH
```

The output will display something like this:

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
```

These directories represent the locations where your terminal will search for commands. For instance, when you enter `ls`, the system knows to search in each of these directories for the `ls` command and execute it. Each directory is separated by a colon (:).

Correctly Expanding the PATH Variable

Suppose you've downloaded and installed a new tool, called "newhackingtool," in the `rootnewhackingtool` directory. You can only use commands from that tool when you're in that directory because it's not included in the PATH variable.

To make the new tool accessible from any directory, you need to add the directory containing it to your PATH variable by typing:

```
kali > PATH=$PATH:rootnewhackingtool
```

This command appends the original PATH variable with the `rootnewhackingtool` directory, effectively including everything that was already in PATH plus the new tool directory.

If you examine the contents of the PATH variable again, you should see that this directory has been appended to the end of PATH.

Replacing The Path Variable

A common mistake made by newcomers to Linux is directly assigning a new directory, like `rootnewhackingtool`, to the PATH variable like this:

```
kali > PATH=rootnewhackingtool
```

```
kali > echo $PATH
```

The result will show:

```
rootnewhackingtool
```

This means your PATH variable is overwritten, and you lose access to system binaries. So make sure you **always** append to the PATH variable instead of overwriting it.

Creating a User-Defined Variable

In Linux, you have the flexibility to create your own custom variables by simply assigning a value to a variable of your choosing. For example:

```
kali > MYNEWVARIABLE= /usr/share/wordlists/
```

To view the value stored in this variable, use the cd command:

```
kali > cd $MYNEWVARIABLE
```

This will switch your directory to the /usr/share/wordlists/ without you having to type it out.

If you wish to remove a user-defined variable or any variable, you can use the unset command:

```
kali > unset MYNEWVARIABLE
```

Now, using cd \$MYNEWVARIABLE will return a blank line.

Saving Environment Variables

Variables are essentially strings, so it's wise to be careful by saving the contents of a variable to a text file before making modifications. For instance, if you're about to modify the HISTSIZE variable which controls the maximum number of commands stored in your command history file, you can first run the following command to save the existing values to a text file in your current user's home directory:

```
kali > echo $HISTSIZE > ~/valueofHISTSIZE.txt
```

Now the contents of HISTSIZE are saved, and you can restore them if needed.

If you'd like to save all environment variables and settings, you can do so by running the following:

```
kali > set > ~/valueofALLon01012024.txt
```

This will save all your environment variables to a file.

Chapter Summary

Understanding and managing environment variables can significantly improve your Linux hacking experience. These variables control the appearance, behavior, and user experience of your system. With proper management, you can customize, export, and create your own environment variables, and even use them to maintain anonymity when needed.

Practice Exercises

Use the more command to view all your environment variables:

- Run `env | more` to find the USER environment variable. Scroll through the output and specifically locate the USER variable, which should show your current username.

Employ the echo command to display the contents of the HOSTNAME variable:

- Use `echo $HOSTNAME` to show the name of your machine (hostname).

Find a method to replace the slash (/) with a backslash (\) in the faux Microsoft cmd PS1 example:

- First, set your PS1 to `/C/Users/Admin:`
 - `PS1="/C/Users/Admin$ "`
- Then, replace the slashes with backslashes by running:
 - `echo $PS1 | sed 's/\\/\\\\/g'`

Create a variable named MYNEWVARIABLE and assign your name to it:

- Specifically, set MYNEWVARIABLE to "JohnDoe" (or your actual name):
 - MYNEWVARIABLE="JohnDoe"

Export MYNEWVARIABLE to make it available in all environments:

- Make the variable globally available with:
 - export MYNEWVARIABLE

Use the echo command to view the contents of the PATH variable:

- Use:
 - echo \$PATH
- Look specifically for /usr/local/bin in the output.

Add your home directory to the PATH variable so that binaries in your home directory can be used from any directory:

- Run:
 - export PATH=\$PATH:\$HOME
- Confirm the addition by running echo \$PATH again and verifying that your home directory (/home/username) is included.

Change your PS1 variable to "World's Greatest Hacker: ":

- Specifically modify your prompt to:
 - PS1="World's Greatest Hacker: "

8. INTRODUCTION TO BASH SCRIPTING

Any proficient hacker or Linux administrator should be able to script. Hackers often need to automate various tasks using commands from different tools, and this is best achieved through self-written scripts. In this chapter, we will begin by creating some simple bash shell scripts to introduce you to scripting. As we progress, we'll enhance these scripts, eventually building one that can identify potential attack targets across a range of IP addresses.

To excel as a hacker, you should also be proficient in scripting languages like Ruby (used in Metasploit exploits), Python (utilized in many hacking tools), or Perl (known for its text-manipulation capabilities).

Basics of Bash Programming

A shell serves as an interface between the user and the operating system, allowing file manipulation and command execution directly from the computer. Unlike graphical interfaces, shells provide flexibility and customization to meet your specific needs. Linux offers various shells, including Korn, Zsh, C shell, and the Bourne-again shell (bash), which is widely available and will be our focus in this chapter.

Bash can execute system commands, utilities, and applications, just like your regular command line, but it also includes built-in commands. You've likely encountered commands like `cd`, `pwd`, `set`, and `umask` in previous chapters. In this section, we'll introduce two more: `echo` and `read`, which will help you create simple yet powerful tools.

To create shell scripts, you'll need a text editor. You can choose from various Linux text editors like **vi**, **vim**, **emacs**, **gedit**, **kate**, etc. For these tutorials, we'll use **vim**, but feel free to use your preferred editor.

Crafting Your First Script: "Hello, SecurityAiLab!"

Let's start with a simple script that displays the message "Hello, SecurityAiLab!" on the screen. Open your text editor and follow these steps:

1. Begin by specifying the script's interpreter using a **shebang** (`#!`). In this case, we want to use the bash interpreter: `#!/bin/bash`.

2. Use the echo command to display "Hello, SecurityAiLab!" on the screen. Wrap the message in double quotation marks.

Here's what your script should look like:

```
#!/bin/bash
# This is my first bash script. Wish me luck.
echo "Hello, SecurityAiLab!"
```

In this script, comments (lines starting with #) are provided for clarity but are ignored by the interpreter, you can write explanations, steps, etc. in them without worrying for possible errors.

Save this file as HelloSecurityAiLab without any file extension.

Setting Permissions for Script Execution

By default, newly created bash scripts aren't executable. To make your script executable, you need to change its permissions using the chmod command. Follow these steps:

1. Check the file's permissions with `ls -l` to ensure it's not executable:

```
kali > ls -l HelloSecurityAiLab
```

The output should look something like this:

```
-rw-r--r-- 1 root root 23 Jan 20 10:35 HelloSecurityAiLab
```

This means the script is readable and writable, but not executable.

2. Give execute permissions to the owner, group, and others using `chmod 755 HelloSecurityAiLab`:

```
kali > chmod 755 HelloSecurityAiLab
```

Now, the file has execute permissions:

```
-rwxr-xr-x 1 root root 23 Jan 20 10:35 HelloSecurityAiLab
```

Executing HelloSecurityAiLab

To execute your script, use the `./` prefix followed by the script's name:

```
kali > ./HelloSecurityAiLab
```

The `./` indicates that you want to run the script from the current directory. This practice avoids potential conflicts with similarly named files in other directories.

When executed, the script will display:

```
Hello, SecurityAiLab!
```

Script Enhancement with Variables and User Interaction

To create more advanced scripts, you'll need to work with variables. Variables are storage areas that hold data, such as strings or numbers, and are essential for adding functionality to your scripts.

In the next script, we'll prompt the user for their name, store it in a variable, ask for the chapter they're on in this book, store that in another variable, and display a personalized message.

Here's the script:

```
#!/bin/bash
# This is your second bash script. In this one, you prompt
# the user for input, place the input in a variable, and
# display the variable contents in a string.

echo "What is your name?"
read name
echo "What chapter are you on in Command Line Hacker?"
read chapter
echo "Welcome $name to Chapter $chapter of Command Line Hacker!"
```

In this script, `read` captures user input and stores it in variables. The final line constructs a personalized message using those variables.

Save this file as `WelcomeScript.sh`.

To execute it, make it executable using `chmod 755 WelcomeScript.sh`, and then run:

```
kali > ./WelcomeScript.sh
```

```
What is your name?
```

```
Dobri
```

```
What chapter are you on in Command Line Hacker?
```

```
8
```

```
Welcome Dobri to Chapter 8 of Command Line Hacker!
```

This script is a basic example of using variables and user input, which are essential concepts for more complex scripting tasks.

Also keep in mind that you can use any other variables in your scripts. For example you can use `$USER`, containing your username, instead of asking for it.

Building a Basic Port Scanner Script

Now that you've acquired some basic scripting skills, let's take things up a notch with a practical hacking tool. We'll develop a simple port scanner that identifies open ports on local systems.

Before we proceed, familiarize yourself with **nmap**, a powerful tool that comes pre-installed on Kali Linux. **Nmap** is used to probe systems to determine their network connectivity and discover open ports.

The basic syntax for running an nmap scan is as follows:

```
nmap <scan type> <target IP> <optionally, target port>
```

For example, a simple **TCP connect scan**(which is just a connection with the web, or to a specific host) on an IP address might look like this:

```
kali > nmap -sT 104.21.82.0
```

```
Starting Nmap 7.80 ( https://nmap.org ) at 2024-09-25 10:00 UTC
```

```
Nmap scan report for 104.21.82.0
```

```
Host is up (0.0010s latency).
```

```
Not shown: 998 filtered ports
```

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
...		

If you want to scan specifically for **port 3306** (the default MySQL port):

```
kali > nmap -sT 104.21.82.0 -p 3306
```

```
Starting Nmap 7.80 ( https://nmap.org ) at 2024-09-25 10:05 UTC
Nmap scan report for 104.21.82.0
Host is up (0.0010s latency).
PORT      STATE SERVICE
3306/tcp  open  mysql
```

Creating a Port Scanner Script

Let's write a simple script that scans a local network for open **MySQL ports** (port 3306).
Input the following script in your text editor:

```
#!/bin/bash
# This script is designed to find hosts with MySQL installed

nmap -sT 104.21.82.0/24 -p 3306 > /dev/null -oG MySQLscan
cat MySQLscan | grep open > MySQLscan2
cat MySQLscan2
```

1. **nmap** performs a scan on the local network (104.21.82.0/24), looking for open MySQL ports.
2. The results are stored in the file MySQLscan.
3. We then filter the results using grep and save the output in MySQLscan2.

Now if you get a state of Filtered rather than open this just means that there's a firewall that stops you from finding the state of the port (which is the case you'll see for most IPs you test) So you can try testing with the site <http://scanme.nmap.org/>

It is specifically designed for playing with nmap.

And here are the details of this script:

```
#!/bin/bash
```

This is called a **shebang**. It tells the system that the script should be run using the Bash shell.

```
-sT
```

The `-sT` flag tells `nmap` to perform a TCP connect scan, which is a basic and reliable scan that attempts to fully establish TCP connections with targets. It's useful in identifying services listening on a port.

```
104.21.82.0/24
```

This is the **target IP** you're scanning. The `/24` at the end means you're scanning all IP addresses in the range from `104.21.82.0` to `104.21.82.255`. This represents a typical local network for all devices.

```
-p 3306
```

This option specifies the **port** you are scanning. In this case, port 3306 is the default port for MySQL, so this command is specifically looking for systems running MySQL.

```
> /dev/null
```

This **redirects** the standard output (the regular result of the command) to `/dev/null`. This means that any output from `nmap` will not be printed to the screen, reducing clutter. Without it, even if we redirected the output we'd still see `nmap`'s output.

```
-oG MySQLscan
```

The `-oG` flag tells `nmap` to output its results in **grepable format** to the file `MySQLscan`. This format is easy to parse with commands like `grep`. Without it when searching for `grep` it wouldn't be very clear what the output is talking about

```
non_grepable:
```

```
- Starting Nmap 7.91 ( https://nmap.org ) at 2024-09-28 12:10 EDT
- Host is up (0.00042s latency). PORT      STATE SERVICE 3306/tcp open mysql
```

```
grepable:
```

```
- Host: 192.168.1.101 () Ports: 3306/open/tcp//mysql///
- Host: 192.168.1.102 () Ports: 3306/closed/tcp//mysql///
```

```
cat MySQLscan | grep open
```

Then we display only the line that contains the open port (which we've filtered to only include the MySQL port of 3306) and we've saved the analysis in the file MySQLscan.

```
> MySQLscan2
```

We're saving that line to a new file.

```
cat MySQLscan2
```

And are displaying that line

Save this file as MySQLscanner.sh, make it executable (chmod 755 MySQLscanner.sh), and run:

```
kali > ./MySQLscanner.sh
```

```
Host: 104.21.82.69 ()    Ports: 3306/open/tcp//mysql///
```

This output shows the IP addresses on your LAN with open MySQL ports. Results will vary depending on whether any devices on your network are running MySQL.

Enhancing the Port Scanner

Now, let's enhance the script by adding user input. This will allow the user to specify the IP range and the port to scan. Here's the modified script:

```
#!/bin/bash
```

```
# Prompt the user for the starting IP address
echo "Enter the starting IP address: "
read FirstIP
```

```
# Prompt for the last octet of the last IP address
echo "Enter the last octet of the last IP address: "
read LastOctetIP
```

```
# Prompt for the port to scan
echo "Enter the port number to scan for: "
read port
```

```
# Perform the nmap scan
nmap -sT $FirstIP-$LastOctetIP -p $port > /dev/null -oG MySQLscan
cat MySQLscan | grep "open" > MySQLscan2
cat MySQLscan2
```

Here the only difference is we're asking the user to input the IP and we're talking about some strange octet.

An octet refers to one of the four parts of an IP address, where each part is an 8-bit value, hence the term "octet." Each octet can range from 0 to 255 because an 8-bit number can represent 256 different values (0 to 255). For example, in the IP address 104.21.82.0, each of the numbers separated by dots is an octet:

- 104 is the first octet.
- 21 is the second octet.
- 82 is the third octet.
- 0 is the fourth octet.

So when asking for the "first IP" we're asking for the whole IP (and expect the user to give us an IP that ends with a fourth octet of 0 or 1 for example). So that then when we ask for the last octet we can tell Nmap to scan all devices that are connected to the same LAN (which is like the Wi-Fi).

Because IPs where only the 4th octet differs are IPs of different devices on the same LAN. And we're analyzing ALL of the devices on the LAN.

You can save this as MySQLscannerAdvanced.sh, make it executable (chmod 755 MySQLscannerAdvanced.sh), and run it.

```
kali > ./MySQLscannerAdvanced.sh
```

```
Enter the starting IP address:
```

```
104.21.82.0
```

```
Enter the last octet of the last IP address:
```

```
255
```

```
Enter the port number to scan for:
```

```
3306
```


Host: 104.21.82.254 () Ports: 3306/open/tcp//mysql///

The script prompts the user for the IP range and port, making it more flexible and useful.

Common Built-in Bash Commands

The table below provides a list of essential built-in bash commands that can be handy in your scripts.

Command	Function
:	Returns 0 (true)
.	Executes a shell script
bg	Puts a job in the background
break	Exits the current loop
cd	Changes the directory
continue	Resumes the current loop
echo	Displays the command arguments
exec	Executes the following command without creating a new process
exit	Quits the shell
export	Makes a variable or function available to other programs
fg	Brings a job to the foreground
jobs	Lists background jobs
pwd	Displays the current directory
read	Reads a line from standard input
set	Lists all variables
test	Evaluates arguments

Chapter Summary

Scripting is an essential skill for both hackers and system administrators. It enables automation of repetitive tasks, saving time and increasing efficiency. Once a script is created, it can be reused indefinitely.

Practice Exercises

Develop your own greeting script:

- Create a script called `GreetUser.sh` that echoes a personalized greeting using the `USER` environment variable.
- The script should look like this:

```
#!/bin/bash
echo "Hello, $USER! Welcome to your system."
```
- Save the file as `GreetUser.sh`, give it execute permissions (`chmod +x GreetUser.sh`), and run it in the `~/.zshrc` file (`/GreetUser.sh` keep in mind there is no dot) to make it start automatically when opening a new terminal.

Create an MSSQL scanner script:

- Modify the existing `MySQLscanner.sh` script to scan for Microsoft SQL Server on port **1433** and save it as `MSSQLscanner.sh`.
- The core of your script might look like this:

```
nmap -sT 104.21.82.0/24 -p 1433 > /dev/null -oG MySQLscan
cat MySQLscan | grep "open" > MySQLscan2
cat MySQLscan2
```

Modify `MSSQLscanner.sh` to prompt for IP addresses and ports, displaying only open ports:

- Extend the script to take input from the user and show only open ports. Modify it like so:

```
#!/bin/bash
read -p "Enter the IP address to scan: " ip
read -p "Enter the port to scan (default 1433): " port

# Set default port to 1433 if none is provided
```

```
port=${port:-1433}
```

```
echo "Scanning $ip on port $port for open ports..."  
nmap -p $port --open $ip
```

- This script will prompt the user for an IP and port, then scan for open ports only.

9. TECHNIQUES IN DATA COMPRESSION AND ARCHIVING

In cybersecurity, the need often arises to download and install new software, as well as exchange multiple scripts and large files. These tasks become more manageable when these files are compressed and bundled into a single archive.

If you're transitioning from the Windows environment, you might recognize this concept from the .zip format, which combines and compresses files to make them more suitable for internet transfers or storage on removable media.

In this chapter, we'll learn various methods of achieving this in Linux, exploring some of the most commonly used tools. Additionally, we'll use the dd command to copy entire drives, including recovering deleted files from those drives.

Understanding Data Compression

Compression is an intriguing subject that could easily fill an entire book, but for our purposes, a basic grasp of the concept will suffice. At its core, compression reduces the size of data, making it more storage-efficient and easier to transmit. For hackers, it's crucial to categorize compression into two main types: **lossy** and **lossless**.

Lossy compression effectively reduces file size but sacrifices some data integrity. In simple terms, the compressed file isn't an exact replica of the original. This approach is particularly effective for graphics, video, and audio files, where minor discrepancies are often imperceptible. Formats like .mp3, .mp4, .png, and .jpg employ lossy compression where the negative effects might be lower resolution and/or quality of audio or video. While changes in a pixel or a single note may go unnoticed, audiophiles claim to discern differences between an .mp3 and an uncompressed .flac file.

However, when you need to transmit files or software with utmost data integrity, lossy compression falls short. In scenarios like sending scripts or documents, preserving the original file's integrity upon decompression is paramount. This chapter focuses on **lossless compression**, which is accessible through various utilities and algorithms. Though not as efficient in terms of compression ratio as lossy compression, it excels in maintaining data integrity, a critical consideration for hackers.

Grouping Files with tar

Typically, the initial step in compressing files involves consolidating them into an archive. In Linux, the go-to command for this task is often "tar," which stands for "tape archive" – a nod to the early days of computing when tape was used for data storage. The "tar" command takes multiple files and combines them into a single file, referred to as an archive, tar file, or tarball.

A tarball is essentially a single file that contains multiple files or directories bundled together. The term "tarball" comes from the .tar format, which stands for Tape Archive. It doesn't compress the files by itself; it simply combines them into one archive for easier storage or sharing. This can be particularly useful when you have several files or directories that you want to move or share as a single entity.

For example, imagine you have three script files (securityailab1, securityailab2, and securityailab3) in a directory. A long listing of the directory shows these files, including their sizes:

```
kali > ls -l
-rwxr-xr-x 1 root root 22311 Nov 27 2024 13:00 securityailab1.sh
-rwxr-xr-x 1 root root 8791 Nov 27 2024 13:00 securityailab2.sh
-rwxr-xr-x 1 root root 3992 Nov 27 2024 13:00 securityailab3.sh
```

Suppose you want to send all three files to another hacker collaborating on a project. You can bundle them into a single archive using the command shown below.

```
kali > tar -cvf SecurityAiLab.tar securityailab1 securityailab2 securityailab3  
  
securityailab1  
securityailab2  
securityailab3
```

Breaking down the command, we have "tar" as the archiving command, along with three options: "c" for create, "v" (optional, stands for verbose) to list the files being archived, and "f" to specify the filename for the new archive – "SecurityAiLab.tar" in this case.

Executing this command combines the three files into a single "SecurityAiLab.tar" file. Upon checking the directory again, you'll notice the new .tar file:

```
kali > ls -l  
  
-rw-r--r-- 1 root root 40960 Nov 27 2024 13:32 SecurityAiLab.tar
```

Notice the size of the tarball – 40,960 bytes. The archiving process adds some overhead to the file size. While the combined size of the three original files was 35,094 bytes, the tarball's size has grown due to this overhead, exceeding 5,000 bytes. This overhead is more pronounced with smaller files but becomes less significant with larger ones.

To view the files within the tarball without extracting them, you can use the "tar" command with the "-t" (list contents) switch:

```
kali > tar -tvf SecurityAiLab.tar
```

This command displays the original files and their sizes. To extract the files from the tarball, you can use the "tar" command with the "-x" (extract) switch:

```
kali > tar -xvf SecurityAiLab.tar
```

If you prefer to extract the files silently, without displaying output, you can omit the "-v" (verbose) switch:

```
kali > tar -xf SecurityAiLab.tar
```

Files extracted this way will replace existing files with the same name.

Now once you have a tarball, you might want to extract its contents — meaning to "unpack" or "take out" the individual files that were bundled together. This process is called extracting files from the archive.

Before extracting files though, you might want to see what's inside the tarball. You can do this without unpacking the files. To view the contents of a tarball (without extracting), you can use the tar command with the **-t** option, which lists the contents:

```
kali > tar -tvf SecurityAiLab.tar
```

Here's what each part of the command does:

- **t**: List the contents of the tarball (it shows the files without unpacking them).
- **v**: Verbose, which means it will display detailed information like file sizes and permissions.

- `f`: Specifies the name of the tarball to operate on, in this case, `SecurityAiLab.tar`, omitting the `-f` option will result in an error. Alternatively you can use an input redirection operator (`<`) and then a text file that should contain the name(s) of `.tar` files, instead of manually inputting it

This command shows the names, sizes, and permissions of all the files inside the tarball, but it does not extract them — it just lists them.

Once you've seen the contents, you can extract (or unpack) the files from the tarball using the `-x` option, which stands for extract:

```
kali > tar -xvf SecurityAiLab.tar
```

Here's what happens:

- `x`: Extract the files from the tarball.
- `v`: Verbose, meaning it will show the names of the files being extracted as they are unpacked.
- `f`: Again, it specifies the name of the tarball, in this case, `SecurityAiLab.tar`.

This command will unpack all the files from the tarball into your current directory. If there are already files with the same names in that directory, they will be **replaced** by the ones being extracted.

If you don't want to see a list of the files being extracted, you can remove the `-v` option (verbose). This means the files will be extracted without displaying any information:

```
kali > tar -xf SecurityAiLab.tar
```

Here's what happens:

- `x`: Extract the files.
- `f`: Specify the tarball name.

This is the same as the previous extraction command, but without showing the file names and details while the extraction happens. It will still unpack the files into your directory, but it won't output any information during the process.

When you extract files from a tarball, any files with the same name already in your current directory will be **replaced** by the versions from the tarball. So, if you have a file named `securityailab1.sh` in your directory, and the tarball also contains a file with the same name, the extracted file will **overwrite** the existing one. Be cautious when extracting files into a directory with existing files to avoid accidentally replacing something important.

Techniques for File Compression

Now that we have an archived file, you might want to further compress it for easier transportation. Linux provides several commands for creating compressed files, including:

"gzip" (resulting in `.tar.gz` or `.tgz` files)

"bzip2" (`.tar.bz2` files)

"compress" (`.tar.z` files)

Each of these commands employs different compression algorithms and offers varying compression ratios.

In general, "compress" is the fastest but results in larger files. "bzip2" is the slowest but produces the smallest files. "gzip" falls in between these two extremes. This diversity is valuable, as you'll encounter various compression types in your hacking endeavors.

Using gzip for Compression

Let's begin with gzip (GNU zip), one of the most widely used compression tools in Linux. You can compress your `SecurityAiLab.tar` file by executing the following command (ensure you're in the directory containing the archived file):

```
kali > gzip SecurityAiLab.*
```


Note the use of the wildcard `*` for the file extension, indicating that the command should be applied to any file starting with "SecurityAiLab" and having any file extension. Similar notation will be used in the subsequent examples. Upon examining the directory with a long listing, you'll observe that "SecurityAiLab.tar" has transformed into "SecurityAiLab.tar.gz," with a significantly reduced file size, now just 3,299 bytes:

```
kali > ls -l
```

```
-rw-r--r-- 1 root root 3299 Nov 27 2024 13:32 SecurityAiLab.tar.gz
```

To decompress this file, you can use the "gunzip" command, a shortened form of "GNU unzip":

```
kali > gunzip SecurityAiLab.*
```

Upon decompression, the file no longer retains the ".tar.gz" extension but reverts to ".tar" Also, its size returns to the original 40,960 bytes. You can confirm this by performing a long listing. It's worth noting that gzip can also be used to extract .zip files.

File Compression with bzip2

Another widely used compression utility in Linux is bzip2. It operates similarly to gzip but offers superior compression ratios, resulting in even smaller files. To compress your "SecurityAiLab.tar" file, use the following command:

```
kali > bzip2 SecurityAiLab.*
```

Upon inspecting the long listing, you'll notice that bzip2 has compressed the file down to just 2,081 bytes. Additionally, the file extension has changed to ".tar.bz2" To decompress the compressed file, utilize "bunzip2" again coming from "Bzip2 unzip":

```
kali > bunzip2 SecurityAiLab.*
```

By executing this command, the file will revert to its original size, and its extension will return to ".tar."

The compress Method

Finally, there's the "compress" command, which allows you to compress the file. Although less commonly used, it's straightforward to remember. To employ it, simply enter the "compress" command followed by the filename:

```
kali > compress SecurityAiLab.*
```

Upon compression, you'll observe that the file size has been reduced to 5,476 bytes, which is more than twice the size of the bzip2-compressed file. Additionally, the file extension is now ".tar.Z," with an uppercase "Z."

```
kali >ls -l
```

```
-rw-r--r-- 1 root root 5476 Nov 27 2024 13:32 SecurityAiLab.tar.Z
```

To decompress the same file, use "uncompress":

```
kali > uncompress SecurityAiLab.*
```

You can also use the "gunzip" command with files that have been compressed using "compress."

Creating Exact Copies of Storage Devices

The `dd` command is a powerful Linux tool that allows you to create **bit-by-bit copies** of files, filesystems, or entire hard drives. One of its most intriguing features is that it can even copy **deleted files** from storage devices. This capability is crucial for forensic investigators and hackers because it means that data, even after deletion, can still be recovered if it hasn't been overwritten.

How does `dd` copy deleted files?

When you delete a file on most filesystems, the file isn't immediately erased. Instead, the space it occupied is marked as "available" for new data, but the actual contents of the file remain on the disk until they are overwritten by new data. The `dd` command operates at a low level, copying **every bit of data on a device**, not just the files that the filesystem currently tracks.

In the `dd` command, you need to specify the **input file** (`if`) and the **output file** (`of`). These aren't always literal files — they can be entire devices or disk partitions.

- **if= (input file):** This is the source that you want to copy. Typically, this will be a device file representing a disk or partition. For example, if you're copying the entire contents of a USB drive, its device file might be `/dev/sdb`. This will capture **everything** from the drive — files, deleted data, and even empty space.
- **of= (output file):** This is where the bit-for-bit copy will be saved. You can specify a file on your local system, like `/root/flashcopy.img`, which will be the destination for the entire drive image. This image can then be mounted, analyzed, or restored onto another drive.

It's important to note that you don't need to know the location of a specific deleted file to use `dd`. Unlike other commands, `dd` doesn't operate at the filesystem level, so you

don't specify the path to individual files like `~/Documents/log.log` in the `if=` argument. Instead, you specify the **entire device** (e.g., `/dev/sdb` for a USB drive or `/dev/sda` for your primary hard drive). This way, `dd` copies everything on the device, including the raw data blocks where deleted files still reside.

So, if you've deleted a log file in `~/Documents/log.log`, you **wouldn't** put that specific file path in the `if=` part of the `dd` command. Instead, you would use the entire storage device (e.g., `/dev/sda`) as the input file, and `dd` would capture all the data, including deleted files that have not been overwritten – which is why it's slow, it captures ALL the data from the WHOLE device storage.

Since `dd` copies **every bit** on the device, including unused or empty areas, it takes time. For example, if you have a 500GB hard drive, `dd` will copy all 500GB of the drive, even if only 100GB is actually used for files. This makes it slower than commands like `cp` that only copy files the filesystem knows about.

For example, if you want to create a bit-for-bit copy of a USB drive (`/dev/sdb`) and save it as an image file (`/root/flashcopy.img`), you would run:

```
kali > dd if=/dev/sdb of=/root/flashcopy.img
```

In this command:

- `if=/dev/sdb`: Specifies the input file as the USB drive. This means you are copying the entire contents of the USB drive, including any deleted files that haven't been overwritten.
- `of=/root/flashcopy.img`: Specifies the output file as an image (`.img`) that will store the exact copy of the USB drive.

The output of `dd` can be saved as a **raw disk image file**, typically not a `.tar` file, but an **image file** (often with extensions like `.img` or `.bin`). This image is essentially a bit-for-bit replica of the entire device — it's exactly the same size as the original storage device. For example:

```
kali > dd if=/dev/sdb of=/root/flashcopy.img
```

- `if=/dev/sdb`: This is the input, specifying the entire USB drive.
- `of=/root/flashcopy.img`: This is the output file where the image will be saved. The output file will be as large as the drive you're copying (e.g., if the USB drive is 16GB, the `.img` file will be 16GB).

Once you have a disk image (e.g., `flashcopy.img`), you can:

- Mount the image as a filesystem to explore and recover files.
- Use forensic tools to recover deleted files from the image (since the image contains all the deleted data too).
- Restore the image to another storage device to recreate an exact copy of the original drive.

Because the `dd` output can be very large, it's important to have enough disk space to store the resulting image. For example, if you copy a 500GB drive, you will need at least 500GB of free space on the target drive where the image is being saved.

You can compress the image to reduce its size by piping the output of `dd` through a compression tool like `gzip` or `bzip2`. Here's an example:

```
kali > dd if=/dev/sdb | gzip > /root/flashcopy.img.gz
```

This command does the following:

- `if=/dev/sdb`: Specifies the USB drive as input.
- `| gzip >`: Pipes the output of `dd` through `gzip` for compression.

- `/root/flashcopy.img.gz`: Saves the compressed image file.

The `dd` command is extremely flexible and can be customized with several options:

- **`bs=block_size`**: This allows you to specify the block size (the number of bytes read or written at once). For example, using `bs=4096` sets the block size to 4KB, which can speed up copying for larger drives.
- **`conv=noerror`**: This tells `dd` to continue copying even if errors are encountered. It's especially useful when copying damaged drives or media.
- **`status=progress`**: Adding this option shows you the progress of the copy operation, which is helpful since `dd` can be slow.

For example, to copy a damaged USB drive (`/dev/sdb`) with a 4KB block size and ensure that errors don't stop the process, you could use:

```
kali > dd if=/dev/sdb of=/root/flashcopy.img bs=4096 conv=noerror status=progress
```

Chapter Summary

In this chapter, we explored various ways to archive and compress files using Linux tools like `tar`, `gzip`, `bzip2`, and `compress`. We also learned about the powerful `dd` command, which allows for copying entire storage devices, including deleted data. These tools are essential in a hacker's toolkit for efficiently managing data transfers and conducting forensic investigations.

Practice Exercises

Before proceeding to Chapter 10, take the opportunity to practice the skills acquired in this chapter through the following exercises:

1. Create three scripts:

- Create three bash script files named `CommandLineHacker1.sh`, `CommandLineHacker2.sh`, and `CommandLineHacker3.sh`.
- Each file should contain a simple script that echoes a unique message, such as `"This is CommandLineHacker1"`.
- Save each file and ensure they are executable.

Solution:

```
nano CommandLineHacker1.sh
# Add the following inside the file
#!/bin/bash
echo "This is CommandLineHacker1"
```

Repeat for `CommandLineHacker2.sh` and `CommandLineHacker3.sh`, replacing the message in the `echo` command.

- Make the scripts executable:

```
chmod +x CommandLineHacker1.sh CommandLineHacker2.sh
CommandLineHacker3.sh
```

2. Generate a tarball and observe the size:

- Archive the three files (`CommandLineHacker1.sh`, `CommandLineHacker2.sh`, `CommandLineHacker3.sh`) into a tarball named `CLH.tar`.
- Check and observe the size of the tarball after creation.

Solution:

```
tar -cvf CLH.tar CommandLineHacker1.sh CommandLineHacker2.sh  
CommandLineHacker3.sh  
ls -lh CLH.tar
```

3. Compress the tarball using gzip:

- Compress the CLH.tar file using gzip.
- Note the size difference between the tar file and the compressed file.
- Uncompress the tarball afterward.

Solution:

```
gzip CLH.tar  
ls -lh CLH.tar.gz # Observe file size  
gunzip CLH.tar.gz # Uncompress the file
```

4. Repeat compression using bzip2 and compress:

- Compress the tarball CLH.tar using bzip2 and check the file size.
- Uncompress the file.
- Next, compress the tarball using compress and note the file size.
- Uncompress the file.

Solution:

Using bzip2:

```
bzip2 CLH.tar
ls -lh CLH.tar.bz2 # Observe file size
bunzip2 CLH.tar.bz2 # Uncompress the file
```

Using compress:

```
compress CLH.tar
ls -lh CLH.tar.Z # Observe file size
uncompress CLH.tar.Z # Uncompress the file
```

5. Use the dd command to create a bit-by-bit copy of a flash drive:

- Assume your flash drive is mounted at /dev/sdb.
- Use the dd command to create a bit-by-bit image of the flash drive and save it as flashdrive_backup.img.

Solution:

```
sudo dd if=/dev/sdb of=flashdrive_backup.img bs=4M
ls -lh flashdrive_backup.img # Check the size of the backup
```

10. MANAGING FILESYSTEMS AND STORAGE DEVICES

If you are coming from a Windows environment, the way that Linux represents and manages storage devices will look rather different to you. You've already seen that the filesystem has no physical representation of the drive, like the C:, D:, or E: system in

Windows, but rather has a file tree structure with / at the top, or root, of it. This chapter takes a look at how Linux represents storage devices such as hard drives, flash drives, and other storage devices.

We first look at how additional drives and other storage devices are mounted upon that filesystem, leading up to the / (root) directory. Mounting in this context simply means attaching drives or disks to the filesystem to make them accessible to the operating system (OS). For you as a hacker, it's necessary to understand the file and storage device management system, both on your own system and, often, the system of your target. Hackers commonly use external media to load data, hacking tools, or even their OS.

Once you're on your target system, you need to understand what you're working with, where to find confidential or other critical files, how to mount a drive to the target, and whether and where you can put those files on your system. We cover all of these topics, plus how to manage and monitor storage devices, in this chapter.

We begin with the directory known as /dev, which you've probably already noticed in the directory structure: dev is short for device, and every device in Linux is represented by its own file within the /dev directory. Let's start out by working with /dev.

Overview of /dev Directory

Linux has a special directory that contains files representing each attached device: the appropriately named /dev directory. As your first introduction, navigate to the /dev directory and then perform a long listing on it. You should see something like Code 10-1.

```
kali >cd /dev
```

```
kali >ls -l
```

```
total 0
```

```
crw----- 1 root root 10,175 May 16 12:44 agpgart
```

```
crw----- 1 root root 10,235 May 16 12:44 autofs
```

```
drwxr-xr-x 1 root root 160 May 16 12:44 block
```

```
lrwxrwxrwx 1 root root 3 May 16 12:44 cdrom -> sr0
```

```
drwxr-xr-x 2 root root 60 May 16 12:44 cpu
```

Code 10-1: A long listing of the /dev directory

The devices are displayed in alphabetical order by default. You may recognize some of the devices, such as `cdrom` and `cpu`, but others have rather cryptic names. Each device on your system is represented by a file in the `/dev` directory, including devices you've probably never used or even realized existed. On the off chance you do, there is a device file waiting to be used for it.

If you scroll down this screen a bit, you should see more listings of devices. Of particular interest are the devices `sda1`, `sda2`, `sda3`, `sdb`, and `sdb1`, which are the hard drive and its partitions and a USB flash drive and its partitions.

```
brw-rw---- 1 root root 8, 0 May 16 12:44 sda
brw-rw---- 1 root root 8, 1 May 16 12:44 sda1
brw-rw---- 1 root root 8, 2 May 16 12:44 sda2
brw-rw---- 1 root root 8, 5 May 16 12:44 sda5
brw-rw---- 1 root root 8, 16 May 16 12:44 sdb
brw-rw---- 1 root root 8, 17 May 16 12:44 sdb1
```

Let's take a closer look at these.

Representation of Storage Devices in Linux

Linux uses logical labels for drives that are then mounted on the filesystem. These logical labels will vary depending on where the drives are mounted, meaning the same hard drive might have different labels at different times, depending on where and when it's mounted.

Originally, Linux represented floppy drives (remember those?) as `fd0` and hard drives as `hda`. You will still occasionally see these drive representations on legacy Linux systems, but today most floppy drives are gone (thank goodness). Even so, old legacy hard drives

that used an IDE or E-IDE interface are still represented in the form hda. Newer Serial ATA (SATA) interface drives and Small Computer System Interface (SCSI) hard drives are represented as sda. Drives are sometimes split up into sections known as partitions, which are represented in the labeling system with numbers, as you'll see next.

When systems have more than one hard drive, Linux simply names them serially by incrementing the last letter in alphabetical order, so the first drive is sda, and the second drive is sdb, the third drive is sdc, and so on (see Table 10-1). The serial letter after sd is often referred to as the major number.

Table 10-1: Device-Naming System

Device file	Description
sda	First SATA hard drive
sdb	Second SATA hard drive
sdc	Third SATA hard drive
sdd	Fourth SATA hard drive

Understanding Drive Partitions

Some drives can be split into partitions in order to manage and separate information. For instance, you may want to separate your hard drive so that your swap file, home directory, and / directory are all on separate partitions—you might want to do this for a number of reasons, including to share resources and to relax the default permissions. Linux labels each partition with a minor number that comes after the drive designation. This way, the first partition on the first SATA drive would be sda1. The second partition would then be sda2, the third sda3, and so on, as illustrated in Table 10-2.

Table 10-2: Partition-Labeling System

Partition	Description
sda1	The first partition (1) on the first (a) SATA drive
sda2	The second (2) partition on the first (a) drive
sda3	The third (3) partition on the first (a) drive
sda4	The fourth (4) partition on the first (a) drive

At times, you may want to view the partitions on your Linux system to see which ones you have and how much capacity is available in each. You can do this by using the fdisk utility. Using the -l switch with fdisk lists all the partitions of all the drives, as shown in Code 10-2.

```
kali >fdisk -l
Disk /dev/sda: 20GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x7c06cd70
Device Boot Start End Sectors Size Id Type
/dev/sda1 * 2048 39174143 39172096 18.7G 83 Linux
/dev/sda2 39176190 41940991 2764802 1.3G 5 Extended
/dev/sda5 39176192 41940991 2764800 1.3G 82 Linux swap / Solaris

Disk /dev/sdb: 29.8 GiB, 31999393792 bytes, 62498816 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0xc3072e18
Device Boot Start End Sectors Size Id Type
/dev/sdb1 32 62498815 62498784 29.8G 7 HPFS/NTFS/exFAT
```

Code 10-2: Code partitions with fdisk

As you can see in Code 10-2, the devices sda1, sda2, and sda5 are listed in the first stanza. These three devices make up the virtual disk from my virtual machine, which is a 20GB drive with three partitions, including the swap partition (sda5), which acts like virtual RAM—similar to page files in Windows—when RAM capacity is exceeded.

If you scan down Code 10-2 to the third stanza, you see a second device output designated sdb1—the b label tells us that this drive is separate from the first three devices. This is my 64GB flash drive. Note that fdisk indicates that it is an HPFS/NTFS/ExFAT filesystem type. These file types—High Performance File System (HPFS), New Technology File System (NTFS), and Extended File Allocation Table (exFAT)—are not native to Linux systems but rather to macOS and Windows systems. It's worth being able to recognize file types native to different systems when you investigate. The filesystem might indicate what kind of machine the drive was formatted on, which can be valuable information. Kali is able to utilize USB flash drives created on many different operating systems.

As you saw in Chapter 1, the Linux filesystem is structured significantly differently than are Windows and other proprietary operating systems. On top of this, the way files are stored and managed is different in Linux, too. New versions of Windows use an NTFS filesystem, whereas older Windows systems use File Allocation Table (FAT) systems. Linux uses a number of different types of filesystems, but the most common are ext2, ext3, and ext4. These are all iterations of the ext (or extended) filesystem, with ext4 being the latest.

Differentiating Character and Block Devices

Something else to note about the naming of device files in the /dev directory is that the first position contains either c or b. You can see this in Code 10-1 at the start of most of the entries, and it looks something like this:

```
crw----- 1 root root 10,175 May 16 12:44 agpgart
```

These letters represent the two ways that devices transfer data in and out. The c stands for character, and these devices are known, as you might expect, as character devices. External devices that interact with the system by sending and receiving data character by character, such as mice or keyboards, are character devices.

The b stands for the second type: block devices. They communicate in blocks of data (multiple bytes at a time) and include devices like hard drives and DVD drives. These devices require higher-speed data throughput and therefore send and receive data in blocks (many characters or bytes at a time). Once you know whether a device is a character or block device, you can easily get more information about it, as you'll see next.

Using lsblk for Device Information

The Linux command lsblk, short for list block, provides essential information about each block device found in the /dev directory. This command yields results akin to the output of fdisk -l but presents the information in a tree-like structure, displaying devices with multiple partitions as branches. Unlike some other commands, lsblk doesn't require root privileges to run. In Code 10-3 below, you can see an example of lsblk in action:

```
kali >lsblk
Name MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
fd0 2:0 1 4K 0 disk
sda1 8:0 0 20G 0 disk
|-sda1 8:1 0 18.7G 0 part /
|-sda2 8:2 0 1K 0 part
|-sda5 8:5 0 1.3G 0 part [SWAP]
sdb 8:16 1 29.8G 0 disk
|-sdb1 8:17 1 29.8G 0 disk /media
sr0 11:0 1 2.7G 0 rom
```

Code 10-3: Code block device information with lsblk

In the output, you can observe devices such as the floppy drive (fd0) and DVD drive (sr0), even if they are not present in my system. These entries are remnants from legacy systems. Additionally, the output provides information about the mount points of the drives. The mount point signifies the location at which the drive has been attached to

the filesystem. Notably, the hard drive sda1 is mounted at /, while the flash drive is mounted at /media. The significance of mount points will become clearer in the next section.

Procedures for Mounting and Unmounting

In most modern operating systems, including contemporary Linux distributions, storage devices are often automounted upon connection. This means that when you attach a new flash drive or hard drive, it is automatically integrated into the filesystem. However, for those who are new to Linux, the concept of mounting may be unfamiliar.

For data to become accessible to the operating system, a storage device must undergo two steps: physical connection to the filesystem and logical attachment to the filesystem. In essence, even if a device is physically connected to the system, it may not be logically attached and available to the operating system. The term "mount" originates from the early days of computing when storage tapes (before hard drives) had to be physically "mounted" onto computer systems, akin to the large computers with spinning tape drives featured in old science fiction films.

As mentioned earlier, the location within the directory tree where devices are attached is termed the "mount point." In Linux, the two primary mount points are /mnt and /media. As a general convention, internal hard drives are mounted at /mnt, while external USB devices, including flash drives and external USB hard drives, are mounted at /media. However, it's essential to note that technically, any directory can be used as a mount point.

Manual Mounting of Storage Devices

In some Linux distributions, it may be necessary to manually mount a drive to access its contents. Therefore, learning how to perform manual mounts is a valuable skill. To mount a drive onto the filesystem, use the "mount" command. The designated mount point for the device should be an empty directory. If you mount a device on a directory that contains subdirectories and files, the mounted device will overlay the contents of that directory, rendering them invisible and inaccessible. To illustrate, if you wish to mount the new hard drive sdb1 at the /mnt directory, execute the following command:

```
kali >mount /dev/sdb1 /mnt
```


Subsequently, the hard drive should be available for access. If you intend to mount the flash drive sdc1 at the /media directory, use this command:

```
kali >mount /dev/sdc1 /media
```

The filesystems mounted on a system are recorded in a file located at /etc/fstab (short for filesystem table). This file is read by the system during each bootup.

Safe Unmounting with umount

If you come from a Mac or Windows background, you may have inadvertently unmounted a drive. Before removing a flash drive from your system, you often "eject" it to prevent potential damage to the stored files. "Ejecting" is synonymous with unmounting.

Similar to the mount command, you can initiate the unmounting process using the "umount" command followed by the file entry of the device found in the /dev directory. It's crucial to note that the command is spelled "umount," without the letter "n":

```
kali >umount /dev/sdb1
```

It's important to remember that you cannot unmount a device that is currently in use. If the system is reading from or writing to the device, attempting to unmount it will result in an error.

Monitoring the Filesystem Health

In this section, we will explore several commands for monitoring the state of the filesystem, a skill that is essential for hackers and system administrators alike. We will

obtain information about mounted disks and address any potential errors. Storage devices are susceptible to errors over time, making it worthwhile to acquire this skill.

Information on Mounted Disks

The "df" command, which stands for "disk free," furnishes basic information about hard disks or mounted devices, including CDs, DVDs, and flash drives. It provides details such as the used and available space. The default behavior of "df" is to display information about the first drive on the system (typically sda). To check a different drive, simply follow the "df" command with the representation of the desired drive, as shown in Code 10-4:

```
kali >df
Filesystem 1K-Blocks Used Available Use% Mounted on
rootfs 19620732 17096196 1504788 92% /
udev 10240 0 10240 0% /dev

/dev/sdb1 29823024 29712544 110480 99% /media/USB3.0
```

Code 10-4: Obtaining information on disks and mounted devices with df

The initial line of output presents category headers, followed by the information. Disk space is reported in 1KB blocks. In the second line of output, we see that rootfs consists of 19,620,732 one-kilobyte blocks, with 17,096,196 blocks (approximately 92%) in use and 1,504,788 blocks available. The "df" command also informs us that this filesystem is mounted at the root of the filesystem (/).

The last line illustrates my USB flash drive, labeled /dev/sdb1. Notably, it is nearly 100% full and is mounted at /media/USB3.0.

To recap, on this system, my virtual disk is designated as sda1 and breaks down as follows:

- "sd": SATA hard drive
- "a": First hard drive
- "1": First partition on that drive

My 64GB flash drive is labeled sdb1, and my external drive is identified as sdc1.

Disk Error Checking

The "fsck" command, an abbreviation of "filesystem check," is employed to inspect the filesystem for errors and, if feasible, repair any damage. In cases of irreparable damage, it marks the problematic area as bad and adds it to the list of bad blocks. When running the "fsck" command, it is necessary to specify the filesystem type, with the default being ext2, and the device file to check. Importantly, you must unmount the drive before conducting a filesystem check. Failure to unmount a mounted device will lead to an error, as demonstrated in Code 10-5:

```
kali >fsck
fsck from util-linux 2.20.1
e2fsck 1.42.5 (29-Jul-2012)
/dev/sda1 is mounted
e2fsck: Cannot continue, aborting.
```

Code 10-5: Attempting (and failing) to run an error check on a mounted drive

Hence, the initial step before performing a filesystem check is to unmount the device. In this scenario, I will unmount my flash drive to proceed with the filesystem check:

```
kali >umount /dev/sdb1
```

To automate the repair of any issues with the device, you can add the "-p" option to the "fsck" command, as follows:

```
kali >fsck -p /dev/sdb1
```

With the device successfully unmounted, you can proceed to check for bad sectors or other problems with the device:

```
kali >fsck -p /dev/sdb1
fsck from util-linux 2.30.2
exfatfsck 1.2.7
Checking file system on /dev/sdb1.
File system version 1.0
Sector size 512 bytes
Cluster size 32 KB
Volume size 7648 MB
Used space 1265 MB
Available space 6383 MB
Totally 20 directories and 111 files.
File system checking finished. No errors found.
```

Chapter Summary

In summary, understanding how Linux identifies and manages its devices is essential for all Linux users and hackers. It is vital to determine which devices are connected to a system and the available storage space. Given that storage devices are prone to errors, the ability to check and rectify these issues using "fsck" is valuable. Moreover, the "dd" command can create a physical copy of a device, including any deleted files.

Practice Exercises

Before proceeding to Chapter 11, I encourage you to practice the skills you've acquired in this chapter with the following exercises:

- Utilize the "mount" and "umount" commands to mount and unmount your flash drive.

- Check the amount of free disk space on your primary hard drive.

- Use "fsck" to inspect your flash drive for errors.

- Employ the "dd" command to duplicate the entire contents of one flash drive to another, including deleted files.

- Utilize the "lsblk" command to retrieve basic information about your block devices.

11. EXPLORING THE SYSTEM LOGGING MECHANISM

For any Linux user, having a deep understanding of log files is crucial. Log files are repositories of information about events that occur during the operation of the operating system and applications. They record everything from routine operations to errors and security alerts. This chapter will guide you in configuring the rules that govern automatic logging.

As a hacker, log files can be a double-edged sword. They can reveal valuable insights into your target's activities and identity, but they can also expose your own actions on someone else's system. Therefore, it's essential for a hacker to know what information can be collected and how to obfuscate their own activities.

On the flip side, individuals responsible for securing Linux systems need to understand how to manage logging functions. This knowledge helps in identifying potential attacks, deciphering what transpired during an incident, and pinpointing the responsible party.

In this chapter, we'll delve into examining and configuring log files. You'll also learn techniques for erasing traces of your activity and, if necessary, disabling logging altogether. Let's begin by exploring the core component of logging, the rsyslog daemon.

Working with rsyslog Daemon

Linux relies on a daemon known as syslogd to automatically log events on your computer. Different Linux distributions use variations of syslog, such as rsyslog and syslog-ng, and although their functions are similar, there may be minor differences. Since Kali Linux is built on Debian, which comes with rsyslog as the default, we will focus on this utility in this chapter. If you're using a different distribution, it's advisable to research their logging systems.

To get started, let's take a closer look at rsyslog on your system. We'll search for all files related to rsyslog. In your Kali Linux terminal, enter the following command:

```
kali > locate rsyslog
```

The output will reveal numerous files containing the keyword "rsyslog," some of which are more relevant than others. The one we are interested in is the configuration file, "rsyslog.conf."

Configuring rsyslog

As with most applications in Linux, rsyslog is configured using a plaintext configuration file. In Linux fashion, this file is typically located in the "/etc" directory. For rsyslog, the configuration file can be found at "/etc/rsyslog.conf." Open this file with any text editor to explore its contents. Below is a snapshot of what you might see:

```
kali > leafpad /etc/rsyslog.conf
```

The rsyslog.conf file is well-documented, featuring numerous comments that explain its purpose. While much of this information may not be immediately relevant, scroll down to around line 50, where you'll encounter the "Rules" section. This is where you define the rules governing what your Linux system will automatically log.

Understanding rsyslog Rules

The rsyslog rules dictate what type of information is logged, which programs have their messages logged, and where those logs are stored. As a hacker, understanding these rules enables you to identify what is being logged and the locations of those logs so you can manipulate or eliminate them. Navigate to line 50, and you'll find something similar to the following:

```
#####  
#### RULES ####  
#####  
#  
# First some standard log files. Log by facility.
```

```
#
auth,authpriv.* varlog/auth.log
*.*;auth,authpriv.none -varlog/syslog
#cron.* varlog/cron.log
daemon.* -varlog/daemon.log
kern.* -varlog/kern.log
lpr.* -varlog/lpr.log
mail.* -varlog/mail.log
user.* -varlog/user.log
#
# Logging for the mail system. Split it up so that
# it is easy to write scripts to parse these files.
#
mail.info -varlog/mail.info
mail.warn -varlog/mail.warn
mail.err varlog/mail.err
```

Each line represents a separate logging rule, defining what messages are logged and where they are logged. The basic format of these rules is as follows:

```
facility.priority action
```

The "facility" keyword refers to the program or service generating the log, such as "mail," "kernel," or "lpr." "Priority" determines the level of messages to log, ranging from lowest (debug) to highest (panic). The "action" keyword specifies the location where the log is stored.

Valid facility codes include:

- auth/authpriv
- cron
- daemon
- kern
- lpr
- mail

- user

An asterisk (*) as a facility code refers to all facilities, and you can select multiple facilities by separating them with commas.

Priority codes include:

- debug
- info
- notice
- warning
- err (deprecated)
- crit
- alert
- emerg
- panic

The priority code you specify determines which messages are logged. For example, specifying "alert" would log messages classified as "alert" or higher but exclude messages marked as "crit" or any lower priority.

Here are some examples of log rules:

- `mail.* varlog/mail`: This rule logs mail events of all priorities to "varlog/mail."
- `kern.crit varlog/kernel`: This rule logs kernel events of critical (crit) priority or higher to "varlog/kernel."
- `*.emerg`: This rule logs all events of emergency (emerg) priority to all logged-on users. With these rules, a hacker can identify the locations of log files, modify priorities, or even disable specific logging rules. Understanding these rules is a key step in managing and manipulating the logging system in Linux.

Log Management with logrotate

Log files occupy valuable disk space, and if left unchecked, they can eventually consume your entire hard drive. Conversely, deleting log files too frequently deprives you of important historical data for future investigation. To strike a balance, you can employ logrotate to efficiently manage your logs.

Log rotation is the practice of regularly archiving log files by relocating them to another location, leaving you with a fresh log file. These archived logs can then be cleaned up after a predefined period.

Your system is already configured to rotate log files using a cron job that utilizes the logrotate utility. You can customize the log rotation frequency by editing the "etc/logrotate.conf" text file. Let's take a look at this configuration file by opening it with a text editor:

```
kali > leafpad /etc/logrotate.conf
```

You'll encounter something similar to what's shown in Code 11-3.

```
# See "man logrotate" for details
# Rotate log files weekly
weekly
# Keep 4 weeks worth of backlogs
rotate 4
# Create new (empty) log files after rotating old ones
create
# Uncomment this if you want your log files compressed
# compress
# Packages drop log rotation information into this directory
include /etc/logrotate.d
# No packages own wtmp, or btmp -- we'll rotate them here
/var/log/wtmp {
    missingok
    monthly
    create 0664 root utmp
    rotate 1
}
```

You can specify the time unit for the rotation numbers . By default, it's set to weekly, meaning that any number following the "rotate" keyword refers to weeks.

Further down, you can configure how frequently logs are rotated. The default is every four weeks, which suits most users. However, you can adjust this setting to match your needs. For example, changing it to "rotate 1" keeps logs for a week, while "rotate 26" retains logs for six months, and "rotate 52" preserves them for a year.

By default, a new empty log file is created when old ones are rotated out. You can also choose to compress rotated log files.

At the end of each rotation period, log files are renamed and moved to the end of the log chain, while a new log file is created, effectively replacing the current log file. For instance, "var/log/auth" becomes "var/log/auth.1," then "var/log/auth.2," and so forth. If you rotate logs every four weeks and keep four sets of backups, you'll have "var/log/auth.4," but there won't be a "var/log/auth.5," as "var/log/auth.4" will be deleted instead of being moved to "var/log/auth.5." You can verify this using the "locate" command, as demonstrated:

```
kali > locate /var/log/auth.log.*  
var/log/auth.log.1  
var/log/auth.log.2  
var/log/auth.log.3  
var/log/auth.log.4
```

For more comprehensive customization and usage details of the logrotate utility, refer to the "man logrotate" page. It's a valuable resource to explore the available functions and variables for tailoring your log management. As you gain more experience with Linux, you'll refine your logging frequency and preferences, making it worthwhile to revisit the logrotate.conf file.

Techniques for Remaining Stealthy

After successfully compromising a Linux system, it becomes imperative to disable logging and eliminate any traces of your intrusion from the log files. Achieving this requires careful consideration of various methods, each carrying its own risks and levels of reliability.

Removing Log Traces

Initially, you'll want to eradicate any records of your activities. Although manually deleting log entries line by line is an option, it can be time-consuming and may leave noticeable gaps in the log files, raising suspicions. Additionally, skilled forensic investigators can often recover deleted files.

A more secure approach is to "shred" the log files. Unlike traditional file deletion methods, which can still leave traces, shredding involves overwriting the file multiple times, making recovery extremely challenging. Fortunately, Linux includes a built-in command for this purpose, aptly named "shred."

To understand how the "shred" command operates, you can review its help screen with the following command:

```
kali > shred --help
```

The "shred" command offers various options, but its basic syntax is straightforward: "shred <FILE>." When used without additional options, "shred" deletes the file and overwrites it several times, typically four times. However, more overwrites increase the difficulty of data recovery, but they also require more time, particularly for large files.

Two useful options are the "-f" option, which adjusts permissions if necessary to allow overwriting, and the "-n" option, enabling you to specify the number of overwrites. For example, you can shred log files in "var/log/auth.log" ten times with the following command:

```
kali > shred -f -n 10 /var/log/auth.log.*
```

The "-f" option grants permission for shredding "auth" files, while the "-n" option defines the number of overwrites. The wildcard asterisk ensures that not only "auth.log" but also logs created by logrotate, such as "auth.log.1," "auth.log.2," and so on, are shredded.

Upon shredding a file, its contents become indecipherable gibberish, as illustrated in Image 11-1.



Now, should a security engineer or forensic investigator inspect the log files, they will find nothing recoverable—effectively erasing all traces of your activities.

Disabling System Logging

Another strategy for concealing your activities is to disable logging entirely. When a hacker gains control of a system, one of their immediate actions might be to disable logging to prevent the system from recording their activities. However, it's important to note that disabling logging typically requires root privileges.

To disable all logging, a hacker can stop the rsyslog daemon. In Linux, stopping any service follows a common syntax, as demonstrated here (further elaborated in Chapter 12):

```
service servicename start|stop|restart
```

So, to halt the logging daemon, you can execute the following command:

```
kali > service rsyslog stop
```

Now, Linux will cease generating any log files until the service is restarted. This allows you to operate without leaving any traces in the log files.

Chapter Summary

Log files serve as comprehensive records of nearly everything that transpires on your Linux system. They are invaluable for analyzing various events, whether they involve system malfunctions or security breaches. For a hacker, log files can potentially expose their actions and identity. However, a savvy hacker can take steps to delete and shred these files and even disable logging altogether, leaving behind no digital breadcrumbs.

Practice Exercises

Before proceeding to Chapter 12, I encourage you to apply the knowledge gained in this chapter through the following exercises:

- Utilize the "locate" command to identify all rsyslog-related files on your system.
- Open the "rsyslog.conf" file and adjust your log rotation settings to one week.
- Experiment with disabling logging on your system and investigate the contents of the "varlog/syslog" file when logging is turned off.
- Employ the "shred" command to securely shred and delete all your kernel log files.

12. MANAGING AND EXPLOITING SERVICES

In Linux terminology, a service refers to an application that quietly runs in the background, awaiting your commands. Your Linux system comes preloaded with numerous services, with the most notable being the ubiquitous Apache Web Server. Apache is renowned for creating, managing, and deploying web servers, but there are several other essential services, including OpenSSH, MySQL, and PostgreSQL, that hold special significance for hackers.

In this chapter, we'll delve into setting up a web server with Apache, conducting surveillance with OpenSSH, accessing data through MySQL, and securely storing hacking information with PostgreSQL.

Service Management: Start, Stop, and Restart

Before we delve into these four pivotal services, let's first explore the basics of starting, stopping, and restarting services in Linux.

While some services can be managed through the graphical user interface (GUI) in Kali Linux, others require the use of the command line, which we'll discuss here. The fundamental syntax for managing services is as follows:

```
service servicename start|stop|restart
```

To initiate the apache2 service (HTTP service or web server), use the following command:

```
kali > service apache2 start
```

To halt the Apache web server, enter:

```
kali > service apache2 stop
```

Typically, when you make configuration changes to an application or service by modifying its plaintext configuration file, you'll need to restart the service to apply the new settings. To achieve this, use the following command:

```
kali > service apache2 restart
```

Now that you're familiar with starting, stopping, and restarting services from the command line, let's proceed to explore the four most crucial Linux services for hackers.

Setting Up an HTTP Server with Apache

The Apache Web Server is arguably the most prevalent service on Linux systems, being present on over 60 percent of the world's web servers. As a hacker aspiring to manipulate websites, it's imperative to comprehend Apache's inner workings, website dynamics, and backend databases. Additionally, Apache can be utilized to establish your web server, which could serve malicious content via cross-site scripting (XSS) or replicate a website to divert traffic through DNS abuse. In both scenarios, a fundamental understanding of Apache is indispensable.

Initial Setup with Apache

If you have Kali Linux running on your system, Apache is likely already installed. Many other Linux distributions also include Apache by default. If you don't have Apache installed, you can obtain and install it from the repositories using the following command:

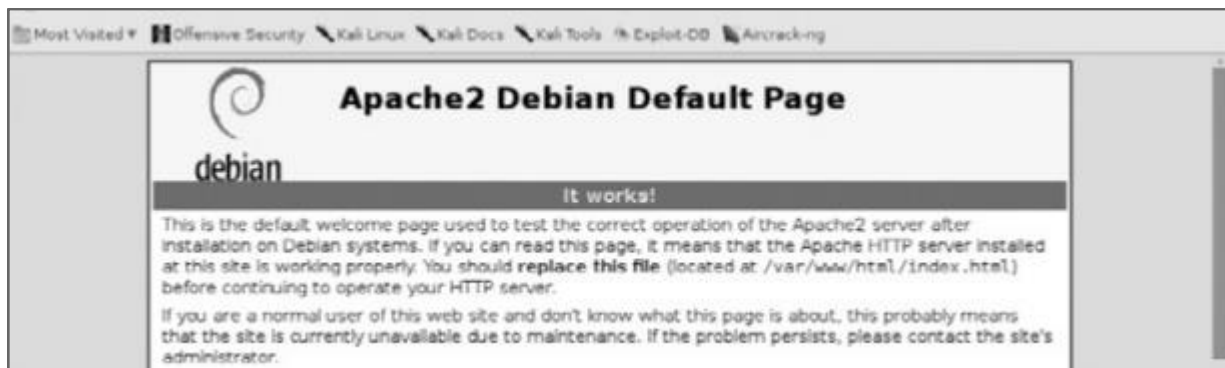
```
kali > apt-get install apache2
```

The Apache Web Server often goes hand in hand with the MySQL database, frequently accompanied by a scripting language like Perl or PHP for web application development. This combination, known as LAMP (Linux, Apache, MySQL, and PHP/Perl), is a powerful platform for creating and deploying web-based applications.

To initiate the Apache daemon, you can either use the Kali GUI by going to "Applications" ▸ "Services" ▸ "HTTPD" and clicking on "Apache start" or use the command line:

```
kali > service apache2 start
```

With Apache now running, it should serve its default web page. To access it, simply enter <http://localhost/> into your web browser, resulting in a page resembling Image 12-1.



The default Apache page displays "It works." This confirms that your Apache Web Server is operational. Now, let's move on to customize it further.

Customizing index.html

Apache's default web page resides at `var/www/html/index.html`. You can modify this `index.html` file to serve any content you desire. Any text editor can be used for this task; I'll be using Leafpad. Open `var/www/html/index.html`, and you'll see something akin to Code 12-1.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transiti
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
```



```

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" I>
  <title>Apache2 Debian Default Page: It works</title>
<style type="text/css" media="screen">
* {
margin: 0px 0px 0px 0px;
padding: 0px 0px 0px 0px;
}
body, html {
padding: 3px 3px 3px 3px;
background-color: #D8DBE2;
font-family: Verdana, sans-serif;
font-size: 11pt;
text-align: center;
}
div.main_page {
position: relative;
display: table;

```

In this file, you can see that the default web page displays "It works" in HTML format . To customize your web server's content, simply edit or replace this file with your desired information.

As you can see, Apache's default page confirms that your Apache Web Server is operational. Now, it's time to make it your own by editing the index.html file to display the content you desire.

Basic HTML Usage

Now that we have our web server up and running with the index.html file open, it's time to add the content we want the web server to serve. We'll be creating some straightforward HTML elements.

Let's create this webpage. In a new file within your text editor, input the code exactly as shown in Code 12-2:

```

<html>
<body>
<h1>SecurityAiLab Is the Best!</h1>
<p>If you want to learn hacking, securityailab.com</p>

```

```
<p>is the best place to learn hacking!</p>
</body>
</html>
```

After entering the code as presented in Code 12-2, save this file as `varwww/html/index.html` and then close your text editor. Your text editor may prompt you that the file already exists, which is fine. Just proceed to overwrite the existing `varwww/html/index.html` file.

Observing Server Response

With our `varwww/html/index.html` file saved, we can now observe what Apache will serve. Open your web browser once again and navigate to <http://localhost>. You should witness a webpage.

Apache has served our webpage exactly as we designed it!

OpenSSH and the Raspberry Spy Pi

SSH, or Secure Shell, is the secure method by which we establish connections to remote system terminals, replacing the insecure telnet of the past. SSH allows us to create access lists, authenticate users with encrypted passwords, and encrypt all communications, reducing the risk of unauthorized access and interception of data. OpenSSH, a widely used Linux SSH service, is typically included in almost every Linux distribution, including Kali.

System administrators often employ SSH for managing remote systems, while hackers may use it to connect to compromised remote systems. In this example, we'll utilize SSH to set up a remote Raspberry Pi system for a unique purpose we'll refer to as the "Raspberry Spy Pi." For this endeavor, you'll need a Raspberry Pi and its accompanying Raspberry Pi camera module.

Before we proceed, initiate the OpenSSH service on your Kali system using the familiar command:

```
kali > service ssh start
```

We'll utilize SSH to construct and oversee a remote spying Raspberry Pi. If you're not yet acquainted with it, the Raspberry Pi is a compact yet powerful computer, roughly the size of a credit card, ideal for remote surveillance. In our setup, we'll employ a Raspberry Pi equipped with a camera module to create a remote spying device. You can easily acquire a Raspberry Pi from various electronics retailers, including Amazon, for less than \$50, and the camera module for approximately \$15.

In our scenario, we'll employ the Raspberry Spy Pi on the same network as our Kali system, enabling the use of private, internal IP addresses. It's important to note that for real-world hacking, you'd likely set it up on a remote network, which involves more complexities beyond the scope of this book.

Configuring a Raspberry Pi for Surveillance

Ensure that your Raspberry Pi is running the Raspbian operating system, a Linux distribution tailored specifically for the Raspberry Pi CPU. You can find instructions for downloading and installing Raspbian at <https://www.raspberrypi.org/downloads/raspbian/>. Almost everything you've learned in this book applies equally to the Raspbian OS on the Raspberry Pi, as well as to Kali, Ubuntu, and other Linux distributions.

Once you have Raspbian OS installed, connect your Raspberry Pi to a monitor, mouse, keyboard, and the internet. If this is new to you, refer to the instructions at <https://www.raspberrypi.org/learning/hardware-guide/>. Once everything is set up, log in using the username "pi" and the password "raspberry."

Image 12-3 (You can add an image here as per the original text)

With this setup, you're ready to proceed with your Raspberry Spy Pi project.

Preparing the Raspberry Spy Pi

The initial step involves ensuring that SSH is both running and enabled on your Raspberry Spy Pi. SSH is typically disabled by default, so you'll need to enable it. Navigate to the Preferences menu and launch Raspberry Pi Configuration. Proceed to

the Interfaces tab, and adjacent to SSH, ensure that it's enabled (check the box if it's not already) and click OK.

Once SSH is enabled, initiate it on your Raspberry Spy Pi by opening a terminal and entering the following command:

```
kali > service ssh start
```

Next, you'll need to attach your camera module. If you're using a Raspberry Pi version 3 board, there's only one designated connection point. First, power off the Pi, attach the camera module to the camera port, and then power it on again. It's vital to handle the camera module with care, ensuring it doesn't come into contact with the general-purpose input/output (GPIO) pins, as any contact could lead to a short circuit and potential damage.

Now, with the SSH service operational, place the Raspberry Spy Pi in the location you wish to monitor, whether it's at home, school, or another target area. It must be connected to the local area network, either via Ethernet cable or, ideally, through Wi-Fi. (Both the Raspberry Pi 3 and Raspberry Pi Zero come with built-in Wi-Fi.)

Subsequently, you'll need to ascertain the IP address of your Raspberry Pi. As explained in Chapter 3, you can retrieve a Linux device's IP address using the "ifconfig" command:

```
pi > ifconfig
```

In my case, the IP address of my Pi is 157.147.165.59, but it's crucial to use the specific IP address of your Raspberry Spy Pi throughout this chapter. Now, from your Kali system, you should be able to establish a direct connection to your Raspberry Spy Pi, allowing you to control it remotely for surveillance purposes.

In this basic scenario, both your system and the Pi need to be on the same network. To connect to the remote Raspberry Spy Pi via SSH from your Kali system, execute the

following command, ensuring that you replace my IP address with the one assigned to your Pi:

```
kali >ssh pi@157.147.165.59
pi@157.147.165.59's password:
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in usrshare/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, the extent
permitted by applicable law
last login: Fri Feb. 4 11:26:10 2024
pi@raspberrypi:: $
```

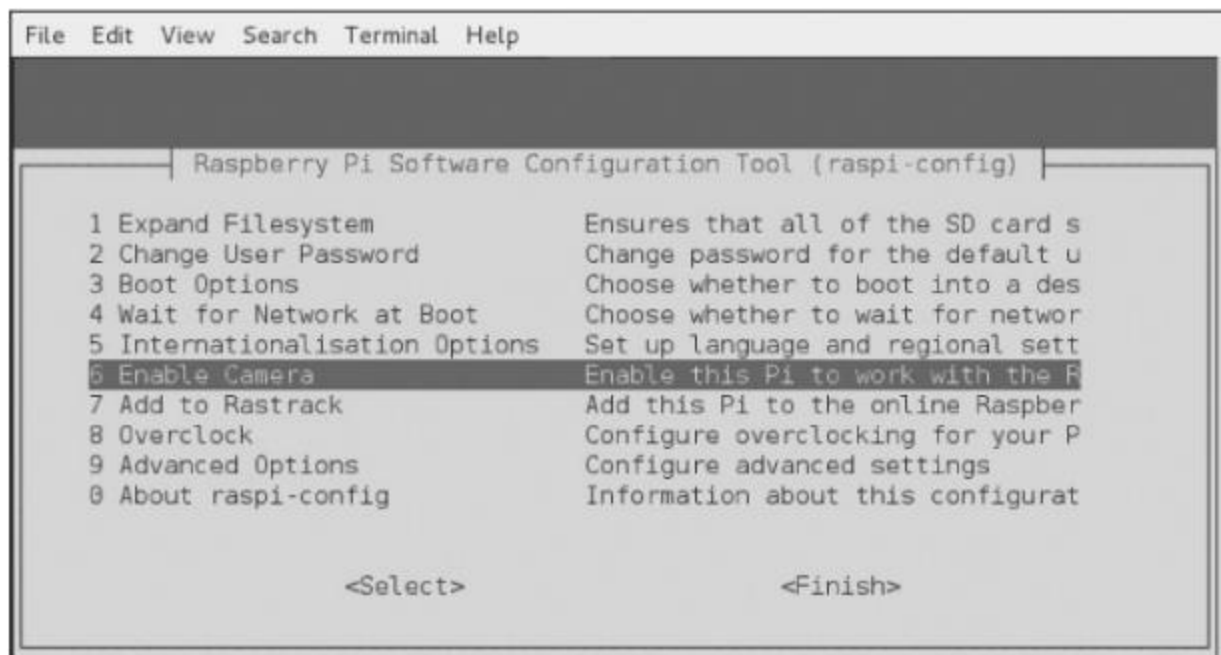
The system will prompt you for a password. In this instance, the default password is "raspberrypi" unless you've modified it.

Camera Configuration

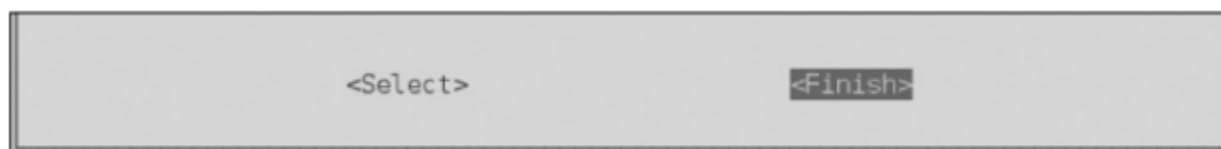
Next, we'll configure the camera. To begin, launch the Raspberry Pi configuration tool using the following command:

```
pi > sudo raspi-config
```

This should initiate a graphical menu resembling Image 12-3.



Scroll down to option 6, "Enable Camera," and press ENTER. Afterward, navigate to the bottom of the menu and select "Finish," then press ENTER, as depicted in Image 12-4.



When the configuration tool inquires if you wish to reboot (as shown in Image 12-5), select "Yes," and press ENTER once more.



Your Raspberry Spy Pi's camera should now be enabled and ready for surveillance purposes!

Initiating Surveillance

Once your Raspberry Spy Pi has completed its reboot and you've successfully logged in to it via SSH from your Kali terminal, you're all set to commence your espionage operations by capturing still images.

The Raspbian operating system boasts an application known as "raspistill," which will be our tool of choice for capturing images using our trusty Raspberry Spy Pi. To get familiar with raspistill's functionalities and available options, simply enter "raspistill" into your terminal:

```
pi@raspberrypi: raspistill
raspistill Camera App v1.3.8
Runs camera for a specific time, and takes a JPG capture at the end if requested
usage: raspistill [options]
Image parameter commands
```

Now, let's employ the Raspberry Spy Pi to capture remote surveillance images. The `raspistill` command boasts a plethora of options for customization, but for now, let's stick with the defaults. To take a picture and save it as a JPEG, execute the following command:

```
pi@raspberrypi: raspistill -v -o firstpicture.jpg
raspistill Camera App v1.3.8
width 2592, Height 1944, quality 85, filename firstpicture.jpg
Time delay 5000, Raw no
```

We've used the `-v` option to enable verbose output and the `-o` option to specify the filename for the captured image. As you can see, the file `"firstpicture.jpg"` has been created.

To confirm the presence of our very first spy picture on our remote Raspberry Spy Pi via SSH, let's perform a long listing:

```
pi@raspberrypi: ls -l
total 2452
drwxr-xr-x 2 pi pi 4096 Mar 18 2024 Desktop
drwxr-xr-x 2 pi pi 4096 Mar 18 2024 Documents
drwxr-xr-x 2 pi pi 4096 Mar 18 2024 Downloads
-rw-r--r-- 1 pi pi 2472219 Mar 18 2024 firstpicture.jpg
drwxr-xr-x 2 pi pi 4096 Mar 18 2024 Music
drwxr-xr-x 2 pi pi 4096 Mar 18 2024 Pictures
```

We've successfully captured our initial espionage image on the remote Raspberry Spy Pi using SSH. Feel free to further explore the capabilities of this versatile tool.

MySQL Information Extraction

MySQL stands as the predominant database system underpinning database-driven web applications. In the modern realm of Web 2.0 technologies, where virtually every

website relies on databases, MySQL reigns supreme as the repository for a substantial portion of the web's data.

Databases hold the keys to the kingdom for hackers, containing vital user information and sensitive data such as credit card numbers. Consequently, databases are frequently the primary targets of hackers. MySQL, akin to Linux, is open source and distributed under the General Public License (GPL), making it a standard inclusion in almost every Linux distribution.

As a free, open source, and robust database system, MySQL has become the preferred choice for numerous web applications, including prominent platforms like WordPress, Facebook, LinkedIn, Twitter, Kayak, Walmart.com, Wikipedia, and YouTube. Notably, other popular content management systems (CMSs) like Joomla, Drupal, and Ruby on Rails also rely on MySQL.

To those with aspirations of developing or probing the backend databases of web applications, MySQL is an essential tool to master. Let's embark on this journey.

Initiating MySQL

Thankfully, Kali Linux already comes equipped with MySQL (if you're using a different distribution, you can obtain and install MySQL from the software repository or directly from <https://www.mysql.com/downloads/>).

To initiate the MySQL service, enter the following command in your terminal:

```
kali > service mysql start
```

Next, you need to authenticate yourself by logging in. Execute the following command, and when prompted for a password, simply press ENTER:

```
kali > mysql -u root -p
Enter password:
Welcome to MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.6.30-1 (Debian)
Copyright (c) 2000, 2016, Oracle and/or its affiliates. All rights reserved
```

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement
mysql >

In the default MySQL configuration, the root user's password is empty, which poses a significant security risk. To ensure security, it's imperative to set a password after your initial login. It's important to note that the usernames and passwords for your operating system and MySQL are distinct. Let's take a proactive step by changing the password for the MySQL root user now.

MySQL History

MySQL was initially developed by MySQL AB of Sweden in 1995 and subsequently acquired by Sun Microsystems in 2008. Eventually, Oracle took ownership in 2009, making MySQL part of Oracle's portfolio. Given Oracle's status as the world's largest database software publisher, concerns arose in the open source community regarding Oracle's commitment to keeping MySQL open source.

As a Linux administrator or hacker, it's prudent to keep a close watch on the development of "Maria," a fork of the MySQL database software. Maria is dedicated to maintaining the software and its future versions as open source, providing an alternative path for those concerned about the direction of MySQL under Oracle's stewardship.

Interacting with MySQL Databases

SQL is a dynamic programming language designed for interacting with databases, often of the relational kind where data is organized into tables with rows and columns that interact. Various SQL implementations come with their own set of commands and syntax, but here are some commonly used SQL commands:

- `select`: Used to retrieve data.
- `union`: Combines results from multiple `select` operations.
- `insert`: Adds new data.
- `update`: Modifies existing data.

- `delete`: Removes data.

Each of these commands can be tailored with specific conditions to precisely define their actions. For instance, the following line of code:

```
select user, password from customers where user='admin';
```

This code retrieves the values for the "user" and "password" fields for any user with a "user" value equal to "admin" in the "customers" table.

Securing MySQL with a Password

Let's check which users already exist in our MySQL system by using the following command (note that MySQL commands are concluded with a semicolon):

```
mysql >select user, host, password from mysql.user;
```

user	host	password
------	------	----------

+-----		
--------	--	--

root	localhost	
------	-----------	--

root	aphrodite.kali.org	
------	--------------------	--

root	128.0.246.0	
------	-------------	--

The result shows that the root users currently have no password configured. To set a password for the root user, we'll first select a database to work with. MySQL comes with some preconfigured databases. You can list them using the following command:

```
mysql >show databases;
```

```
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+

3 rows in set (0.23 sec)
```

Out of these databases, we'll work with the non-administrative database named "mysql."
To switch to this database, enter:

```
mysql >use mysql;
```

This command connects us to the "mysql" database. Now, we can set the password for the root user using the following command:

```
mysql >update user set password = PASSWORD("SecurityAiLab") where user = 'root';
```

This command updates the user by assigning the password "SecurityAiLab" to the root user.

Remote Database Access Techniques

To access a MySQL database locally on the same machine, you can use the following syntax:

```
kali >mysql -u <username> -p
```

This command defaults to using the MySQL instance on the local machine if no hostname or IP address is provided. However, to access a remote database, you need to specify the hostname or IP address of the system hosting the MySQL database. Here's an example:

```
kali >mysql -u root -p 157.147.165.59
```

This command connects to the MySQL instance at IP address 157.147.165.59 and prompts for a password. Keep in mind that I'm demonstrating this by connecting to a MySQL instance on my local area network (LAN). If you have a system on your network with MySQL installed, use its IP address. By default, the MySQL database has no password.

Upon successful connection, you'll be presented with the MySQL command line interface, indicated by the `mysql >` prompt. While MySQL offers GUI interfaces like MySQL Workbench, Navicat, and TOAD for MySQL, for hacking purposes, the command line interface is often the preferred choice. Unauthorized users are unlikely to be granted access to user-friendly GUIs.

Note: All MySQL commands must conclude with a semicolon or `\g`, unlike Microsoft's SQL Server. You can access help by entering `help;` or `\h`.

Now that we're logged in as the system administrator, we have unrestricted access to the database. Had we logged in as a regular user, our access would have been constrained by the permissions granted by the system administrator for that user.

Connecting to a Database

With access to the system, our next step is to explore the available databases. To discover which databases are on the accessed system, use the following command:

```
mysql >show databases;
```

```
| Database |
+-----+
| information schema |
| mysql |
| creditcardnumbers |
| performance_schema |
+-----+

4 rows in set (0.12 sec)
```

This command provides a list of databases, and you'll likely encounter a database of interest. In this example, we've identified a database named "creditcardnumbers," which seems worth exploring.

To connect to a specific database, use the command `use databasename;`. For instance:

```
mysql > use creditcardnumbers;
```

The "Database changed" response indicates that we are now connected to the "creditcardnumbers" database. Keep in mind that database administrators typically do not give databases easily recognizable names like "creditcardnumbers," so you might need to explore further to find a database of interest.

Understanding Database Structures

We are now connected to the creditcardnumbers database and can begin exploring its contents. In a database, data is organized into tables, each potentially holding different sets of related information. To discover what tables are present in this database, execute the following command:

```
mysql > show tables;
```

```
+-----+
| Tables_in_creditcardnumbers |
+-----+
| cardnumbers |
+-----+

1 row in set (0.7 sec)
```

The result will show the available tables in the database, and in this case, we have just one table called "cardnumbers."

Usually, databases contain multiple tables, requiring a more thorough investigation. However, in this instance, we are fortunate to focus solely on the "cardnumbers" table, which holds the hacker's potential treasure trove.

Now that we've identified the table we want to examine, let's delve into understanding its structure. This knowledge will enable us to extract the relevant information.

You can obtain the structure of the table by using the "describe" statement, like so:

```
mysql > describe cardnumbers;
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| customers  | varchar(15)   | YES  |     | NULL    |            |
| address    | varchar(15)   | YES  |     | NULL    |            |
| city       | varchar(15)   | YES  |     | NULL    |            |
| state      | varchar(15)   | YES  |     | NULL    |            |
| cc         | int(12)       | NO   |     | 0       |            |
+-----+-----+-----+-----+-----+-----+
```

The result provides vital insights into the table's structure. It lists the field names, data types (commonly varchar for text and int for integers), whether null values are accepted, the existence of keys (which link tables), default values, and any additional notes.

Data Analysis in MySQL

To view the actual data within the table, we utilize the SELECT command. To execute a SELECT command, you need to specify:

The table containing the data you wish to retrieve.

The columns within that table from which you want to retrieve data.

The SELECT command is structured as follows:

```
SELECT columns FROM table
```

As a convenient shortcut to view data from all columns, you can use an asterisk (*) as a wildcard instead of listing each column individually. To display all data from the "cardnumbers" table, execute the following:

```
mysql > SELECT * FROM cardnumbers;
```

```
+-----+-----+-----+-----+-----+
| Customers | Address          | City          | State | CC Number |
+-----+-----+-----+-----+-----+
| Smith     | 100 River Rd     | Chicago       | IL    | 9876543210 |
| Johnson   | 45 Oxford Street | Manchester    | UK    | 8765432190 |
| Williams  | 789 Hill Blvd    | San Francisco | CA    | 7654321987 |
+-----+-----+-----+-----+-----+
```

MySQL will display all the information from the "cardnumbers" table on your screen.

Using PostgreSQL with Metasploit

PostgreSQL, often referred to as Postgres, is another open-source relational database widely used for large-scale, internet-facing applications due to its scalability and ability to handle heavy workloads. It was first released in July 1996 and is maintained by a dedicated group of developers known as the PostgreSQL Global Development Group.

PostgreSQL is pre-installed in Kali, but if you're using a different Linux distribution, you can likely install it from your repository using the following command:

```
kali > apt-get postgres install
```

As a hacker, PostgreSQL is of particular significance because it serves as the default database for Metasploit, one of the most widely used penetration testing and hacking frameworks. Metasploit relies on PostgreSQL to store modules, scan results, and exploit data, making it a crucial tool for penetration tests and hacks.

To start PostgreSQL, as with most Linux services, you can use the command:

```
kali > service postgresql start
```

Once PostgreSQL is up and running, you can initiate Metasploit by entering:

```
kali > msfconsole
```

Upon startup, Metasploit will provide you with the "msf >" prompt.

Teaching the intricacies of Metasploit for hacking and exploitation is beyond the scope of this book. However, we will configure the database that Metasploit will use to store its information.

With Metasploit running, you can set up PostgreSQL using the following command to ensure it stores data from Metasploit activity on your system:

```
msf > msfdb init

[*] exec :msfdb init

Creating database use 'msf'

Enter password for new role

Enter it again:

Creating databases 'msf' and 'msf_test'

Creating configuration file usrshare/metasploit-framework/config/database.yml

Creating initial database schema
```

This command creates the necessary database structure for Metasploit, along with a configuration file.

To proceed, you'll need to log in to PostgreSQL as the root user. You can use the "su" command to switch to the PostgreSQL user with root privileges:

```
msf > su postgres
```

Once logged in, you can create a new user and set a password:

```
postgres@kali:/root$ createuser msf_user -P
```

Specify your desired password when prompted.

Next, create a database and grant permissions to the "msf_user." Name the database "security_ai_lab_db":

```
postgres@kali:/root$ createdb --owner=msf_user security_ai_lab_db
```

Exit from PostgreSQL using the "exit" command to return to the Metasploit prompt:

```
postgres@kali:/root$ exit
```

Back in Metasploit, you can connect the Metasploit console ("msfconsole") to the PostgreSQL database by providing the user, password, host, and database name:

```
msf > db_connect msf_user:password@192.168.0.1/security_ai_lab_db
```

Ensure you enter the password you previously set. The IP address used here is typically "localhost" (192.168.0.1) unless you configured the database on a remote system.

Finally, you can verify the PostgreSQL database's status to confirm the connection:

```
msf > db_status
```

Metasploit will confirm that PostgreSQL is connected and ready for use. Now, when conducting system scans or running exploits with Metasploit, the results will be stored in the PostgreSQL database. Additionally, Metasploit modules are stored in the Postgres database, simplifying module searches and enhancing performance.

Chapter Summary

Linux boasts a multitude of background services, each available for use when needed. While the Apache Web Server is the most prominent, hackers should also familiarize themselves with MySQL, SSH, and PostgreSQL for various tasks. In this chapter, we've covered the basics of getting started with these services. As you become more proficient with your Linux system, I encourage you to explore each service further.

Practice Exercises

Before proceeding to Chapter 13, I recommend practicing the skills you've acquired in this chapter with the following exercises:

- Start your apache2 service through the command line.

- Create a simple website using the index.html file to announce your entry into the world of hacking.

- Initiate your SSH service via the command line and connect to your Kali system from another system on your LAN.

- Start your MySQL database service and change the root user password to "SecurityAiLab." Switch to the "mysql" database.

- Start your PostgreSQL database service and configure it as described in this chapter for use with Metasploit.

13. ENHANCING SECURITY AND ANONYMITY

In today's digital landscape, virtually every action we take on the internet leaves a trace. Whether it's Google tracking our online searches, website visits, and emails, or the National Security Agency (NSA) compiling our digital activities, our online movements are meticulously recorded, cataloged, and analyzed for various purposes. Both the average individual and the more tech-savvy hacker must grasp the importance of limiting this surveillance and preserving a semblance of online anonymity.

This chapter explores strategies to navigate the World Wide Web with a degree of anonymity, recognizing that no single method guarantees absolute privacy. However, implementing these methods can significantly impede trackers' efforts.

Internet Traces and Privacy Concerns

To begin, let's delve into the high-level overview of how our online activities are tracked. While we won't delve into every tracking method or elaborate extensively on any single one, it's crucial to understand the fundamental concepts.

First and foremost, your IP address serves as your unique identifier as you traverse the internet. Data transmitted from your device is typically tagged with your IP address, making your online actions easily traceable. Secondly, services like Google and other email providers analyze your email content to identify keywords, facilitating more targeted advertising. While there are more sophisticated tracking methods that demand substantial time and resources, our focus in this chapter is on mitigating these primary forms of tracking.

Let's begin by examining how IP addresses expose us on the internet. When you send a data packet across the internet, it carries the IP addresses of both the source and destination. This information guides the packet to its intended target and back. Each packet traverses multiple internet routers until it reaches its destination and then returns to the sender. For general internet usage, each hop represents a router that the packet passes through. There may be as many as 20 to 30 hops between the sender and destination, but in most cases, packets reach their destination in fewer than 15 hops.

During its journey across the internet, anyone intercepting the packet can discern its sender, route history, and ultimate destination. This is how websites identify you upon arrival and automatically log you in. It's also how someone can track your online activities.

To visualize the route that a packet might take from your location to its destination, you can use the "traceroute" command, as demonstrated below. Simply input "traceroute" followed by the destination IP address or domain, and the command will send packets to the destination and trace their route.

```
kali > traceroute google.com
```

```
traceroute to google.com (142.250.180.78), 30 hops max, 60 bytes packets
```

```
1 192.168.0.1 (192.168.0.1) 3.456 ms 2.897 ms 2.764 ms
```

```
2 10.1.10.1 (10.1.10.1) 4.321 ms 5.678 ms 5.896 ms
```

```
3 73.158.65.101 (73.158.65.101) 10.123 ms 12.456 ms 13.789 ms
```

```
18 sea30s01-in-f14.1e100.net (142.250.180.78) 87.654 ms 45.321 ms 43.210 ms
```

As you can observe, the number of hops to reach www.google.com can vary. The results depend on your location and the distribution of Google's servers worldwide. Additionally, packets don't consistently follow the same route across the internet, so subsequent packets to the same site may take different paths. Now, let's explore how we can obscure this process using the Tor network.

Introduction to Tor and Its Mechanism

In the 1990s, the US Office of Naval Research (ONR) initiated a project to enable anonymous internet navigation, primarily for espionage purposes. The objective was to establish a separate network of routers, distinct from the internet's routers, capable of encrypting traffic. This network would store only the unencrypted IP address of the previous router, while encrypting all other router addresses along the way. The idea was to render it impossible for anyone monitoring the traffic to determine the data's origin or destination. This research effort later evolved into the "The Onion Router (Tor) Project" in 2002, and it's now accessible for anyone seeking a relatively secure and anonymous online experience.

Operational Dynamics of Tor

When packets are transmitted via Tor, they bypass the conventional routers closely monitored by various entities. Instead, they traverse a network comprising over 7,000 routers worldwide, thanks to volunteers who permit their computers to participate in the Tor network. Additionally, Tor encrypts data, destination, and sender IP addresses for each packet. At every hop, this information is encrypted and then decrypted by the succeeding hop upon reception. Consequently, each packet retains information solely about the previous hop in the path and conceals the IP address of the source. In case of traffic interception, observers can only identify the IP address of the preceding hop, while website owners can discern solely the IP address of the last router that forwarded the traffic (refer to Image 13-1). This mechanism ensures a degree of anonymity while navigating the internet.

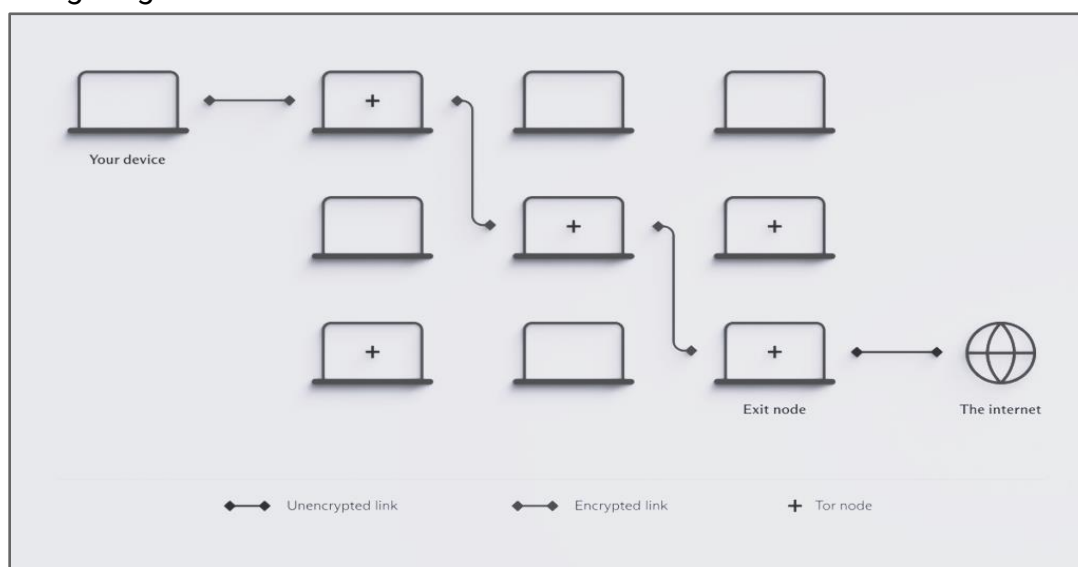


Image 13-1: Overview of the Tor network

To enable Tor, you can install the Tor browser from <https://www.torproject.org/>. Once installed, the browser interface will resemble the one depicted in Image 13-2. You can use this browser just like any standard web browser. By doing so, you'll be accessing the internet through a distinct set of routers, thereby evading tracking by surveillance entities. However, be prepared for slower browsing speeds due to the limited bandwidth in the Tor network, which has fewer routers compared to the conventional internet.



Image 13-2: Tor browser Homepage

Moreover, apart from accessing nearly any website on the regular internet, the Tor browser grants access to the dark web. Websites on the dark web prioritize anonymity and permit access exclusively through the Tor browser, bearing addresses ending in ".onion" for their top-level domain (TLD). While the dark web is infamous for illicit activities, it also hosts legitimate services. Caution is advised when exploring this realm, as you may encounter content that many find offensive.

Tor-related Security Aspects

The Tor network has garnered the attention of intelligence and spy services, not just in the United States but globally. They perceive Tor as a potential threat to national security, viewing it as a tool that allows foreign governments and terrorists to communicate without surveillance. Consequently, various robust research projects have been initiated with the aim of compromising Tor's anonymity.

It's important to understand that Tor's anonymity has been breached in the past by these authorities and is likely to face such attempts again. For instance, the NSA operates its own Tor routers, meaning that when you use Tor, your traffic might pass through NSA-controlled routers. If your traffic exits through these routers, it's even riskier, as the exit router always knows your destination. The NSA employs techniques such as traffic correlation, which involves identifying patterns in incoming and outgoing traffic, and has been successful in breaking Tor's anonymity. While these efforts to compromise Tor may not significantly affect its ability to conceal your identity from commercial services like Google, they can limit its effectiveness in protecting your anonymity from government spy agencies.

Utilizing Proxy Servers

Another strategy for achieving online anonymity involves using proxy servers. These servers act as intermediaries for internet traffic: the user connects to a proxy, and the traffic is routed through the proxy's IP address before reaching its intended destination (refer to Image 13-3). Upon returning from the destination, the proxy forwards the traffic back to the source. Consequently, the traffic appears to originate from the proxy rather than the user's actual IP address.

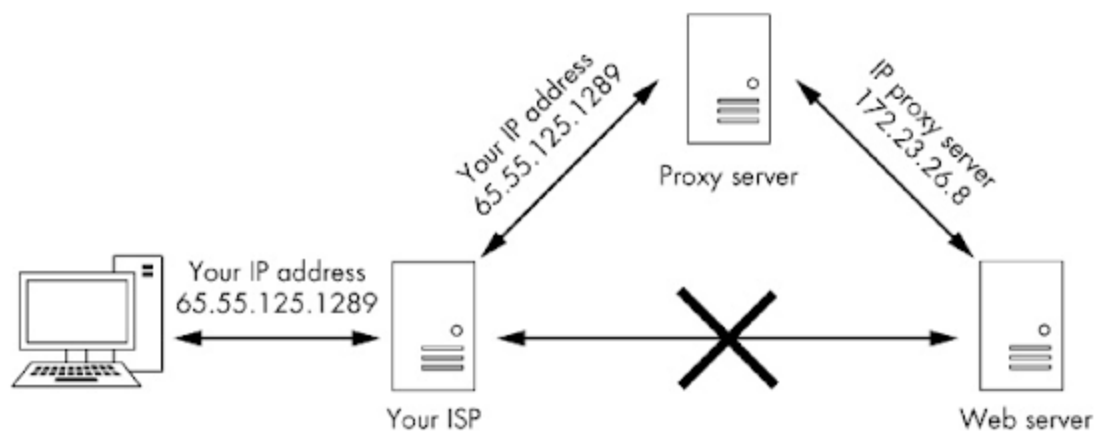


Image 13-3: Running traffic on a Proxy Server

Of course, it's worth noting that the proxy server may log your traffic. However, investigators typically require a subpoena or search warrant to access these logs. To further enhance the difficulty of tracing your traffic, you can employ multiple proxies in what's known as a proxy chain, a concept we'll explore later in this chapter.

Kali Linux offers a powerful proxying tool called "proxychains" that simplifies the process of obscuring your traffic. The syntax for the proxychains command is straightforward, as demonstrated below:

```
kali > proxychains <the command you want to proxy> <arguments>
```

For example, if you want to use proxychains to anonymously scan a website with nmap, you would enter the following command:

```
kali > proxychains nmap -sT -Pn <IP address>
```

This command routes the nmap command through a proxy to the specified IP address. Proxychains automatically constructs the proxy chain, eliminating the need for manual configuration.

Configuring Proxy Settings

In this section, we'll configure the proxychains command to use proxies of our choice. Like many applications in the Linux/Unix ecosystem, proxychains configuration is managed through a configuration file, specifically "etcproxychains.conf." You can open this config file in your preferred text editor with the following command:

```
kali > leafpad etcproxychains.conf
```

```
# proxychains.conf VER 3.1

# HTTP, SOCKS4, SOCKS5 tunneling proxifier with DNS.

# The option below identifies how the ProxyList is treated.

# only one option should be uncommented at time,

# otherwise the last appearing option will be accepted

#

# dynamic_chain

#

# Dynamic - Each connection will be done via chained proxies

# all proxies chained in the order as they appear in the list

# at least one proxy must be online to play in chain
```

```
# (dead proxies are skipped)

# otherwise EINTR is returned to the app strict chain

# Strict - Each connection will be done via chained proxies

# all proxies chained in the order as they appear in the list

# all proxies must be online to play in chain

# otherwise EINTR is returned to the app M
```

Scrolling down to line 61, you'll find the "ProxyList" section, as shown in Code 13-2. This section allows you to add the IP addresses and ports of the proxies you intend to use.

We can incorporate proxies by providing the IP addresses and ports in this list. For now, we'll use some freely available proxies, which can be found through online searches or websites like <http://www.proxysite.com>. However, it's essential to note that using free proxies for real hacking activities is not advisable, as we'll discuss in more detail later in this chapter. The example below is solely for educational purposes:

LAST CHECKED	IP ADDRESS	PORT	LOCATION	SPEED	PROTOCOL	ANONYMITY
12 second ago	111.251.33.169	3128	 Taipei, Taiwan	★★★★☆	HTTP(S)	Elite
12 second ago	78.160.160.209	80	 Istanbul, Turkey	★☆☆☆☆	HTTP(S)	Elite
12 second ago	223.252.33.201	33925	 Brisbane, Australia	★★★★☆	HTTP(S)	Elite
13 second ago	92.241.113.77	8080	 Smolensk, Russia	★★★★☆	HTTP(S)	Elite
13 second ago	171.244.80.17	65301	 Hanoi, Vietnam	★★★★☆	HTTP(S)	Elite
13 second ago	194.110.77.188	8080	 Poland	★★★★☆	HTTP(S)	Elite
14 second ago	58.153.108.102	8080	 Central District, Hong Kong	★★★★☆	HTTP(S)	Elite
15 second ago	41.151.67.133	8080	 Cape Town, South Africa	★★★★☆	HTTP(S)	Elite
15 second ago	51.143.97.158	80	 United Kingdom	★★★★☆	HTTP(S)	Elite
15 second ago	202.166.165.58	8080	 Pakistan	★☆☆☆☆	HTTP(S)	Elite
15 second ago	138.68.145.201	8118	 London, United Kingdom	★☆☆☆☆	HTTP(S)	Elite
15 second ago	202.92.201.42	8080	 Jakarta, Indonesia	★☆☆☆☆	HTTP(S)	Elite
16 second ago	113.65.21.132	9292	 Guanzhou, China	★★★★☆	HTTP(S)	Elite

Example:

[ProxyList]

add proxy here...

socks4 114.134.186.12 22020

meanwhile

defaults set to "tor"

socks4 192.168.0.1 8750

Code 13-2: Adding proxy file section in config

By default, proxychains is configured to use Tor if you haven't added your own proxies. The last line in the configuration directs proxychains to route traffic through the host at 192.168.0.1 on port 8750, which is the default Tor configuration. If you're not adding your own proxies and wish to use Tor, leave this line as is. If you opt not to use Tor, you'll need to comment out this line by adding a "#" symbol before it.

While Tor remains a valuable tool, it's important to acknowledge that it is often slow. Additionally, given the NSA's previous successes in compromising Tor's anonymity, some individuals may choose not to rely on it entirely for anonymity. Consequently, they may comment out the Tor-related line in the configuration and introduce their own set of proxies.

Let's put this into practice. In the following example, I will use the proxychains command to open the Firefox browser and anonymously access <https://www.securityailab.com/>, routing the traffic through a proxy.

The command is this:

```
kali > proxychains firefox www.securityailab.com
```

This command successfully opens <https://www.securityailab.com/> in Firefox via the selected proxy, concealing my IP address from anyone tracing this traffic.

Exploring Additional Proxy Options

Now that we have proxychains up and running, let's explore additional configurations available through the proxychains.conf file. Currently, we are using a single proxy, but we have the flexibility to incorporate multiple proxies. We can employ all of them, select a limited number from the list, or have proxychains randomize their order. Let's delve into these options.

Adding More Proxies

First, let's expand our list of proxies. Return to <https://www.proxysite.com/> and discover more proxy IP addresses. Add a few of these additional proxies to your proxychains.conf file, as shown below:

```
[ProxyList]
# add proxy here...
socks4 114.134.186.12 22020
socks4 188.187.190.59 8888
socks4 181.113.121.158 335551
```

Save this configuration file and execute the following command:

```
kali > proxychains firefox www.securityailab.com
```

Although you won't notice any immediate difference, your data packets are now traversing through several proxies.

Dynamic Chaining

With multiple IP addresses in our proxychain.conf file, we can establish dynamic chaining. This method routes our traffic through every proxy on the list and seamlessly switches to the next proxy if one becomes unavailable or unresponsive. Without this setup, a single malfunctioning proxy could disrupt our requests.

Access your proxychains configuration file again and locate the dynamic_chain line (line 10). Uncomment it, as demonstrated below. Ensure that you've also commented out the strict_chain line if it's not already done.

```
# dynamic_chain
#
```



```
# Dynamic - Each connection will be done via chained proxies
# all proxies chained in the order as they appear in the list
# at least one proxy must be online to play in chain
```

By enabling dynamic chaining of our proxies, we enhance anonymity and ensure smoother hacking. Save the configuration file and feel free to test it.

Random Chaining

Our final proxy technique is random chaining, where proxychains randomly selects a subset of IP addresses from our list to create our proxy chain. This results in a different proxy combination each time we use proxychains, making it more challenging to trace our traffic back to its source. This option falls under the "dynamic" category because it seamlessly transitions to the next proxy if one fails.

Open the etcproxychains.conf file once more and comment out the lines dynamic_chain and strict_chain by adding a "#" at the beginning of each line. Then uncomment the random_chain line. Note that we can only use one of these three options at a time, so ensure that you've commented out the other options before employing proxychains.

Next, locate and uncomment the line with chain_len and assign it a reasonable number. This line determines how many IP addresses from your list will be used to construct your random proxy chain.

```
# dynamic_chain
#
# Dynamic - Each connection will be done via chained proxies
# all proxies chained in the order as they appear in the list
# at least one proxy must be online to play in chain
#
# strict_chain
#
# Strict - Each connection will be done via chained proxies
# all proxies chained in the order as they appear in the list
# all proxies must be online to play in chain
# otherwise EINTR is returned to the app
#
```

```
random_chain
# Random - Each connection will be done via random proxy
# (or proxy chain, see chain_len) from the list.
# this option is good to test your IDS :)
chain_len = 3
```

In this example, I've uncommented chain_len and set it to 3, indicating that proxychains will utilize three proxies from my list in the etcproxychains.conf file. These proxies will be selected randomly, and if any of them are unavailable, proxychains will seamlessly switch to the next. Keep in mind that while this method enhances anonymity, it may introduce higher latency in your online activities.

Now that you have mastered proxychains, you can engage in hacking activities with a reasonable level of anonymity. However, it's essential to acknowledge that achieving absolute anonymity in the face of surveillance agencies like the NSA and FSB is challenging. Nonetheless, with the aid of proxychains, we can significantly complicate their efforts at tracking us.

Proxy Security Considerations

As a final note on proxy security, exercise caution when selecting your proxies. The effectiveness of proxychains depends on the quality of the proxies you use. If your goal is to maintain anonymity, avoid free proxies, as mentioned earlier. Professional hackers opt for paid proxies that can be trusted. In fact, free proxies often compromise your IP address and browsing history for their own gain. As the renowned cryptographer and security expert Bruce Schneier once aptly put it, "If something is free, you're not the customer; you're the product." In other words, free products typically gather your data and monetize it. Why else would they offer a proxy service for free?

While your traffic leaving the proxy will be anonymized in terms of IP address, other methods exist for surveillance agencies to identify you. For instance, the proxy owner may possess your identity and, under pressure from espionage or law enforcement agencies with jurisdiction, might disclose your information to safeguard their interests. It's crucial to understand the limitations of proxies as a means of achieving anonymity.

Implementing Virtual Private Networks

Utilizing a virtual private network (VPN) can be an effective strategy to maintain a reasonable level of anonymity and security for your web traffic. A VPN serves as a bridge to connect you to an intermediary internet device, typically a router. This device then forwards your traffic to its final destination, attributing it with the router's IP address.

While a VPN can indeed enhance your online security and privacy, it's essential to note that it doesn't guarantee complete anonymity. To ensure data transmission, the internet device you connect to must record or log your IP address. This information could potentially be accessed by individuals or organizations, revealing details about your online activities.

The beauty of VPNs lies in their simplicity and ease of use. You can easily set up an account with a VPN provider and then seamlessly connect to the VPN each time you log in to your computer. You can continue to use your browser as you normally would while surfing the web, but to observers, it will appear as if your traffic is originating from the IP address and location of the internet VPN device, rather than your own. Furthermore, all data exchanged between you and the VPN device is encrypted, ensuring that even your internet service provider cannot monitor your traffic.

Beyond privacy, VPNs can be effective in circumventing government-imposed content restrictions and information censorship. For instance, if your national government restricts access to websites with specific political content, you can often use a VPN located outside your country to access such content. Additionally, some media corporations, including Netflix, Hulu, and HBO, limit content access based on the originating IP address's nation. Utilizing a VPN based in a region supported by these services can often bypass these restrictions.

According to CNET, some of the best and most popular commercial VPN services include:

- IPVanish
- NordVPN
- ExpressVPN
- CyberGhost
- Golden Frog VPN
- Hide My Ass (HMA)

- Private Internet Access
- PureVPN
- TorGuard
- Buffered VPN

Most of these VPN services typically charge an annual fee ranging from \$50 to \$100, and many offer a free 30-day trial. To learn more about setting up a VPN, choose one from the list and visit the provider's website, where you'll find download, installation, and usage instructions that are relatively straightforward.

The key strength of a VPN is that all data leaving your computer is encrypted, safeguarding you against eavesdropping. Furthermore, when you visit a website, your IP address is masked by the VPN's IP address. However, similar to a proxy server, the VPN provider possesses information about your originating IP address (otherwise, they wouldn't be able to route your traffic back to you). In some cases, VPN providers might yield this information under pressure from espionage agencies or law enforcement. To mitigate this risk, consider using VPN services that commit to not storing or logging any user data, assuming they uphold their privacy claims. This approach ensures that, in case someone demands user data from the VPN service provider, no data will be available.

The Concept of Encrypted Email

Popular free email services such as Gmail, Yahoo!, and Outlook Web Mail (formerly Hotmail) come at no cost for a reason: they serve as platforms for tracking your interests and delivering targeted advertisements. As previously mentioned, when a service is free, you are often the product, not the customer. Additionally, the servers of these email providers, like Google, have access to the unencrypted content of your emails, even if you use HTTPS.

One effective measure to prevent email eavesdropping is to adopt encrypted email services. ProtonMail, as shown in Image 13-5, offers end-to-end encryption, meaning that your emails are encrypted not only in transit but also on ProtonMail's servers—so even ProtonMail administrators cannot access your email.

ProtonMail was established by a group of young scientists at the CERN supercollider facility in Switzerland, a country with a longstanding tradition of safeguarding secrets (think of those Swiss bank accounts). ProtonMail's servers are located in the European Union, which has more stringent regulations regarding personal data sharing compared

to the United States. ProtonMail provides a basic account for free and offers premium accounts for a nominal fee. It's worth noting that when exchanging emails with non-ProtonMail users, the encryption level may vary. Refer to the ProtonMail support knowledge base for comprehensive details.

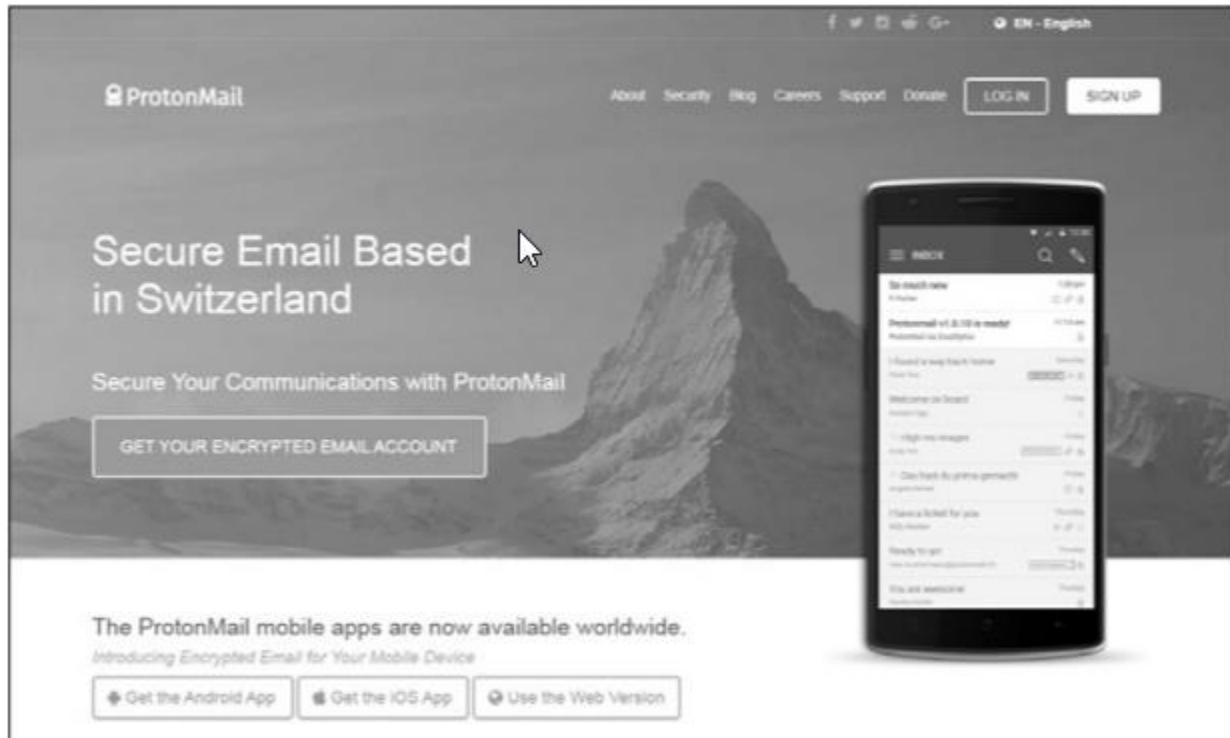


Image 13-5: The ProtonMail

Chapter Summary

In an age of constant surveillance by commercial entities and national intelligence agencies, safeguarding your data and online activities is paramount. Implementing one or more of the security measures discussed in this chapter can significantly reduce your digital footprint and enhance the security of your data.

Practice Exercises

Before moving on to Chapter 14, you can test the skills you've acquired in this chapter by completing the following exercises:

Execute the "traceroute" command to your favorite website. How many hops are displayed between you and the website?

Download and install the Tor browser. Browse the web anonymously with it, similar to any other browser, and observe if there are any noticeable differences in speed.

Experiment with proxychains using the Firefox browser to access your favorite website.

Explore commercial VPN services from the vendors listed in this chapter. Select one and test their free trial.

Create a free ProtonMail account and send a secure greeting to `salprotonmail@gmail.com`

14. WIRELESS NETWORK ANALYSIS AND INSPECTION

The ability to scan for and connect to other network devices from your system is crucial for becoming a successful hacker. With wireless technologies such as Wi-Fi (IEEE 802.11) and Bluetooth becoming the standard, the task of finding and controlling Wi-Fi and Bluetooth connections becomes essential. Hacking a wireless connection grants access to a device and its confidential information. The initial step in this journey is to learn how to locate these devices.

In Chapter 3, we delved into basic networking commands in Linux, covering the fundamentals of wireless networking. In this chapter, as promised, we will explore two of the most prevalent wireless technologies in Linux: Wi-Fi and Bluetooth.

Basics of Wi-Fi Networking

We'll commence with Wi-Fi. In this section, I'll guide you on how to discover, examine, and connect to Wi-Fi access points. Before we proceed, let's take a moment to familiarize ourselves with some fundamental Wi-Fi terminology and technologies. This understanding will prove invaluable when interpreting the results of various queries throughout this chapter:

- **AP (access point):** This is the device to which wireless users connect for internet access.
- **ESSID (extended service set identifier):** Essentially, this is the same as the SSID, as discussed in Chapter 3. However, it can be used for multiple APs in a wireless LAN.
- **BSSID (basic service set identifier):** Each AP is uniquely identified by its BSSID, which corresponds to the MAC address of the device.
- **SSID (service set identifier):** This is the human-readable name of the network.
- **Channels:** Wi-Fi operates on one of 14 channels (1–14). Note that in the United States, Wi-Fi is confined to channels 1–11.
- **Power:** The strength of the Wi-Fi signal varies depending on proximity to the AP. A stronger signal facilitates easier connection cracking.
- **Security:** Security protocols on the Wi-Fi APs come in various forms. Originally, Wired Equivalent Privacy (WEP) was used, but it was notoriously insecure. Its successor, Wi-Fi Protected Access (WPA), offered improved security. Currently, WPA2-PSK, a highly secure option using a preshared key (PSK), is the standard for most Wi-Fi APs, excluding enterprise-grade setups.
- **Modes:** Wi-Fi can operate in one of three modes: managed, master, or monitor. These modes will be explained in detail in the following section.
- **Wireless Range:** In the United States, Wi-Fi APs are legally limited to broadcasting signals at an upper limit of 0.5 watts. This translates to a typical range of about 300 feet (100 meters). High-gain antennas have the potential to extend this range significantly, reaching up to 20 miles.
- **Frequency:** Wi-Fi is designed to operate on two primary frequency bands, 2.4GHz and 5GHz. Modern Wi-Fi APs and wireless network cards often support both frequency bands.

Having grasped these essential Wi-Fi concepts, we can now dive deeper into the world of wireless networking, exploring its modes, capabilities, and much more in the following sections.

Essential Wireless Commands

In Chapter 3, you were introduced to the fundamental Linux networking command "ifconfig," which provides a list of all activated network interfaces on your system, including crucial information such as the IP address for each interface. Now, let's revisit the results from running "ifconfig" but this time, we'll focus on wireless connections.

```
kali >ifconfig
eth0Linkencap:EthernethWaddr 00:0c:29:ba:82:0f
inet addr:188.187.190.59 Bcast:188.187.190.59 Mask:577.577.577.0

lo Linkencap:Local Loopback
inet addr:188.187.190.59 Mask:577.0.0.0

wlan0 Link encap:EthernethWaddr 00:c0:ca:3f:ee:02
```

In the output above, the Wi-Fi interface is represented as "wlan0". In Kali Linux, Wi-Fi interfaces are typically designated as "wlanX," where X corresponds to the number of that interface. For instance, the first Wi-Fi adapter is labeled "wlan0," the second "wlan1," and so forth.

If you only wish to view your Wi-Fi interfaces and their statistics, Linux provides a dedicated command similar to "ifconfig" specifically for wireless interfaces. This command is "iwconfig." When executed, it displays only your wireless interfaces and their essential information:

```
kali >iwconfig
lo no wireless extensions
wlan0 IEEE 802.11bg ESSID:off/any
Mode:Managed Access Point:Not-Associated Tx-Power=20 dBm
Retry short limit:7 RTS thr:off Fragment thr:off
Encryption key:off
Power Management:off
eth0 no wireless extensions
```


In this output, we observe only the wireless interfaces (network cards) along with key data about them, including the wireless standard in use, ESSID status, and mode. The mode can have three settings: "managed," indicating readiness to join or connection to an access point (AP); "master," indicating readiness to act as or already functioning as an AP; and "monitor," which we'll discuss in more detail later in this chapter. We can also determine if any clients have associated with the interface and view its transmit power, among other details. In this example, "wlan0" is in a mode ready for connecting to a Wi-Fi network but is not currently connected. We will revisit this command once the wireless interface is connected to a Wi-Fi network.

If you're uncertain about which Wi-Fi AP to connect to, you can use the "iwlist" command to see all the wireless access points that your network card can reach. The syntax for "iwlist" is as follows:

```
iwlist interface action
```

For our purposes, we'll use the "scan" action to discover all the Wi-Fi APs in your vicinity. Keep in mind that with a standard antenna, your range is typically 300–500 feet, but this can be extended with an inexpensive high-gain antenna:

```
kali >iwlist wlan0 scan
wlan0 Scan completed:
Cell 01 - Address:89:AD:44:37:B3:65
Channel:1
Frequency:2.452GHz (Channel 1)
Quality=70/70 Signal level ==-48 dBm
Encryption key:off
ESSID:"SecurityAiLab"
```

The output of this command should include all the Wi-Fi APs within the range of your wireless interface, along with crucial data about each AP, such as its MAC address, operating channel and frequency, signal quality, encryption status, and ESSID. To

perform any kind of Wi-Fi hacking, you will need the MAC address of the target AP (BSSID), the MAC address of a client (another wireless network card), and the channel on which the AP is operating. This information is invaluable.

Another valuable command for managing your Wi-Fi connections is "nmcli" (the network manager command line interface). The network manager is a Linux daemon that provides a high-level interface for network interfaces, including wireless ones. While Linux users are often familiar with its graphical user interface (GUI), it can also be utilized from the command line.

The "nmcli" command allows you to view nearby Wi-Fi APs and their essential data, similar to "iwlist," but it provides additional information. Use it in the format "nmcli dev networktype," where "dev" stands for devices and the type, in this case, is "wifi," as follows:

```
kali >nmcli dev wifi
* SSID MODE CHAN RATE SIGNAL BARS SECURITY
SecurityAiLab Infra 1 85 Mbits/s 91 WPA1 WPA2
Xfinitywifi Infra 1 85 Mbits/s 83 WPA2
TPTV1 Infra 11 85 Mbits/s 46 WPA1 WPA2
```

In addition to displaying the Wi-Fi APs within range and their essential details, including SSID, mode, channel, transfer rate, signal strength, and security protocols, "nmcli" can also be used to connect to APs. The syntax for connecting to an AP is as follows:

```
nmcli dev wifi connect AP-SSID password APpassword
```

For example, based on the results from our previous commands, we know there is an AP with an SSID of "SecurityAiLab" and it uses WPA1 WPA2 security. To connect to this network, we can use the following command:

```
kali >nmcli dev wifi connect SecurityAiLab password 12345678
Device 'wlan0' successfully activated with '394a5bf4-8af4-36f8-49beda6cb530'.
```

Feel free to try this on a network you have access to. After successfully connecting to the wireless AP, run "iwconfig" again to observe the changes:

```
kali >iwconfig
lo no wireless extensions
wlan0 IEEE 802.11bg ESSID:"SecurityAiLab"
Mode:Managed Frequency:2.452GHz Access Point:00:25:9C:97:4F:48
Bit Rate=12 Mbs Tx-Power=20 dBm
Retry short limit:7 RTS thr:off Fragment thr:off
Encryption key:off
Power Management:off
Link Quality=64/70 Signal level=-46 dBm
Rx invalid nwid:0 Rx invalid crypt:0 Rx invalid frag:0
Tx excessive retries:0 Invalid misc:13 Missed beacon:0
eth0 no wireless extensions
```

Notice that "iwconfig" now indicates the ESSID as "SecurityAiLab," and the AP is operating at a frequency of 2.452GHz. In a Wi-Fi network, multiple APs can be part of the same network, so there may be several APs contributing to the "SecurityAiLab" network. The MAC address "00:25:9C:97:4F:48" corresponds to the AP we are connected to. Understanding the security, frequency, ESSID, and AP MAC address is vital for Wi-Fi hacking. With these basic commands covered, we are now prepared to delve into Wi-Fi hacking.

Wi-Fi Recon with aircrack-ng

One of the most popular exploits for novice hackers to attempt is cracking Wi-Fi access points. As mentioned earlier, before launching an attack on a Wi-Fi AP, you must gather crucial information, including the MAC address of the target AP (BSSID), the MAC address of a client, and the channel on which the AP operates.

All of this information and more can be obtained using the tools provided by the aircrack-ng suite. I've mentioned this suite of Wi-Fi hacking tools on several occasions,

and now it's time to put them into action. The aircrack-ng suite is bundled with every version of Kali, eliminating the need for separate downloads or installations.

To make effective use of these tools, you must first switch your wireless network card into monitor mode. In this mode, the network card can intercept all traffic passing through it, similar to promiscuous mode on wired network cards. To enable monitor mode, utilize the airmon-ng command from the aircrack-ng suite. The command syntax is straightforward:

```
airmon-ng start|stop|restart interface
```

For example, to enable monitor mode for your wireless network card (typically designated as wlan0), execute the following command:

```
kali >airmon-ng start wlan0
```

You may encounter a message indicating that certain processes could cause issues, and it may suggest running 'airmon-ng check kill' if airodump-ng, aireplay-ng, or airtun-ng stops functioning after a brief period.

In the output, you will notice that airmon-ng may rename your wireless interface; in my case, it was renamed to "wlan0mon." Take note of this new name because you'll need it in the next step.

Now, let's employ another tool from the aircrack-ng suite to extract essential data from the wireless traffic. The airodump-ng command is used to capture and display key data from broadcasting APs (Access Points) and any clients connected to those APs or in proximity. The command is simple; just provide airdump-ng followed by the interface name obtained from the airmon-ng command earlier.

When you issue this command, your wireless card will capture critical information from nearby APs and their associated clients. The information displayed includes:

- BSSID: The MAC address of the AP or client.
- PWR: Signal strength.
- ENC: Encryption method.
- #Data: Data throughput rate.
- CH: The channel on which the AP is operating.
- ESSID: The name of the AP.

Here's an example of running airodump-ng:

```
kali >airodump-ng wlan0mon
```

```
CH 6 ][ Elapsed: 1 min ][ 2024-02-17 10:00
```

BSSID	PWR	Beacons	#Data, #/s		CH	MB	ENC	CIPHER	AUTH	ESSID
00:11:22:33:44:55 MyHomeNetwork	-40	200	50	0	6	54e	WPA2	CCMP	PSK	
66:77:88:99:AA:BB OpenNetwork	-80	100	0	0	11	54	WEP	WEP		
AA:BB:CC:DD:EE:FF CafeWireless	-60	150	20	0	6	54e	WPA2	CCMP	PSK	

BSSID	STATION	PWR	Rate	Lost	Frames	Probes
00:11:22:33:44:55 MyHomeNetwork	11:22:33:44:55:66	-50	0 - 1	2	12	
AA:BB:CC:DD:EE:FF	22:33:44:55:66:77	-70	0 - 54	30	8	CafeWireless

The output is divided into an upper and lower portion. The upper part provides details about broadcasting APs, including BSSID, signal strength, beacon frames, data throughput, packets, channel, theoretical throughput limit, encryption protocol, cipher, authentication type, and ESSID (commonly known as SSID). In the lower part, information about associated clients is displayed, indicating whether they are connected to any AP.

At this point, you have gathered all the necessary information to crack the AP's Wi-Fi password. The process for cracking involves utilizing the client MAC address, AP MAC address, target channel, and a password list. While the complete process is beyond the scope of this book, I'll provide an overview.

To crack the Wi-Fi password, open three terminals. In the first terminal, execute commands similar to the following, ensuring you replace the client and AP MAC addresses and the channel:

```
airodump-ng -c 10 --bssid 01:01:AA:BB:CC:22 -w SecurityAiLabPSK wlan0mon
```

This command captures all packets passing through the AP on channel 10 using the -c option.

In another terminal, use the aireplay-ng command to deauthenticate anyone connected to the AP, forcing them to reauthenticate. This process allows you to capture the password hash exchanged in the WPA2-PSK four-way handshake:

```
aireplay-ng --deauth 100 -a 01:01:AA:BB:CC:22 -c A0:A3:E2:44:7C:E5 wlan0mon
```

Finally, in the third terminal, utilize a password list (wordlist.dic) to attempt to find the password within the captured hash (SecurityAiLabPSK.cap):

```
aircrack-ng -w wordlist.dic -b 01:01:AA:BB:CC:22 SecurityAiLabPSK.cap
```

Exploring Bluetooth Connectivity

In today's world, Bluetooth is nearly ubiquitous, integrated into various gadgets, mobile devices, and systems. Our computers, smartphones, iPods, tablets, speakers, game controllers, keyboards, and a multitude of other devices all embrace Bluetooth technology. Mastering Bluetooth hacking opens doors to device compromise, control, unauthorized data transfer, and much more.

To effectively exploit this technology, we must first comprehend how it operates. While a comprehensive understanding of Bluetooth is beyond the scope of this book, I will equip you with fundamental knowledge to scan for and connect to Bluetooth devices, laying the groundwork for potential hacking endeavors.

The Functioning of Bluetooth

Bluetooth is a universal protocol designed for low-power, near-field communication. It operates within the frequency range of 2.4–2.485GHz, employing spread spectrum technology and frequency hopping at a rate of 1,600 hops per second (a security measure). This remarkable technology was conceived in 1994 by Ericsson Corp. of Sweden and carries the namesake of the 10th-century Danish king, Harald Bluetooth (it's worth noting that in the 10th century, Sweden and Denmark were a single country).

The Bluetooth specification stipulates a minimum range of 10 meters, although manufacturers have the liberty to implement more extensive ranges in their devices. Some devices boast ranges of up to 100 meters, and with specialized antennas, this range can be extended even further.

The process of connecting two Bluetooth devices is termed "pairing." Practically any two Bluetooth devices can establish a connection, but this can only occur when both devices are in "discoverable mode." A Bluetooth device in discoverable mode transmits crucial information, including its name, class, list of services, and technical details. During the pairing process, a secret or link key is exchanged, which is stored by each device for future pairings.

Each Bluetooth device possesses a unique 48-bit identifier akin to a MAC address, as well as a typically manufacturer-assigned name. These pieces of information prove invaluable when identifying and accessing a device.

Bluetooth Scanning Techniques

Linux boasts an implementation of the Bluetooth protocol stack known as BlueZ. We'll utilize BlueZ for scanning Bluetooth signals. Most Linux distributions, including Kali Linux, include BlueZ by default. If your distribution lacks it, you can typically install it from your repository using the command:

```
kali >apt-get install bluez
```

BlueZ equips us with a set of straightforward tools for managing and scanning Bluetooth devices. Here are some of the essential tools at our disposal:

hciconfig: This tool resembles ifconfig for Bluetooth devices. In Code 14-1, I've employed it to activate the Bluetooth interface and retrieve device specifications.

hcidtool: An inquiry tool that provides device name, device ID, device class, and device clock information, facilitating synchronous operation between devices.

hcidump: This tool allows us to intercept Bluetooth communication, effectively capturing data transmitted over the Bluetooth signal.

The initial step in Bluetooth scanning and reconnaissance is to verify if the Bluetooth adapter on our system is recognized and enabled, allowing us to scan for other devices. The built-in BlueZ tool, hciconfig, accomplishes this task, as demonstrated in Code 14-1.

```
kali >hciconfig
hci0: Type: BR/EDR Bus: USB
BD Address: 10:AE:60:58:F1:37 ACL MTU: 310:10 SCO MTU: 64:8
UP RUNNING PSCAN INQUIRY
RX bytes:131433 acl:45 sco:0 events:10519 errors:0
TX bytes:42881 acl:45 sco:0 commands:5081 errors:0
```


As evident from the output, my Bluetooth adapter is recognized and assigned the MAC address 10:AE:60:58:F1:37. It has been designated as `hci0`. The subsequent step involves confirming that the connection is enabled. This can also be achieved with `hciconfig` by specifying the name and issuing the "up" command:

```
kali >hciconfig hci0 up
```

If executed successfully, this command will yield no output, only a fresh prompt.

With `hci0` up and running, we are now poised to harness its capabilities.

Scanning for Bluetooth Devices with `hcitool`

Now that we've confirmed that our adapter is operational, let's delve into another tool from the BlueZ suite called `hcitool`. This versatile tool is used for scanning and interacting with Bluetooth devices within range.

To initiate a scan for Bluetooth devices in discovery mode, meaning they are actively sending out discover beacons, use the simple `hcitool scan` command, as shown in Code 14-2:

```
kali >hcitool scan
Scanning...
72:6E:46:65:72:66 ANDROID BT
22:C5:96:08:5D:32 SCH-I535
```

In this example, my system detected two devices: "ANDROID BT" and "SCH-I535." Keep in mind that your results may differ based on the Bluetooth devices in your vicinity. For

testing purposes, try putting your phone or another Bluetooth device in discovery mode and observe if it appears in the scan results.

To gather more detailed information about the detected devices, we can utilize the `inq` (inquiry) function:

```
kali >hcitool inq
Inquiring...
24:C6:96:08:5D:33 clock offset:0x4e8b class:0x5a020c
76:6F:46:65:72:67 clock offset:0x21c0 class:0x5a020c
```

This command provides us with valuable information, including the MAC addresses of the devices, their clock offsets, and device classes. The device class indicates the type of Bluetooth device you've discovered. To decode the device class, you can refer to the Bluetooth SIG site at [Bluetooth SIG](#).

It's important to note that `hcitool` is a robust command-line interface to the Bluetooth stack, offering a wide range of capabilities. Code 14-3 displays some of the available commands:

```
kali >hcitool --help
hcitool - HCI Tool ver 4.99
Usage:
hcitool [options] <command> [command parameters]
Options:
--help Display help
-i dev HCI device
Commands
dev Display local devices
inq Inquire remote devices
scan Scan for remote devices
```

```
name Get name from remote devices
```

Numerous Bluetooth-hacking tools you encounter often employ these commands within scripts. You can even craft your own custom tools using these commands in bash scripts.

Scanning for Services with sdptool

The Service Discovery Protocol (SDP) is a Bluetooth protocol used to search for various Bluetooth services. BlueZ offers the `sdptool` tool, which allows us to browse a device for the services it provides. Notably, the target device doesn't need to be in discovery mode to be scanned.

To initiate a service scan, use the following syntax:

```
sdptool browse MACaddress
```

In Code 14-4, I demonstrate the usage of `sdptool` to search for services on one of the previously detected devices:

```
kali >sdptool browse 76:6E:46:63:72:66
Browsing 76:6E:46:63:72:66...
Service RecHandle: 0x10002
Service Class ID List:
""(0x1800)
Protocol Descriptor List:
```

```
"L2CAP" (0x0100)
PSM: 31
"ATT" (0x0007)
uint16: 0x1
uint16: 0x5
```

In this example, `sdptool` successfully retrieved information about all the services supported by the target device. Notably, we identify that this device supports the ATT Protocol (Low Energy Attribute Protocol), providing potential insights into the device's capabilities and possible interaction avenues.

The power of `sdptool` lies in its ability to unveil the services offered by Bluetooth devices, aiding us in understanding and interacting with them effectively.

Seeing Whether the Devices Are Reachable with l2ping

Once we've gathered the MAC addresses of all nearby devices, our next step is to determine if these devices are reachable, regardless of whether they're in discovery mode. This helps us identify active devices within range. To send out a ping to a device, we employ the `l2ping` command with the following syntax:

```
l2ping MACaddress
```

Let's take a look at an example in Code 14-5 where I'm pinging an Android device discovered in Code 14-2.

```
kali >l2ping 76:6E:46:63:72:66 -c 4
Ping: 76:6E:46:63:72:66 from 10:AE:60:58:F1:37 (data size 44)...
44 bytes 76:6E:46:63:72:66 id 0 time 37.57ms
```

```
44 bytes 76:6E:46:63:72:66 id 1 time 27.23ms
44 bytes 76:6E:46:63:72:66 id 2 time 27.59ms
```

Code 14-5: Pinging a Bluetooth device

The output in this example indicates that the device with the MAC address 76:6E:46:63:72:66 is indeed within range and reachable. This information is valuable because knowing whether a device is reachable is a prerequisite before we can even consider hacking it.

Chapter Summary

Wireless devices are at the forefront of connectivity and hacking. Linux equips us with specialized commands for scanning and connecting to Wi-Fi Access Points (APs), laying the foundation for hacking these systems. The aircrack-ng suite, comprising tools like airmon-ng and airodump-ng, allows us to scan and collect crucial information from wireless devices within range. The BlueZ suite complements this with tools like hciconfig and hcitool, essential for scanning and gathering information on Bluetooth devices within reach. It also offers a plethora of other tools waiting to be explored.

Practice Exercises

Before proceeding to Chapter 15, let's put the knowledge gained from this chapter to the test with the following exercises:

Use `ifconfig` to check your network devices. Take note of any wireless extensions.

Run `iwconfig` and make note of any wireless network adapters.

Utilize `iwlist` to identify the Wi-Fi APs within range.

Employ `nmcli` to discover the Wi-Fi APs within range. Compare your experience with `nmcli` to that with `iwlist` and determine which one you find more useful and intuitive.

Connect to your Wi-Fi AP using `nmcli`.

Activate your Bluetooth adapter with `hciconfig` and scan for nearby discoverable Bluetooth devices using `hcitool`.

Verify whether these Bluetooth devices are within reachable distance by conducting `l2ping` tests.

15. HANDLING THE LINUX KERNEL AND KERNEL MODULES

In the realm of operating systems, two fundamental components stand out. The first and foremost is the kernel, serving as the heart of the operating system. It assumes control over every facet of the operating system's operations, from memory management to CPU regulation, even extending to what the user perceives on the screen. The second component, often referred to as "user land," encompasses nearly everything else within the operating system.

The kernel is deliberately designed as a protected or privileged domain, accessible only to root or other privileged accounts. This precaution is by no means arbitrary, as breaching the kernel's defenses would grant nearly unrestricted access to the entire operating system. Consequently, most operating systems confine users and services to user land, where they can access a wide array of resources without assuming control of the entire system.

Nevertheless, access to the kernel bestows the power to modify the operating system's behavior, appearance, and functionality. It also harbors the potential to crash the operating system, rendering it inoperable. In certain circumstances, system administrators must cautiously delve into the kernel for operational and security purposes.

This chapter explores the art of altering the kernel's behavior and introduces the addition of new modules to enhance its functionality. It should be emphasized that tampering with the kernel can grant control over the entire system, a fact well known to

hackers. For instance, in scenarios like man-in-the-middle (MITM) attacks, where a hacker interposes themselves between a client and server, kernel manipulation becomes a necessity. This manipulation allows the attacker to eavesdrop on or modify communications.

First, we'll delve into the structure of the kernel and its modules.

Introduction to Kernel Modules

The kernel serves as the central nervous system of your operating system, orchestrating every aspect, from managing hardware interactions to initiating essential services. Positioned between the user applications and the hardware components such as the CPU, memory, and hard drive, the kernel plays a pivotal role in mediating these interactions.

Linux adopts a monolithic kernel architecture, enabling the integration of kernel modules. Consequently, modules can be dynamically added to and removed from the kernel. Periodically, the kernel necessitates updates, often involving the installation of new device drivers (e.g., video cards, Bluetooth devices, USB devices), filesystem drivers, and system extensions. These drivers must be embedded within the kernel to ensure their full functionality. While some systems demand the complete rebuilding, compilation, and rebooting of the kernel to add a driver, Linux offers the convenience of adding specific modules to the kernel without requiring this extensive process. These modules are aptly named "loadable kernel modules," or LKMs.

LKMs, by their very nature, possess access to the deepest levels of the kernel, rendering them highly susceptible to exploitation by hackers. Rootkits, a category of malware, often infiltrate the kernel through these LKMs. Once a rootkit infiltrates the kernel, the attacker gains complete control over the operating system.

Should a hacker successfully entice a Linux administrator to load a new module into the kernel, they not only seize control of the target system but also manipulate the information reported by the target system regarding processes, ports, services, hard drive space, and virtually any other aspect.

Hence, the act of luring a Linux admin into installing a seemingly innocuous driver, concealing a rootkit, is a potent strategy employed by some of the most insidious rootkits targeting Linux and other operating systems.

Comprehending LKMs is indispensable for both adept Linux administrators and stealthy hackers. In the following sections, we will explore the ways in which the kernel can be manipulated for both benevolent and malevolent purposes.

Verifying Your Kernel Version

The initial step in understanding the kernel is to ascertain the version of the kernel currently running on your system. This can be accomplished through at least two methods. The first involves executing the following command:

```
kali > uname -a
```

Upon execution, the kernel provides information about your OS distribution (in this case, Linux Kali), the kernel build (e.g., 4.6.4), and the architecture for which it was compiled (e.g., x86_64). Additionally, it mentions whether it possesses symmetric multiprocessing (SMP) capabilities, which are relevant for systems with multiple cores or processors. Furthermore, it specifies the kernel's build date, such as July 21, 2016, in this example.

It's worth noting that the output may vary depending on your system's kernel and CPU. This information may prove crucial when installing or loading a kernel driver.

Another method to obtain this information, along with additional useful details, involves using the "cat" command to access the "/proc/version" file, like so:

```
kali > cat /proc/version
```

The output provided by this file echoes the same information as before.

Kernel Adjustments with sysctl

Kernel tuning entails the ability to adjust memory allocations, enable network features, and enhance kernel security against external threats. Modern Linux kernels leverage the "sysctl" command to facilitate these adjustments. It's imperative to note that changes made via sysctl are effective only until the system is rebooted. To ensure permanent changes, one must edit the "etc/sysctl.conf" configuration file directly.

A word of caution is necessary here: exercising caution while employing sysctl is paramount. Without adequate knowledge and experience, it's perilously easy to render your system unbootable and unusable. Therefore, meticulous consideration is essential before committing to permanent changes.

To inspect the contents of sysctl and explore available options, you can use the following command:

```
kali > sysctl -a | less
dev.cdrom.autoclose = 1
dev.cdrom.autoeject = 0
dev.cdrom.check_media = 0
dev.cdrom.debug = 0
```

This command reveals a plethora of parameters that Linux administrators can manipulate to optimize kernel behavior. Among these, certain options may prove particularly useful for a hacker's purposes. One example is enabling packet forwarding.

In the context of a man-in-the-middle (MITM) attack, where the attacker intercepts communication between hosts, the ability to forward packets is invaluable. By scrolling through the sysctl output or filtering for "ipv4" using the command below, you can identify the relevant parameter:

```
kali > sysctl -a | less | grep ipv4
```

The parameter of interest is "net.ipv4.ip_forward," which determines whether the kernel forwards received packets. The default setting is 0, indicating packet forwarding is disabled.

To enable IP forwarding, you can issue the following command:

```
kali > sysctl -w net.ipv4.ip_forward=1
```

It's important to remember that sysctl changes take effect immediately but are not preserved across system reboots. To make these changes permanent, you must edit the "etc/sysctl.conf" configuration file.

```
#etc/sysctl.conf - Configuration file for setting system variables
# See etc/sysctl.d/ for additional system variables.
# See sysctl.conf (5) for information.
#
#kernel.domainname = example.com
# Uncomment the following to stop low-level messages on console.
#kernel.printk = 3 4 1 3
#####3
# Functions previously found in netbase
#
# Uncomment the next two lines to enable Spoof protection (reverse-path
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks.
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1
# Uncomment the next line to enable TCP/IP SYN cookies
#
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1
See http://lwn.net/Articles/277146/
# Uncomment the next line to enable packet forwarding for IPv4
```

```
#net.ipv4.ip_forward=1
```

This file can also be utilized to harden your system by, for instance, disabling ICMP echo requests with the addition of "net.ipv4.icmp_echo_ignore_all=1." However, such changes require careful consideration, as they may impact the system's functionality. After adding such lines to the configuration file, you can apply them using the "sysctl -p" command.

Working with Kernel Modules

Linux offers at least two methods for managing kernel modules. The older approach involves using a set of commands centered around the insmod suite, where "insmod" stands for "insert module" and is designed for module-related tasks. The second method, which we'll explore later in this chapter, uses the "modprobe" command. To begin, we'll utilize the "lsmod" command from the insmod suite to list the installed modules within the kernel:

```
kali > lsmod
```

Module	Size	Used by
nfnetlink_queue	20480	0
nfnetlink_log	201480	0
nfnetlink	16384	2 nfnetlink_log, nfnetlink_queue
bluetooth	516096	0
rfkill	0	2 bluetooth

The output of the "lsmod" command provides a comprehensive list of kernel modules, along with details about their size and any other modules that may depend on them. For instance, the "nfnetlink" module, which facilitates message-based communication between the kernel and user space, occupies 16,384 bytes and is utilized by both the "nfnetlink_log" and "nf_netlink_queue" modules.

Within the insmod suite, we can load or insert a module using "insmod" and remove a module using "rmmod" (short for "remove module"). It's essential to note that these commands may not always account for module dependencies, potentially leading to an unstable or unusable kernel. To mitigate this risk, modern Linux distributions have introduced the "modprobe" command, which automatically manages dependencies during module loading and removal. We'll delve into "modprobe" shortly, but first, let's explore how to acquire more information about our modules.

Gathering Module Info with modinfo

To gain deeper insights into any of the kernel modules, we can employ the "modinfo" command. The syntax is straightforward: "modinfo" followed by the name of the module you wish to investigate. For instance, if you're interested in retrieving fundamental information about the Bluetooth kernel module you encountered when running the "lsmod" command earlier, execute the following:

```
kali > modinfo bluetooth
```

```
filename: libmodules/4.6.0-kali-amd64/kernel/net/bluetooth/bluetooth.ko
```

```
alias: net-pf-31
```

```
license: GPL
```

```
version: 2.21
```

```
description:Bluetooth Core ver 2.21
```

```
author: Marcel Holtman <marcel@holtmann.org>

srcversion: FCFDE98577FEA911A3DAFA9

depends: rfkill, crc16

intree: Y

vermagic: 4.6.0-kali1-amd64 SMP mod_unload modversions

parm: disable_esco: Disable eSCO connection creation (bool)

parm: disable_ertm: Disable enhanced retransmission mode (bool)
```

The output of the "modinfo" command unveils substantial details about the specified kernel module, which are vital for Bluetooth functionality on your system. Among various other details, it includes a list of module dependencies, such as "rfkill" and "crc16." These dependencies are essential modules that must be present for the Bluetooth module to operate correctly.

Module Management with modprobe

Most modern Linux distributions, including Kali Linux, come equipped with the "modprobe" command for managing Loadable Kernel Modules (LKMs). To add a module to your kernel, utilize the "modprobe" command along with the "-a" (add) switch, like this:

```
kali > modprobe -a <module name>
```

For removing a module, employ the "-r" (remove) switch with "modprobe," followed by the module's name:

```
kali > modprobe -r <module to be removed>
```

A significant advantage of using "modprobe" over "insmod" is that "modprobe" comprehends dependencies, options, and installation and removal procedures. It carefully considers all these factors before making changes. As a result, adding and removing kernel modules with "modprobe" is both easier and safer.

Direct Kernel Module Insertion and Removal

Let's put your knowledge to the test by inserting and removing a test module, allowing you to become better acquainted with the process. Imagine that you've just installed a new video card and need to install its drivers. Typically, device drivers are installed directly into the kernel to grant them the necessary access for proper functioning. However, this also makes drivers a potential target for malicious hackers aiming to install rootkits or other surveillance tools.

For demonstration purposes only (do not execute these commands), let's assume we want to add a new video driver named "SecurityAiLabNewVideo" to our kernel. You can accomplish this by entering the following command:

```
kali > modprobe -a SecurityAiLabNewVideo
```

To verify whether the new module loaded correctly, you can run the "dmesg" command, which outputs the kernel's message buffer. Then, filter for "video" and inspect for any alerts indicating potential issues:

```
kali > dmesg | grep video
```

Any kernel messages containing the word "video" will be displayed. If nothing appears, there are no messages related to that keyword.

Subsequently, to remove the same module, you can employ the same command but with the "-r" (remove) switch:

```
kali > modprobe -r SecurityAiLabNewVideo
```

Remember, loadable kernel modules provide convenience to Linux users and administrators, but they also represent a significant security vulnerability—one that professional hackers are well-versed in exploiting. As mentioned earlier, LKMs can serve as the ideal vehicle for infiltrating the kernel with a rootkit, potentially causing havoc.

Chapter Summary

The kernel serves as the core component of the operating system and is highly protected. Any inadvertent addition to the kernel can disrupt the entire operating system and potentially lead to complete control being seized.

Loadable Kernel Modules (LKMs) offer system administrators the ability to inject modules directly into the kernel without the need to rebuild the entire kernel. However, this convenience also presents a security risk. If a hacker manages to persuade a system administrator to add a malicious LKM, they can potentially assume complete control of the system without the administrator's awareness.

Practice Exercises

Before progressing to Chapter 16, solidify your skills from this chapter by completing the following exercises:

- Determine the version of your kernel.
- List the modules currently loaded in your kernel.
- Enable IP forwarding using a sysctl command.

Modify your `/etc/sysctl.conf` file to enable IP forwarding. Then, disable IP forwarding.

Select one kernel module and gather more information about it using the `"modinfo"` command.

16. TASK AUTOMATION WITH JOB SCHEDULING

Like all Linux users, both administrators and hackers often have tasks, scripts, or other operations they want to execute periodically. For instance, you might want to establish a schedule for automated regular system backups or log file rotation, as discussed in Chapter 11. Meanwhile, hackers might wish to run the `MySQLscanner.sh` script from Chapter 8 at specific intervals, such as every night or during their work or school hours. These scenarios all involve scheduling automatic jobs, allowing tasks to run without manual intervention, and optimizing system resource usage during periods of inactivity.

Linux administrators, and hackers alike, may also have a need to configure certain scripts or services to launch automatically when the system boots. In Chapter 12, we explored the use of the PostgreSQL database in conjunction with the Metasploit hacker/pentest framework. Rather than manually starting the PostgreSQL database before launching Metasploit every time, you can set it up, along with any other script or service, to start automatically during system boot.

In this chapter, you'll delve deeper into how to utilize the cron daemon and crontab to automate script execution, even when the system is unattended. You'll also discover how to configure startup scripts that initiate whenever the system boots, ensuring you have the essential services ready for your day of hacking.

Fundamentals of Automated Scheduling

The cron daemon (`crond`) and the cron table (`crontab`) are the Linux administrator's best friends for scheduling routine tasks. The `crond` daemon operates silently in the background, periodically checking the cron table to determine which commands to execute at specified times. To schedule tasks or jobs, you add entries to the cron table file, located at `/etc/crontab`. This table consists of seven fields: the first five schedule the time to execute the task, the sixth specifies the user, and the seventh denotes the

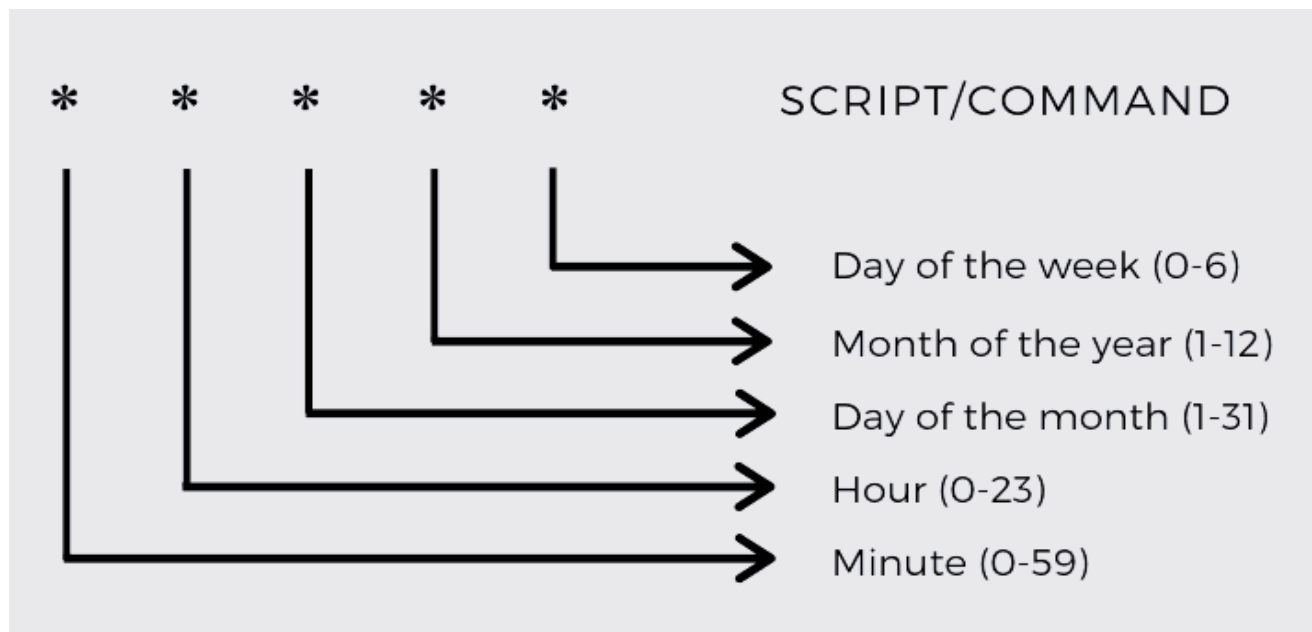
absolute path to the command to execute. For scripts, you can simply provide the absolute path to the script in the seventh field.

Each of the five time fields represents a distinct aspect of time: minute, hour, day of the month, month, and day of the week, in that sequence. All time elements must be represented numerically; for example, March is represented as 3 (not "March"). Days of the week are numbered from 0 (Sunday) to 7 (also Sunday). You can refer to Table 16-1 for a summary of these time representations.

Table 16-1: Time Representations for Use in the crontab

Field	Time Unit	Representation
1	Minute	0–59
2	Hour	0–23
3	Day of the month	1–31
4	Month	1–12
5	Day of the week	0–7

Suppose you've developed a script to scan for vulnerable open ports worldwide and want it to run every night at 2:30 AM from Monday through Friday. In such a case, you can schedule it within the crontab file. We will guide you through the process of adding this information to the crontab shortly, but first, let's discuss the required format, as shown in Code 16-1.



Examples:

min	hour	day/month	month	day/week	Execution time
30	0	1	1,6,12	*	– 00:30 Hrs on 1st of Jan, June & Dec.
0	20	*	10	1-5	–8.00 PM every weekday (Mon-Fri) only in Oct.
0	0	1,10,15	*	*	– midnight on 1st ,10th & 15th of month
5,10	0	10	*	1	– At 12.05,12.10 every Monday & on 10th of every month

Code 16-1: The format for scheduling commands

The crontab file provides clear column labels. The first field designates the minute (30), the second indicates the hour (2), the fifth specifies the days (1-5, representing Monday through Friday), the sixth identifies the user (root), and the seventh contains the script's path. The third and fourth fields are set as asterisks (*) because the script should run every day from Monday through Friday, regardless of the day of the month or the month.

In Code 16-1, the fifth field defines a range for the day of the week using a dash (-) between numbers. If you want to execute a script on multiple noncontiguous days of the week, you can separate those days with commas (.). For example, Tuesday and Thursday would be represented as 2,4.

To edit the crontab, you can use the "crontab -e" command:

```
kali > crontab -e
Select an editor. To change later, run 'select-editor'.
 1. binnano <----easiest
 2. /usrbinmcedit
 3. /usrbinvim.basic
 4. /usrbinvim.gtk
 5. /usrbinvim.tiny
Choose 1-5 [1]:
```

The first time you run this command, it will prompt you to choose an editor. The default is "bin/nano" (option 1), which opens directly to crontab. Alternatively, you can open crontab in your preferred text editor, as shown below:

```
kali > leafpad /etc/crontab
```

This command opens crontab in Leafpad, a text editor. You can see a portion of the file in Code 16-2.

```
# etcrcrontab: system-wide crontab

# Unlike any other crontab, you don't have to run the 'crontab'

# command to install the new version when you edit this file

# and files in etcrcron.d. These files also have username fields,

# which no other crontabs do.

SHELL=binsh

PATH=usrlocal/sbin:usrlocal/bin:/sbin:/bin:usrsbin:usrbin

# m h dom mon dow user  command

17 root cd / && run-parts --report etcrcron.hourly

25 6 * root test -x usr/sbin/anacron || ( cd / && run-parts

47 6 7 root test -x usr/sbin/anacron || ( cd / && run-parts

52 6 1 root test -x usr/sbin/anacron || ( cd / && run-parts

#
```

Code 16-2: The crontab file in use in a text editor

To schedule a new recurring task, add a new line to the file and save it.

Setting Up Automated Backup Tasks

Now, let's explore how to schedule a backup task, which is particularly important for system administrators. As a system administrator, you often need to perform backups of all your files during off-hours when system resources are abundant and not in high

demand. Typically, this means scheduling backups in the middle of the night on the weekend, a time when you probably have other priorities and don't want to be manually initiating backups.

It's important to note that the hour field in the cron format uses a 24-hour clock instead of the AM/PM notation. For instance, 1 PM is represented as 13:00. Additionally, days of the week (DOW) are numbered starting with Sunday (0) and ending with Saturday (6).

To create a scheduled job, you only need to edit the crontab file by adding a line in the specified format. Let's say you want to set up a regular backup job for a user named "backup." You would create a script for system backup, save it as "systembackup.sh" in the "/bin" directory, and then schedule it to run automatically every Saturday night/Sunday morning at 2 AM by adding the following line to the crontab:

```
00 2 0 backup /bin/systembackup.sh
```

The asterisk (*) wildcard is used to indicate "any," and when used in place of a digit for the day of the month, month, or day of the week, it means "all" days or months.

So, breaking down the line above:

- The job runs at the top of the hour (00).
- In the second hour (2).
- On any day of the month (*).
- In any month (*).
- On Sunday (0).
- As the user "backup."
- It executes the script located at "/bin/systembackup.sh."

The cron daemon will execute this script every Sunday morning at 2 AM, every month.

If you want the backup to run only on the 15th and 30th of every month, regardless of the day of the week, you can modify the entry in the crontab as follows:

```
00 2 15,30 backup /bin/systembackup.sh
```

Here, the day of the month (DOM) field specifies 15,30, indicating that the script should run only on the 15th and 30th of every month, approximately every two weeks. When specifying multiple days, hours, or months, you separate them with commas.

Now, let's assume your company requires stringent data backup procedures, and you need to back up data every weeknight to minimize data loss in case of power outages or system crashes. You can achieve this by adding the following line:

```
00 23 1-5 backup /bin/systembackup.sh
```

This job will run at 11 PM (hour 23) every day of the month, every month, but only from Monday through Friday (days 1–5). It's worth noting that you can designate days of the week as an interval (1-5) or as individual days (1,2,3,4,5); both methods work perfectly.

Scheduling with crontab for MySQLscanner

Now that you have a grasp of scheduling jobs with the crontab command, let's schedule the MySQLscanner.sh script, designed to locate open MySQL ports, as discussed in Chapter 8. This scanner identifies systems running MySQL by searching for open port 3306.

To schedule your MySQLscanner.sh in the crontab file, simply edit the file to provide the job details, similar to what we did for system backups. Let's schedule it to run during the day when you're at work, ensuring it doesn't consume resources while you're using your home system. You can enter the following line in your crontab:

```
00 9 * user /usr/share/MySQLsscanner.sh
```

This job is configured to run at the 00th minute, during the ninth hour, every day of the month (), every month (), every day of the week (*), and it runs as a regular user. Saving this crontab file schedules the job.

Suppose you want to be cautious and run the scanner only on weekends at 2 AM when network traffic is less likely to be closely monitored. Additionally, you want it to run only during the summer months, specifically in June, July, and August. In this case, your job would look like this:

```
00 2 * 6-8 0,6 user /usr/share/MySQLscanner.sh
```

You can add this to your crontab, and now your MySQLscanner.sh will run only on weekends in June, July, and August at 2 AM:

```
# etcrcrontab: system-wide crontab
```

```
# Unlike any other crontab, you don't have to run the 'crontab'
```

```
# command to install the new version when you edit this file
```

```
# and files in etcrcron.d. These files also have username fields,
```

```
# which none of the other crontabs do.
```

```
SHELL=binsh
```

```
PATH=usrlocal/sbin:usrlocal/bin:/sbin:/bin:usrsbin:usrbin
```

```
# m h dom mon dow user command
```

```
17 root cd / && run-parts --report etcrcron.hourly
```

```
25 6 * root test -x usr/sbin/anacron || ( cd / && run-parts --report etcrcron.daily )
```

```
47 6 7 root test -x usr/sbin/anacron || ( cd / && run-parts --report etcrcron.weekly )
```

```
52 6 1 root test -x usr/sbin/anacron || ( cd / && run-parts --report etcrcron.monthly )
```

```
00 2 * 6-8 0,6 user usrshare/MySQLsscanner.sh
```

Efficient crontab Shortcuts

The crontab file provides built-in shortcuts that you can use instead of specifying the time, day, and month individually. These include:

- @yearly
- @annually
- @monthly
- @weekly
- @daily
- @midnight
- @noon
- @reboot

For example, if you want the MySQL scanner to run every night at midnight, you can add the following line to the crontab file:

```
@midnight user /usr/share/MySQLsscanner.sh
```

Using rc Scripts for Startup Jobs

When you start your Linux system, a series of scripts known as rc scripts are executed to set up the environment. These scripts are responsible for initializing services and configurations. After the kernel has loaded all its modules, the init or init.d daemon starts and begins running scripts found in /etc/init.d/rc. These scripts include commands for starting essential services required for your Linux system to operate as expected.

Understanding Linux Runlevels

Linux operates with multiple runlevels, each specifying which services should start at bootup. For instance, runlevel 1 represents single-user mode, where essential services like networking remain inactive. The execution of rc scripts depends on the chosen runlevel:

- 0: Halts the system
- 1: Single-user/minimal mode
- 2–5: Multiuser modes
- 6: Reboots the system

Integrating Services into rc.d

To add services to the rc.d script for startup, you can utilize the `update-rc.d` command. This command allows you to add or remove services from the rc.d script. The syntax is straightforward; you provide the command, followed by the script or service name, and specify the action to perform, as demonstrated below:

```
kali > update-rc.d <script or service name> <remove|defaults|disable|enable>
```

As an illustrative example, let's say you want the PostgreSQL database to start automatically when your system boots. This enables your Metasploit framework to utilize it for storing pentesting and hacking results. You can achieve this by using `update-rc.d` to add a line to your rc.d script, ensuring PostgreSQL is up and running each time you boot your system.

Before proceeding, let's check if PostgreSQL is currently running on your system. You can use the `ps` command and filter for PostgreSQL using `grep`, like so:

```
kali > ps aux | grep postgresql
```

If you receive output similar to what's shown below, it indicates that PostgreSQL is not running:

```
root 3885 0.0 0.0 12720 964pts/1 S+ 14.24 0.00 grep postgresql
```

Now, let's update our rc.d script to ensure PostgreSQL starts automatically at boot:

```
kali > update-rc.d postgresql defaults
```

This command adds the necessary line to the rc.d file. To activate the change, you need to reboot your system. After doing so, use the `ps` command with `grep` again to check if PostgreSQL is running:

```
kali > ps aux | grep postgresql
```

```
postgresql 757 0.0 0.1 287636 25180 ? S March 14
```

```
0.00 usr/lib/postgresql/9.6/bin/postgresql -D
```

```
var/lib/postgresql/9.6/main
```

```
-c config_file=/etc/postgresql/9.6/main/postgresql.conf
```

```
root 3876 0.0 0.0 12720 964pts/1 S+ 14.24 0.00 grep postgresql
```

You should observe output indicating that PostgreSQL is running without manual intervention, automatically initializing when your system boots, and ready for use with your Metasploit framework.

GUI Methods for Bootup Service Addition

If you prefer a graphical user interface (GUI) for adding services at startup, you can download the simple GUI tool `rcconf` from the Kali repository. Here's how to do it:

```
kali > apt-get install rcconf
```

Once the installation is complete, start `rcconf` by entering:

```
kali > rcconf
```

This will open a straightforward GUI similar to the one depicted in Image 16-1. You can browse through the available services, select the ones you want to launch at boot, and click "OK."

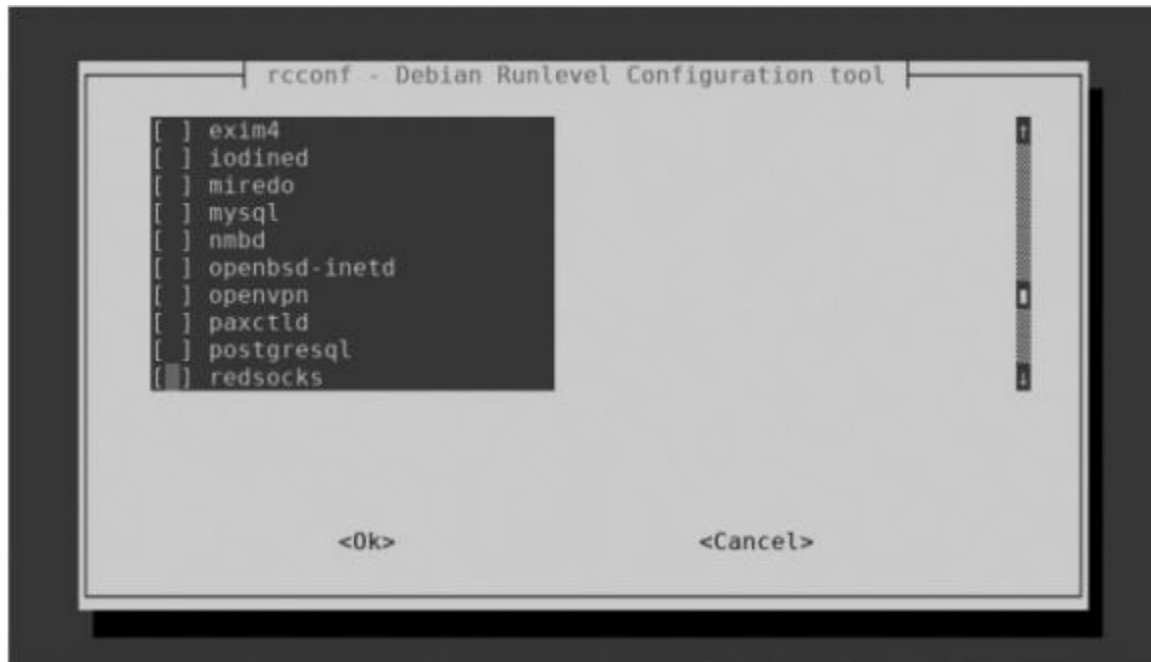


Image 16-1: The rcconf GUI for adding services to startup

In the figure, PostgreSQL is listed second from last. To select this service, press the spacebar, then press TAB to highlight `<Ok>`, and finally, press ENTER. The next time you boot Kali, PostgreSQL will initiate automatically.

Chapter Summary

Both system administrators and hackers frequently require scheduling services, scripts, and utilities to run at regular intervals. Linux offers the flexibility to schedule almost any script or utility for routine execution using the cron daemon, which operates jobs from the cron table. Additionally, you can ensure services start automatically at boot by utilizing the `update-rc.d` command or the GUI-based tool `rcconf` to update the `rc.d` scripts.

Practice Exercises

Now you can practice the skills learned in this chapter with the following exercises:

- Schedule your `MySQLscanner.sh` script to run every Wednesday at 3 PM.
- Schedule your `MySQLscanner.sh` script to run on the 10th day of the month in April, June, and August.

Schedule your MySQLscanner.sh script to run from Tuesday through Thursday at 10 AM.

Schedule your MySQLscanner.sh script to run daily at noon using shortcuts.

Update your rc.d script to launch PostgreSQL automatically during system boot.

Download and install `rcconf`, then configure PostgreSQL and MySQL databases to start at bootup.

Conclusion

As we conclude Command Line Hacker, take a moment to reflect on your journey from a beginner to a fully equipped Linux professional. This book has prepared you for the Linux+ certification and the Kali Linux Certified Professional (KLCP) exam by covering essential topics such as Linux fundamentals, Bash scripting, network management, and cybersecurity.

With these skills, you are now equipped to pursue a role as a Junior Linux System Administrator. And I hope you gain real-world experience and eventually transition into a penetration tester role.

Thank you for choosing Command Line Hacker. I sincerely hope it has been a valuable resource for you. And until we meet again!